

版本信息:

版本 REV2024

时间 11/11/2022

米联客 2024 版 FPGA 程序设计 Verilog 语法入门篇

Milianke Anlogic FPGA Course

微信扫码注册米联客技术论坛www.uisrc.com免费享受更多资源



扫码关注微信公众平台“米联客(MILIANKE)”掌握更多信息动态



技术支持

客户可以通过以下链接获取米联客官方店铺的客服联系方式，加入米联客企业微信技术支持群，获得专业的技术支持和解答。感谢您的支持！

京东米联客旗舰店：<https://milianke.jd.com>

天猫米联客旗舰店：<https://milianke.tmall.com>

淘宝米联客生态店：<https://milianke.taobao.com>

售后

1: 7 天无理由退货(人为原因除外)

2: 质保期限: 本产品自快递签收之日起, 提供一年质保服务(主芯片, 比如 FPGA 或者 CPU 等除外)。

3: 维修换货, 需提供淘宝订单编号或合同编号, 联系销售/技术支持安排退回事宜。

售后维修请登录工单系统: https://www.uisrc.com/plugin.php?id=x7ree_service

4: 以下情形不属于质保范畴

A: 由于用户使用不当造成板子的损坏: 比如电压过高造成的开发板短路, 自行焊接造成的焊盘脱落、铜线起皮等。

B: 用户日常维护不当造成板子的损坏: 比如放置不当导致线路板腐蚀、基板出现裂纹等

5: 质保范畴外(上方第 4 条)及质保期限以外的产品, 本司提供有偿维修服务。维修仅收取器件材料成本, 往返运费全部由客户承担。

6: 寄回地址, 登录网页获取最新的售后地址: <https://www.uisrc.com/t-1982.html>

常州米联客信息科技有限公司

版权所有

常州溧阳总部: 常州溧阳市中关村吴潭渡路雅创高科制造谷 10-1 幢楼

南京研发基地: 南京市江宁区秣陵街道吉印大道 3118 号联东 U 谷·南京九龙湖智造创新港 3 号楼 9 楼

版本	时间	描述
Rev2022	2022-12-12	第一版

写给即将开启 FPGA 学习之旅的用户

FPGA 编程又称为硬件编程语言和流行的各类软件编程语言 C/C++、JAVA、python 等相比,掌握基础的硬件编程语言不是难事,难点是 FPGA 在每个专业领域的应用,只有充分理解了 FPGA,并且具有对自己所处行业专业背景认知,才能真正理解 FPGA 应该用在什么场合更加合适。

很多人问我学习 FPGA 是否有前途,这个问题着实难以回答。我们可以一起来探讨以下几种情况,会是 FPGA 发挥作用的场合:

1)、数字 IC 设计工作

数字 IC 设计还是主要以硬件编程语言去设计数字 IC 芯片,通过硬件编程语言,软件可以把语言翻译成电路,自动布线工具可以完成布局布线,之后流片。由于流片费用非常贵,前期也可以用 FPGA 芯片模拟设计的数字 IC 芯片的功能。

2)、数字信号处理

高速的 ADC,DAC 的数模信号处理的领域也是必然需要用到 FPGA,在无线通信、雷达信号处理等领域也都会用到。

3)、数字信号高速通信

FPGA 具备的高速接口也非常适合用于高速通信,比如 PCIE 通信、光通信、以太网通信

4)、视频图像

包括图像的拼接、缩放、高效的实时传输等, 4K 视频、8K 视频领域、OSD 菜单、高效的 FPGA 图像处理算法

5)、硬件加速算法

FPGA 硬件加速领域应用目前是热点,已经有很多公司利用 FPGA 的硬件加速实现了很好的经济效益。

6)、神经网络

利用 FPGA 的高效实时性实现的神经网络算法通常有比传统方法更高的实时性、更低的功耗、高性价比。难点是初期开发工作量较大。

7)、通用 CPU GPU 无法完成的工作

如果通用的 CPU 和 GPU 无法完成的工作任务,可以考虑下 FPGA 或者带 FPGA 的 SOC。

不能为了专门学习 FPGA 而学习 FPGA, FPGA 只是一个工具。如果不知道如何应用 FPGA,那么这是非常悲哀的一件事情,学习 FPGA 只是学习一门技能,而结合自己专业背景选择最合适的解决方案,解决问题才是最终的目的。

本章节课程介绍安路 TD 软件安装、JTAG 驱动安装、JTAG 下载程序、创建 FPGA 工程、添加代码、添加 IP、约束管脚、添加时序约束、使用在线逻辑分析仪、编译、下载、固化程序到 FLASH。同时也介绍了第三方 vscode 代码编辑软件的安装和 TD 调用 vscode 编辑代码的方法。

本章节课程适合初次使用 TD 软件的用户快速熟悉和掌握 TD 软件使用。

目录

米联客 2024 版 FPGA 程序设计 Verilog 语法入门篇	1
写给即将开启 FPGA 学习之旅的用户	3
第一课 Verilog 语法_Verilog_HDL 语言简介	9
1 概述	9
2 Verilog HDL 简介	9
3 Verilog HDL 发展历史及现状	9
4 主要应用场景	10
4.1 可编程逻辑器件	10
4.2 半定制或全定制 ASIC	10
4.3 混合 ASIC	10
第二课 Verilog 语法_环境搭建	11
1 概述	11
2 TD 软件下载	12
3 TD 软件安装	12
4 安装 license	14
5 TD 和 Modelsim 联合仿真	16
第三课 Verilog 语法_基本设计方法	20
1 概述	20
2 基本设计方法	20
3 设计流程	20
3.1 需求整理	21
3.2 功能划分	21
3.3 Verilog 设计	22
3.4 仿真验证	22
3.5 逻辑综合	22
3.6 布局布线	22
3.7 时序仿真	22
3.8 位流生成和调试	22
第四课 Verilog 语法_基础语法	23
1 概述	23
2 语法简介	23
2.1 格式	23
2.2 注释	23
2.3 关键字与标识符	24
2.4 数值种类及表示方法	26
2.5 字符串	26
第五课 Verilog 语法_数据类型及表达式	27

1 概述	27
2 数据类型	27
2.1 wire 线网	27
2.2 reg 寄存器	28
2.3 integer 整数	28
2.4 real 实数	28
2.5 time 时间	29
2.6 数组	29
2.7 字符串	29
2.8 常量	29
3 表达式	30
3.1 操作数	30
3.2 操作符	30
第六课 Verilog 语法_过程结构与赋值	33
1 概述	33
2 过程结构	33
2.1 initial 语句	33
2.2 always 语句	34
3 过程赋值	34
3.1 阻塞赋值	34
3.2 非阻塞赋值	34
3.3 阻塞与非阻塞赋值的使用注意	35
第七课 Verilog 语法_时序控制与语句块	37
1 概述	37
2 时序控制简介	37
2.1 时延控制	37
2.2 事件控制	38
2.2.1 边沿触发控制	38
2.2.2 电平敏感控制	40
3 语句块介绍	41
3.1 顺序块	41
3.2 并行块	41
3.3 嵌套块	41
3.4 命名块	42
第八课 Verilog 语法_条件与循环语句	43
1 概述	43
2 条件语句	43

2.1 if.....	43
2.2 case.....	45
3 循环语句.....	46
3.1 forever 语句.....	46
3.2 repeat 语句.....	47
3.3 while 语句.....	47
3.4 for 语句.....	48
第九课 Verilog 语法_模块端口与例化.....	49
1 概述.....	49
2 模块端口.....	49
3 模块例化.....	50
第十课 Verilog 语法_竞争与冒险.....	53
1 概述.....	53
2 竞争与冒险.....	53
2.1 什么是竞争与冒险.....	53
2.2 如何消除竞争冒险.....	54
第十一课 Verilog 语法_一般设计规范.....	55
1 概述.....	55
2 设计规范.....	55
2.1 时序逻辑电路中, 对寄存器用非阻塞赋值。.....	55
2.2 组合逻辑电路中, 对寄存器用阻塞赋值。.....	56
2.3 同一个 always 块中建立时序和组合逻辑模型时, 用非阻塞赋值。.....	56
2.4 同一个 always 块中不要既使用阻塞赋值又使用非阻塞赋值。.....	56
2.5 不允许在多个 always 块中为同一个变量赋值.....	57
2.6 设计避免产生 latch.....	57
第十二课 Verilog 语法_函数与任务.....	59
1 概述.....	59
2 函数与任务.....	59
2.1 task 语句.....	59
2.2 function 语句.....	60
第十三课 Verilog 语法_仿真文件设计.....	62
1 概述.....	62
2 仿真文件设计.....	62
2.1 信号声明.....	69
2.2 时钟生成.....	69
2.3 复位生成.....	70
2.4 激励部分.....	70

2.5 模块例化	70
2.6 打印信息	70
第十四课 Verilog 语法_流水线设计	71
1 概述	71
2 流水线简介	71
2.1 什么是流水线	71
2.2 设计流水线	72
第十五课 Verilog 语法_同步与异步设计	75
1 概述	75
2 同步时钟	75
3 异步时钟	76
第十六课 Verilog 语法_跨时钟域设计	78
1 概述	78
2 跨时钟域慢速到快速时钟	78
2.1 单比特信号	78
2.2 多比特信号	78
3 跨时钟域快速到慢速时钟	80
3.1 单比特信号	80
3.2 多比特信号	82
第十七课 Verilog 语法_复位设计	83
1 概述	83
2 复位电路简介	83
2.1 同步复位	83
2.2 异步复位	84
2.3 异步复位同步化	84
2.4 异步复位同步释放	85
第十八课 Verilog 语法_时钟分频设计	87
1 概述	87
2 时钟分频	87
2.1 偶数分频	87
2.2 奇数分频	88
2.3 小数分频	90
第十九课 Verilog 语法_FIFO 设计	93
1 概述	93
2 同步 FIFO	93
3 异步 FIFO	98
第二十课 Verilog 语法_低功耗设计	109

1 概述	109
2 低功耗简介	109
3 常用的低功耗设计技术	109
4 Verilog 中使用的低功耗设计方法	110
4.1 并行设计	110
4.2 流水线设计	110
4.3 资源共用	111
4.4 状态编码	111
4.5 门控时钟	111

第一课 Verilog 语法_Verilog_HDL 语言简介

软件版本：无

操作系统：WIN10 64bit

硬件平台：适用所有系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑！

1 概述

本节主要介绍 Verilog HDL 的特点，发展历史及现状，主要应用场景。

2 Verilog HDL 简介

Verilog HDL 是一种硬件描述语言，用于从算法级、门级电路到开关级电路的多种抽象设计层次的数字系统建模。被建模的数字系统的复杂性可以介于简单的门和完整的电子数字系统之间。数字系统能够按层次描述，并可在相同描述中显式地进行时序建模。

Verilog HDL 语言具有下述描述能力：设计的行为特性、设计的数据流特性、设计的结构组成以及包含响应监控和设计验证方面的时延和波形产生机制。所有这些都使用同一种建模语言。此外，Verilog HDL 语言提供了编程语言接口，通过该接口可以在模拟、验证期间从设计外部访问设计，包括模拟的具体控制和运行。

Verilog HDL 语言不仅定义了语法，而且对每个语法结构都定义了清晰的模拟、仿真语义。因此，用这种语言编写的模型能够使用 Verilog 仿真器进行验证。Verilog HDL 语言从 C 编程语言中继承了多种操作符和结构。Verilog HDL 提供了扩展的建模能力，其中许多扩展最初很难理解。但是，Verilog HDL 语言的核心子集非常易于学习和使用，这对大多数建模应用来说已经足够。当然，完整的硬件描述语言足以对从最复杂的芯片到完整的电子系统进行描述。

3 Verilog HDL 发展历史及现状

Verilog 是由 Gateway 设计自动化公司的工程师于 1983 年末创立的。当时 Gateway 设计自动化公司还叫做自动集成设计系统 (Automated Integrated Design Systems)，1985 年公司将名字改成了前者。该公司的菲尔·莫比 (Phil Moorby) 完成了 Verilog 的主要设计工作。1990 年，Gateway 设计自动化被 Cadence 公司收购。

1990 年代初，开放 Verilog 国际 (Open Verilog International, OVI) 组织 (即现在的 Accellera) 成立，Verilog 面向公有领域开放。1992 年，该组织寻求将 Verilog 纳入电气电子工程师学会标准。最终，Verilog 成为了电气电子工程师学会 1364-1995 标准，即通常所说的 Verilog-95。

设计人员在使用这个版本的 Verilog 的过程中发现了一些可改进之处。为了解决用户在使用此版本 Verilog 过程中反映的问题，Verilog 进行了修正和扩展，这部分内容后来再次被提交给电气电子工程师学会。这个扩展后的版本后来成为了电气电子工程师学会 1364-2001 标准，即通常所说的 Verilog-2001。Verilog-2001 是对 Verilog-95 的一个重大改进版本，它具备一些新的实用功能，例如敏感列表、多维数组、生成语句块、命名端口连接等。目前，Verilog-2001 是 Verilog 的最主流版本，被大多数商业电子设计自动化软件包支持。

2005 年，Verilog 再次进行了更新，即电气电子工程师学会 1364-2005 标准。该版本只是对上一版本的细微修正。

这个版本还包括了一个相对独立的新部分，即 Verilog-AMS。这个扩展使得传统的 Verilog 可以对集成的模拟和混合信号系统进行建模。容易与电气电子工程师学会 1364-2005 标准混淆的是加强硬件验证语言特性的 SystemVerilog(电气电子工程师学会 1800-2005 标准)，它是 Verilog-2005 的一个超集，它是硬件描述语言、硬件验证语言（针对验证的需求，特别加强了面向对象特性）的一个集成。

2009 年，IEEE 1364-2005 和 IEEE 1800-2005 两个部分合并为 IEEE 1800-2009，成为了一个新的、统一的 SystemVerilog 硬件描述验证语言（hardware description and verification language, HDVL）。

4 主要应用场景

Verilog HDL 用于设计专用集成电路(ASIC)，ASIC 就是具有专门用途和特殊功能的独立集成电路器件。Verilog HDL 作为硬件描述语言，主要用来生成专用集成电路。同时，可以作为 CPLD、FPGA 等可编程器件的程序开发语言。

4.1 可编程逻辑器件

FPGA 和 CPLD 是实现这一途径的主流器件。直接面向用户级，具有极大的灵活性和通用性，实现快捷开发，测试简单方便，开发效率高而成本较低。

4.2 半定制或全定制 ASIC

通俗来讲，就是利用 Verilog HDL 来设计具有某种特殊功能的专用芯片。根据基本单元工艺的差异，又可分为门阵列 ASIC，标准单元 ASIC 和全定制 ASIC。

4.3 混合 ASIC

主要指既具有面向用户的 FPGA 可编程逻辑功能和逻辑资源，同时也含有可方便调用和配置的硬件标准单元模块，如 CPU，RAM，锁相环(PLL)，乘法器(DSP)等。

第二课 Verilog 语法_环境搭建

软件版本：无

操作系统：WIN10 64bit

硬件平台：适用所有系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑！

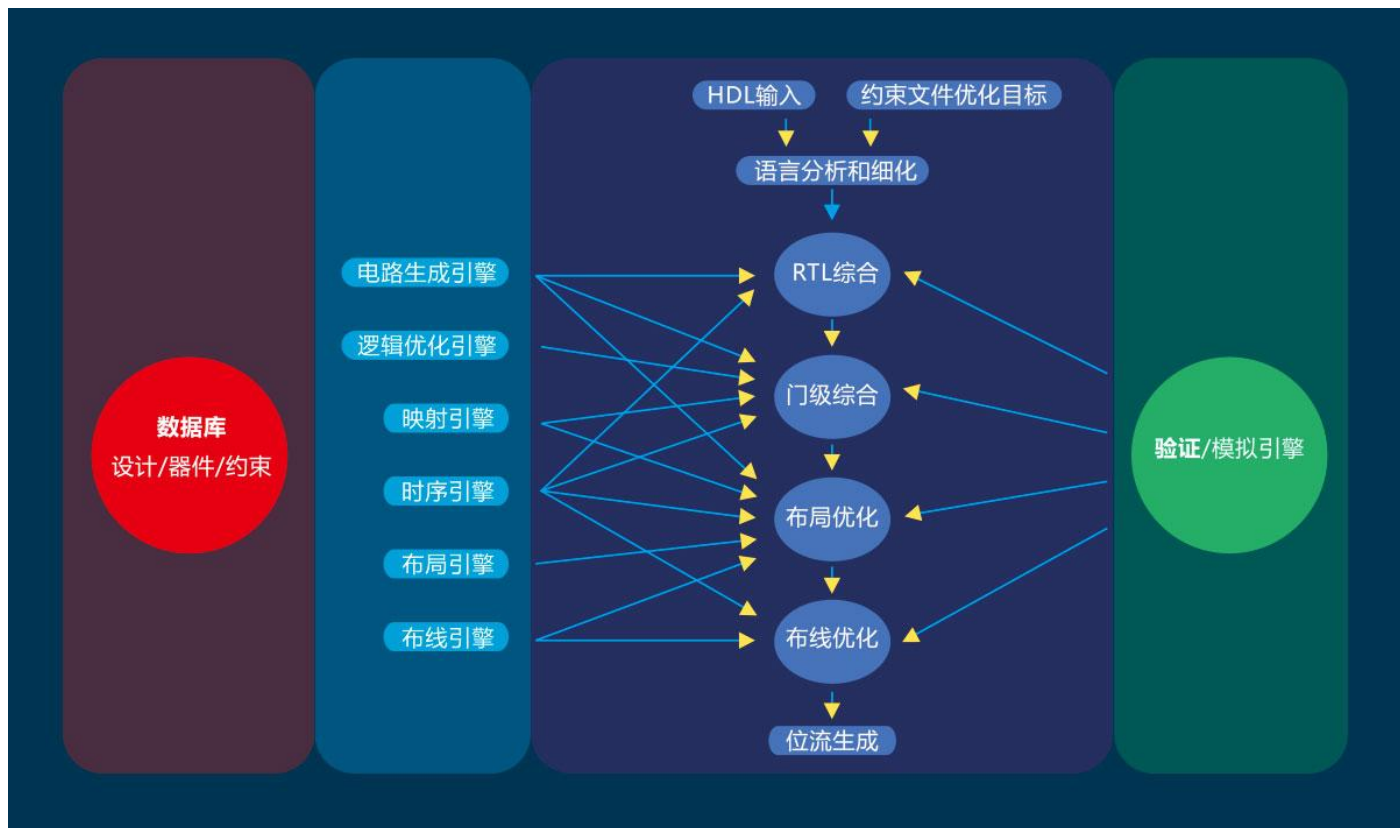
在此文档中，重点介绍安路 TangDynasty（简称 TD）软件与 Modelsim 的联合仿真方法，适用于 64 位 Windows 10 操作系统环境。这种联合仿真方法为用户提供了一种高效的设计验证解决方案，其实现流程包括：安装与配置开发环境、建立项目、引入设计文件、配置仿真参数、运行仿真以及分析仿真结果等步骤。

1 概述

上海安路信息科技股份有限公司成立于 2011 年，总部位于浦东新区张江科学城。安路科技专注于 FPGA 芯片设计领域，通过多年的技术积累，公司在 FPGA 芯片设计技术、SoC 系统集成技术、FPGA 专用 EDA 软件技术、FPGA 芯片测试技术和 FPGA 应用解决方案等领域均有技术突破。

安路 FPGA 目前已经形成了以 SALPHOENIX®高性能产品系列、SALEAGLE®高性价比产品系列和 SALELF®低功耗产品系列组成的产品矩阵。产品广泛应用于工业控制、网络通信、消费电子、数据中心等领域。

TangDynasty®(TD)软件是安路科技自主开发的 FPGA 集成开发环境，支持工业界标准的设计输入，包含完整的电路优化流程以及丰富的分析与调试工具，并提供良好的第三方设计验证工具接口，为所有基于安路科技 FPGA 产品的应用设计提供有力支持。



2 TD 软件下载

登录安路官方：<http://www.anlogic.com/>，下载 TD 版本，本文以 TD 某个版本安装为例，用户需根据自己需要的器件选择对应的版本进行安装。

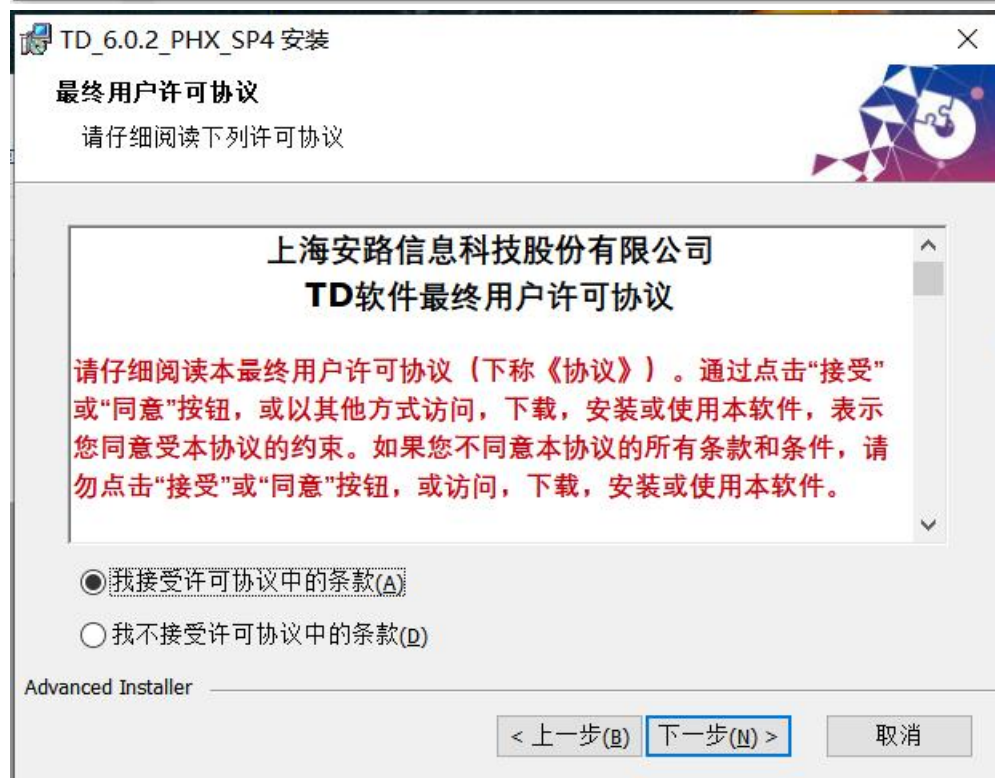
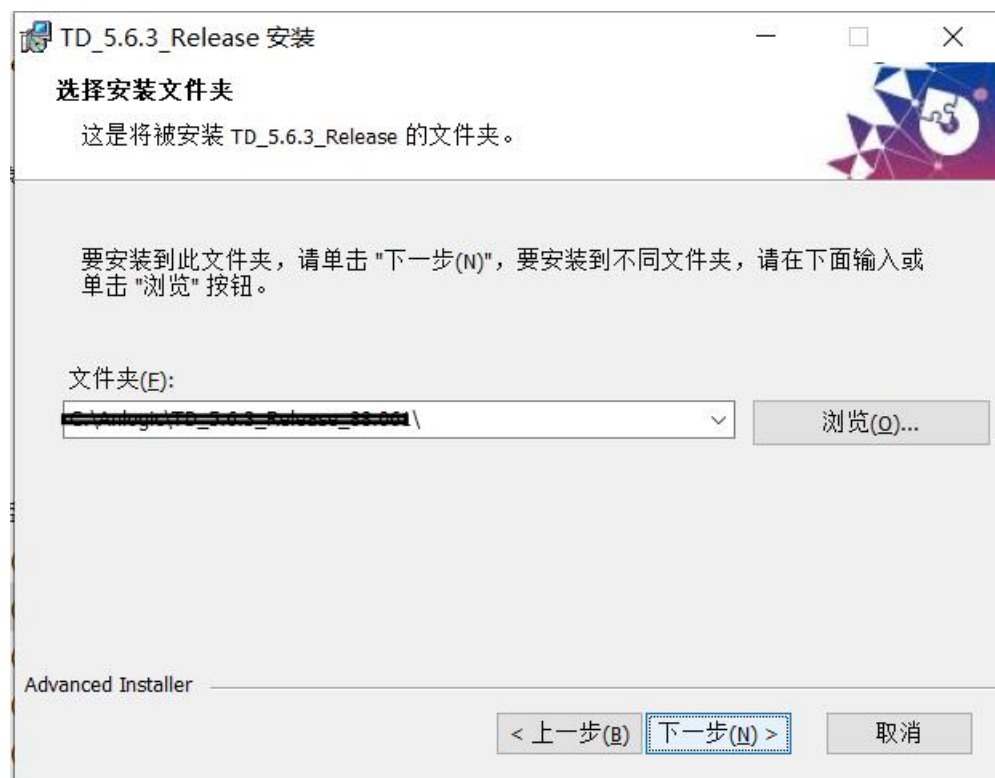
技术文档	+	标题 ▾	日期 ▾	版本	支持系列	操作
开发板设计	+	TD_6.0.2_PHX_SP4_135568_NL.msi	2024-11-26	TD_6.0.2		↓
		压缩码: [SHA-256] 8d9e22a13f463d7f78de6d03f18f8633703489103edfe287c80ba25a1fd1a10e				
软件工具	-	TD_6.0.2_PHX_SP3_132435_NL.msi	2024-10-16	TD_6.0.2		↓
		压缩码: [SHA-256] 1d4f66fe97d0ca5f656953c2d7178c70aadeceb1f39407875c189998904cc2420				
TD Windows	^	TD_6.0.2_PHX_SP2_129875_NL.msi	2024-10-03	TD_6.0.2		↓
		压缩码: [SHA-256] 41dcd53d55f54e4da56d5d058480b3b2348cba1feff4663d578f2630278d45fd				
TD_4.6		TD_6.0.2_PHX_SP1_122666_NL.msi	2024-08-12	TD_6.0.2		↓
TD_5.0		压缩码: [SHA-256] 94cbbdfe27cd6872741797a84cea435ae7bfa28911e65fe9394e29da33fc2a7b				
TD_5.5		TD_6.0.2_PHX_Release_117864_NL.msi	2024-07-05	TD_6.0.2		↓
TD_5.6		压缩码: [SHA-256] e92f0b2abc50cb0b903a0775f6d5c4f39d1f83f630ae7f8ef299c8c1c84fb0aa				
TD_6.0						
TD_5.9						

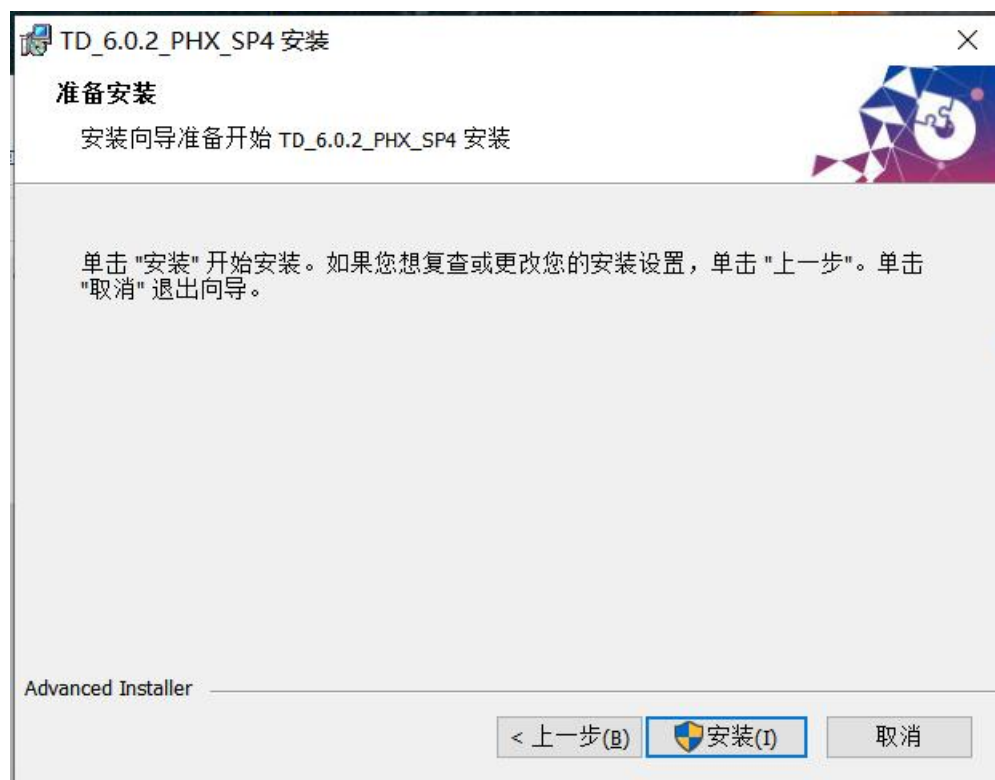
3 TD 软件安装

双击运行

电脑 > 新加卷 (E:) > TD > TD6.0.2_SP4 >				在 TD6.0.2_SP
名称	修改日期	类型	大小	
TD_6.0.2_PHX_SP4_135568_NL.msi	2024/12/2 15:54	Windows Install...	1,632,363...	





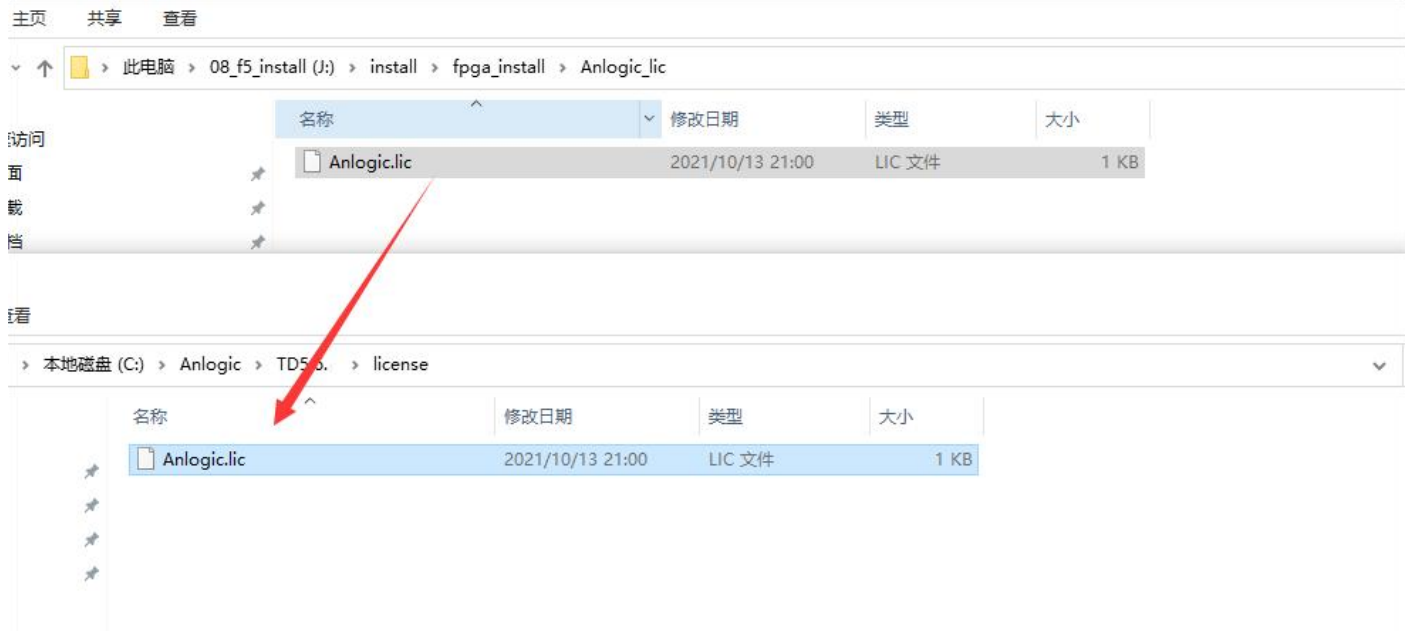


4 安装 license

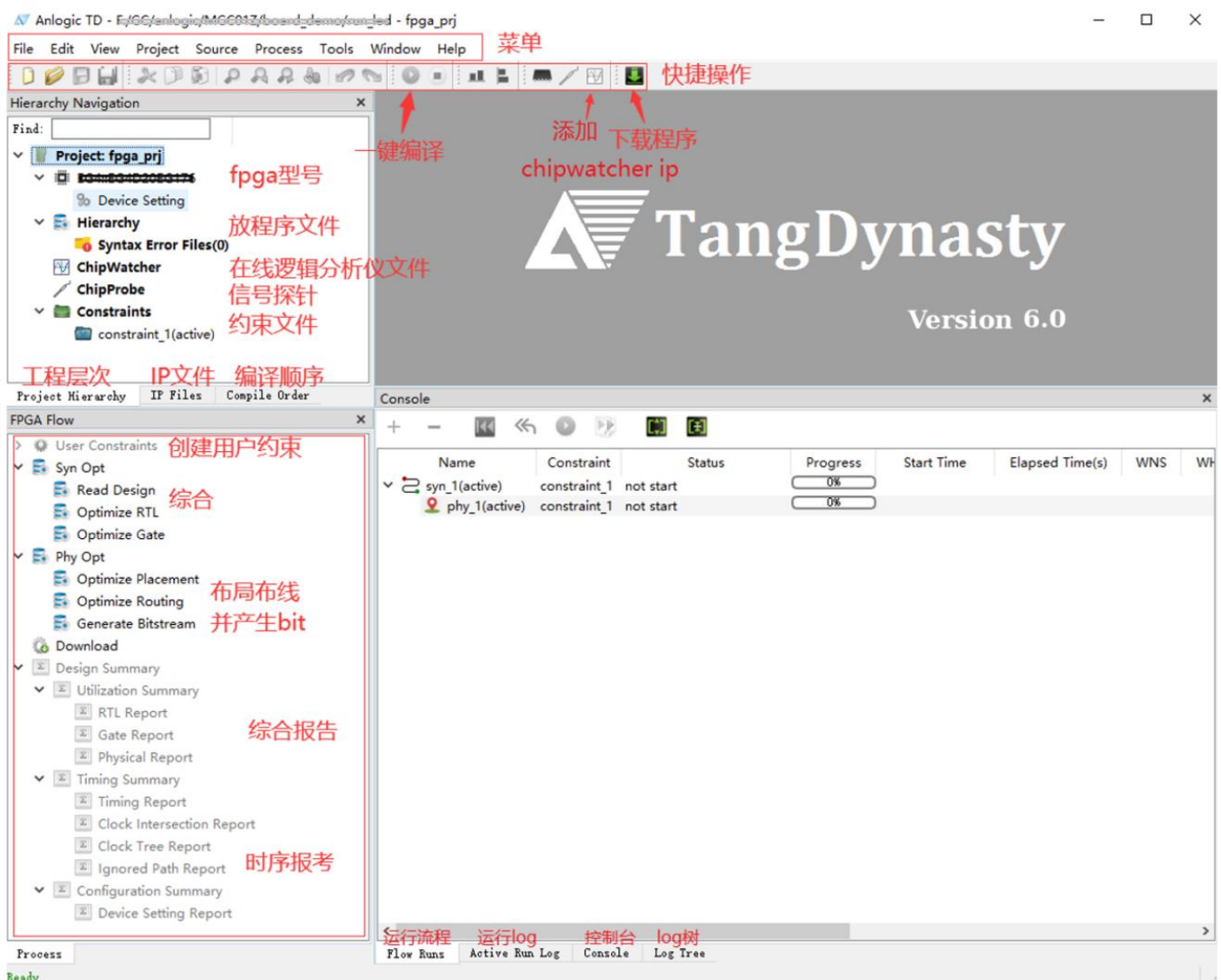
still (1) > install > fpga_install > Anlogic_lic

名称	修改日期	类型	大小
 Anlogic.lic	2021/10/13 21:00	LIC 文件	1 KB

复制 Anlogic.lic 到路径



以下是对软件工具的大概介绍，更多可以结合具体应用加以理解：



具体如何使用，可以看我们“米联客-安路 FPGA TD 软件快速入门课程”，本文不再详细介绍。

5 TD 和 Modelsim 联合仿真

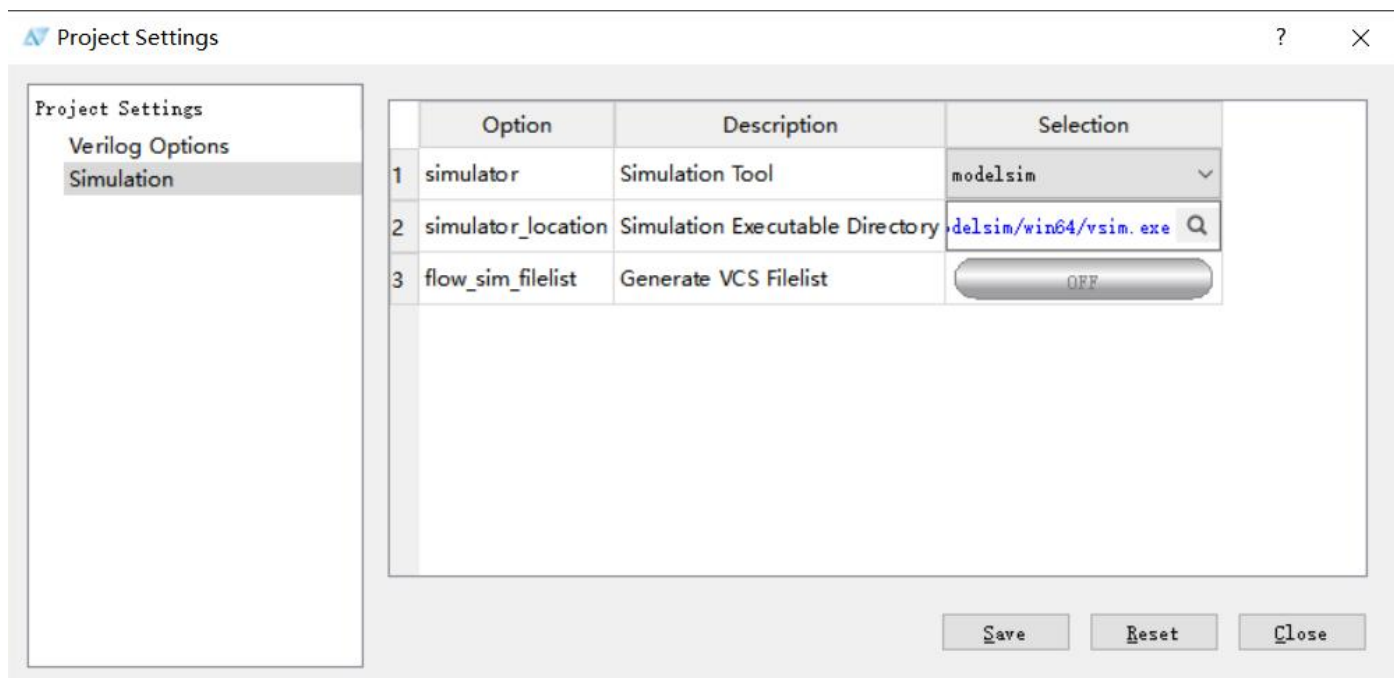
下载地址: https://pan.baidu.com/s/1tw91nD92egyWQoZ_IlatdQ?pwd=jbsg

Modelsim 选择 modelsim-win64-10.6d-se 版本。也需要修改下安装路径, 然后按照默认设置进行操作即可。
安装完毕后可能提示需要重启电脑, 重启即可。

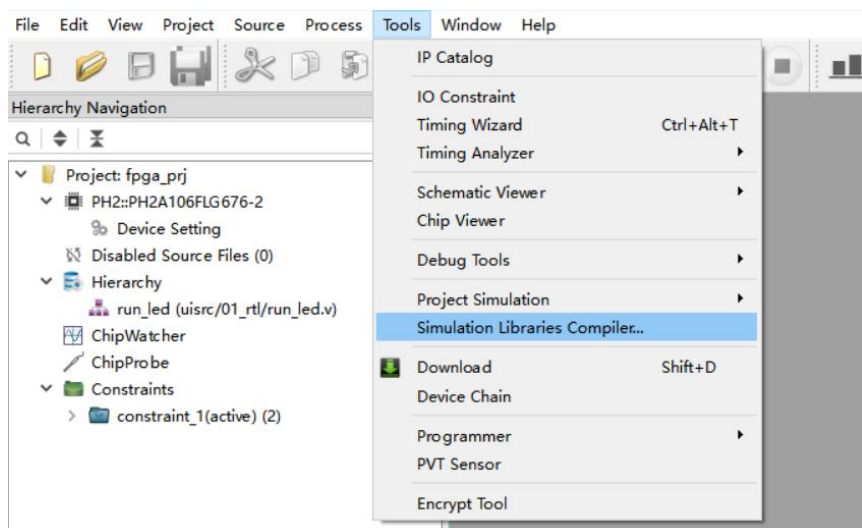
TD 软件集成 Simulation Libraries Compiler 以及 Project Simulation 两个联合仿真工具以协助 用户实现设计流程中各个阶段的工程仿真, 前者的功能是编译 TD 器件库以支持第三方仿真工具调用, 后者的功能是在 TD 各个 flow 阶段仿真时自动生成对应的仿真文件并实现一键自动仿真。

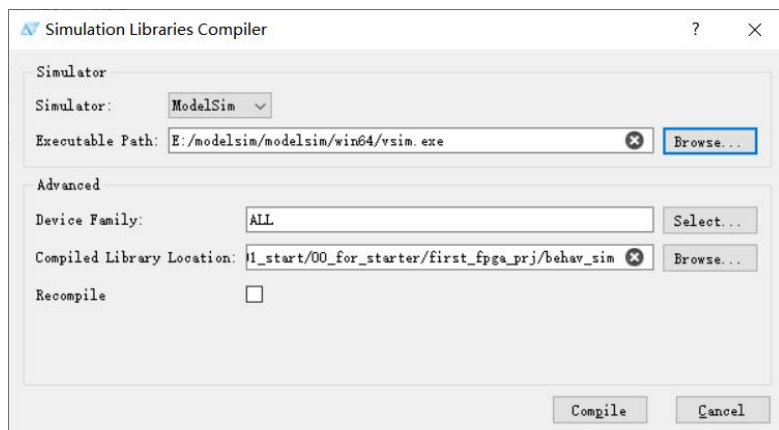
5.1 仿真参数设置

1.选择 Project→Project Setting 打开 Properties Setting 窗口, 点击 Simulation 进行仿真相关的设置, 设置与当前创建的工程相关联。



2. TD 软件中集成有 Simulation Libraries Compiler 工具, 可实现对安路器件库/IP 仿真库的一键编译。选择 Tools → Simulation Libraries Compiler 即可。





(1) Simulator: 选择仿真工具, 目前支持 Modelsim 以及 Questasim;

(2) Executable Path: 选择仿真工具的可执行文件路径, Windows 系统的 TD 版本支持在默认状态下自动识别目标仿真工具的可执行路径, 若目标仿真工具未添加到 PATH 环境变量或者需要覆盖环境变量中的路径时需要设置;

(3) Device Family: 选择编译的器件库, 可选择 common 库、EF2、EF3、EF4、EG、PH1、PH2、SF1 以及 all, TD 支持的器件库以界面选项为准, 默认状态下选择编译所有器件库;

(4) Compiled Library Location: 选择保存编译库结果的路径, TD 将在该路径下创建相应的文件夹保存对应器件库的编译结果;

Recompile: 选中该选项后会对已编译过的器件库进行重新编译, 默认状态不会重编译。

点击 compile 后 TD 启动编译器件库。

5.2 仿真文件 do 文件设置

1:运行 HDL2Bit Flow

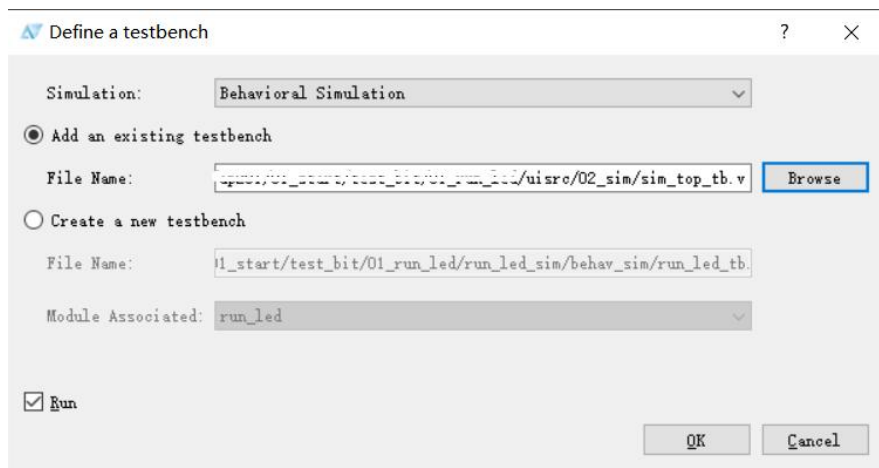
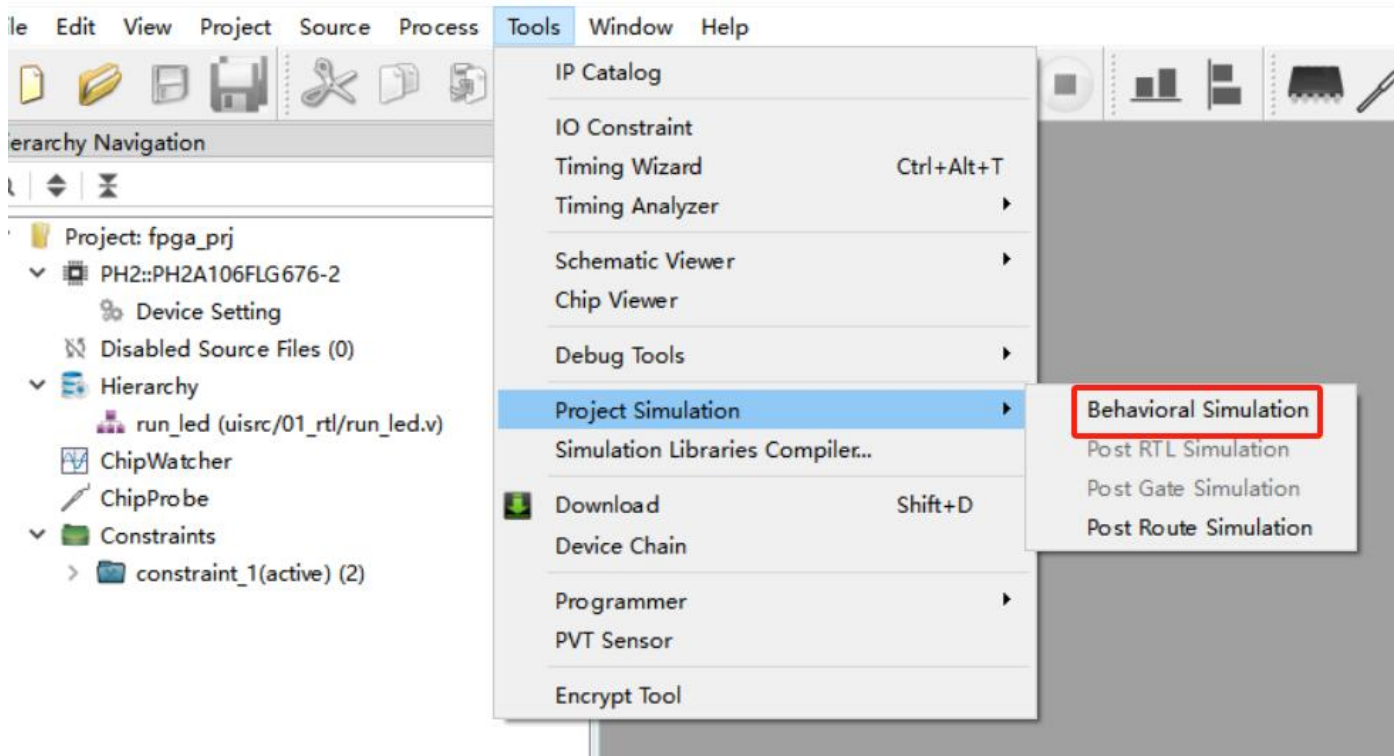
对于向执行什么级别的仿真, 就执行对应的 HDL2Bit Flow

当 HDL2Bit Flow 运行至 Read Design 这一步时, 可执行 Behavioral Simulation;

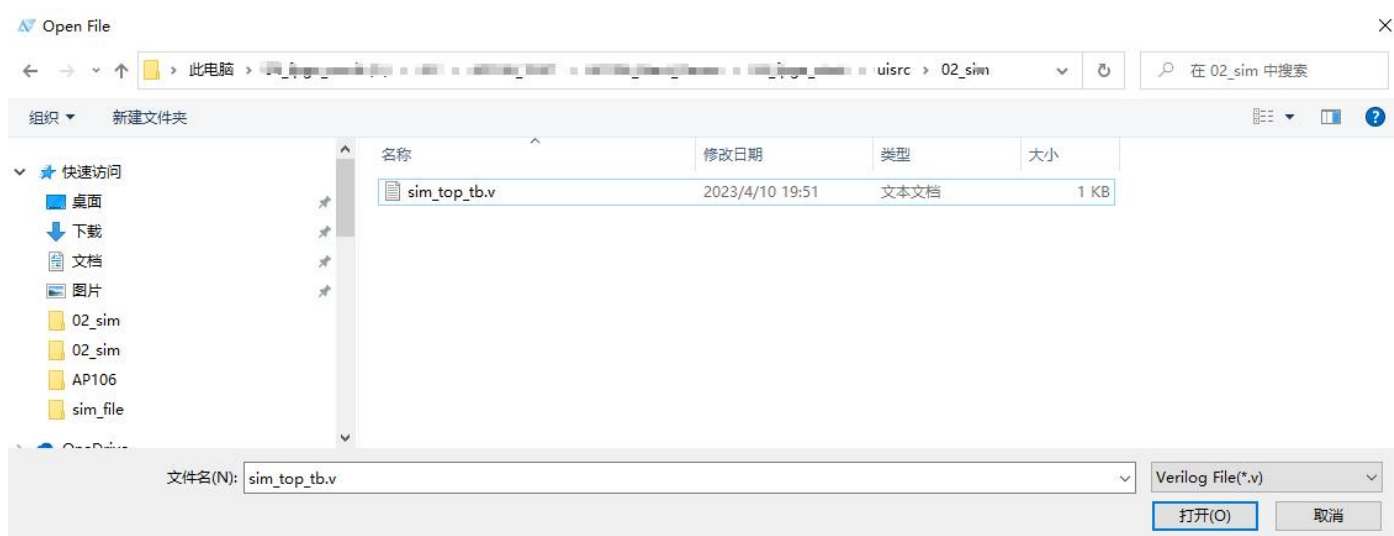
当 HDL2Bit Flow 运行至 Optimize RTL 这一步时, 可执行 Post-RTL Simulation;

当 HDL2Bit Flow 运行至 Optimize Gate 这一步时, 可执行 Post-Gate Simulation;

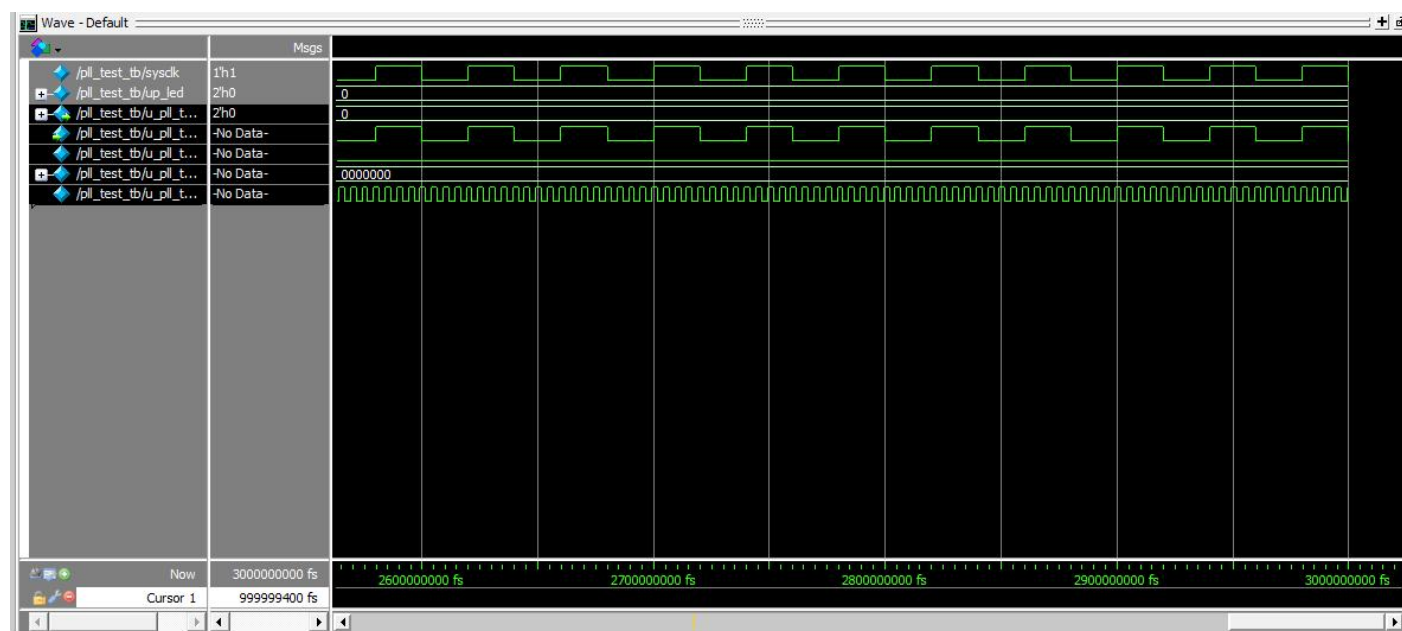
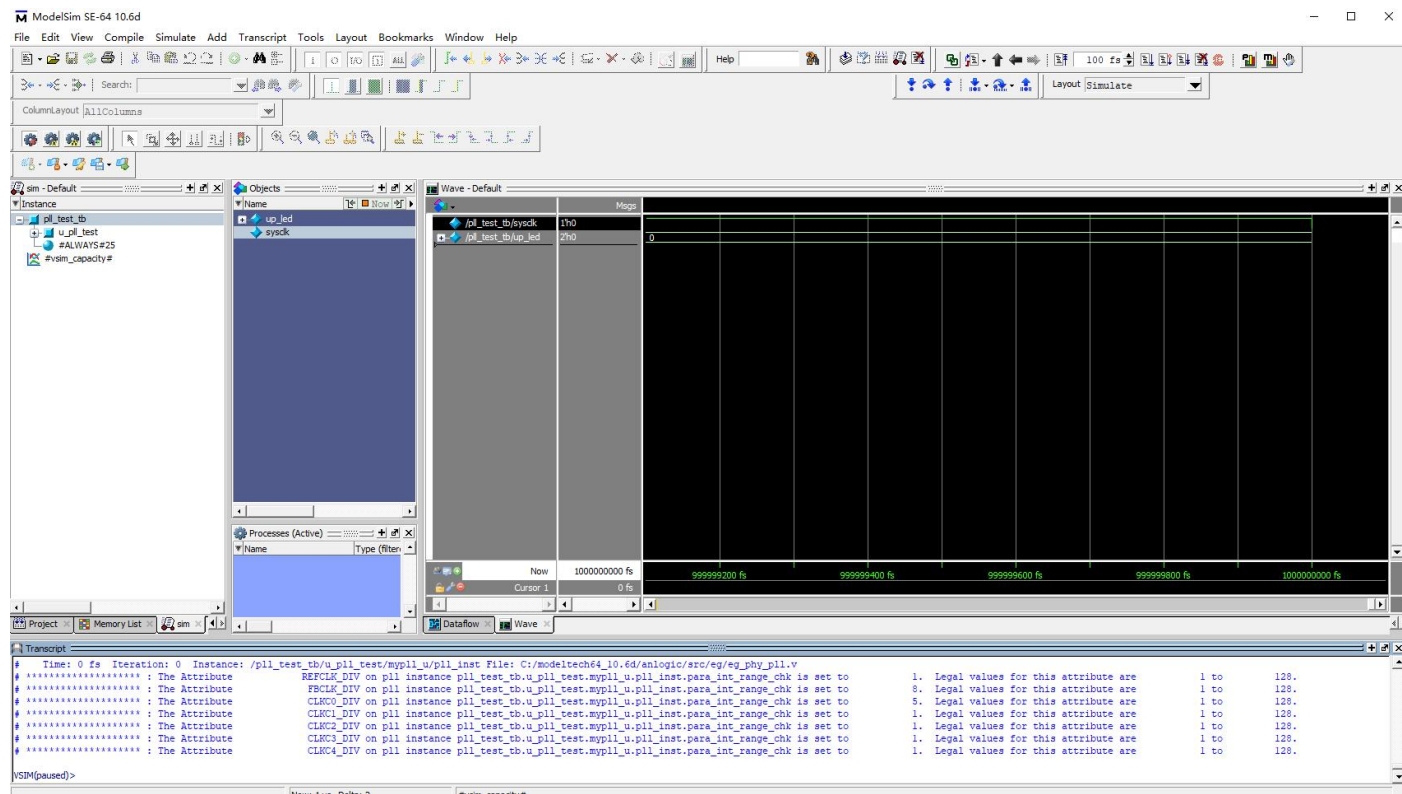
当 HDL2Bit Flow 运行至 Optimize Routing 这一步时, 可执行 Post-Route Simulation。



testbench 文件可以新建一个 tb 或者添加一个已经编写好的 tb 仿真文件，这里我们添加已经编写好的。



点击 OK 之后会自动打开 modelsim 软件



第三课 Verilog 语法_基本设计方法

软件版本：无

操作系统：WIN10 64bit

硬件平台：适用所有系列 FPGA

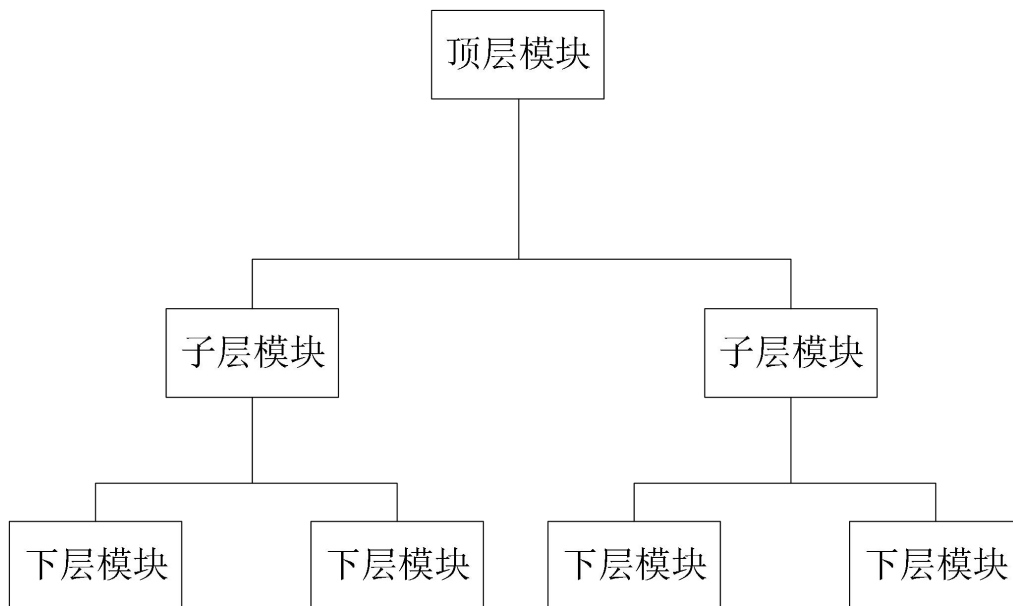
登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑！

1 概述

本节主要讲解 Verilog 的基本设计方法及设计流程。

2 基本设计方法

Verilog 的设计方法有两种，一种采用自上而下的设计方法，另一种采用自下向上的设计方法。自上而下的设计方法先定义顶层模块功能，进而分析要构成顶层模块的必要子模块，然后对各个子模块进行分解、设计，直到到达无法进一步分解的下层模块。这个方法，可以把一个大的系统，分成多个小系统，从时间、工作量上分配给更多的人员进行设计，从而提高了设计效率，缩短了相应的开发时间。

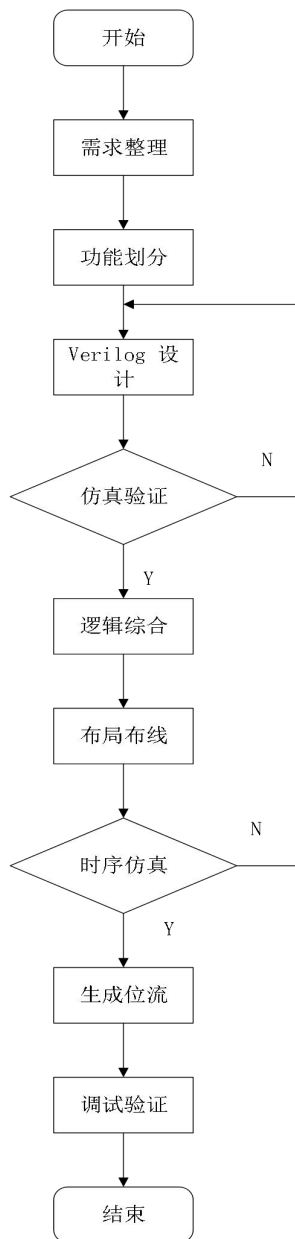


自下向上的设计方法，与自上向下的设计方法思路相反，先从下层模块进行设计，然后仿真与验证。最后联合各级模块组装成一个大的系统。

通常在系统较复杂，子模块较多，需要多人进行联合设计、仿真验证的大系统都是采用自上向下的设计方法。先由系统架构工程师分析系统架构，按照需求划分功能模块，定义模块接口，最后将功能模块分配到具体工程师进行设计。一般当系统规模较小，需要的设计人员较少，设计时一般采用自下向上的设计方法。

3 设计流程

Verilog 的设计流程，通常包括以下几个步骤，如图所示：



下面分小节说明各个流程的具体内容。

3.1 需求整理

设计工程师需要对用户提出的功能要求进行理解分析，规划整个系统的功能模块，整理详细的技术指标，确定初步方案。例如，设计一个以太网通信路由器，需要考虑 FPGA 的时钟工作频率，考虑大致的 FPGA 逻辑资源消耗评估，以太网 PHY 芯片的型号选择，电源芯片的选择，各板级芯片的供电设计，产品的 PCB 走线设计，综合物料成本考虑等等。

3.2 功能划分

分析了用户的功能需求后，接下来就可以进行逻辑功能的设计，设计整个电路的功能、接口和结构，按照自上向下或自下向上的设计方法，进行各个模块的功能设计，设计各子模块的接口和时序，向各工程师进行合理分配子模块设计任务。

3.3 Verilog 设计

可以使用喜欢的文本编辑工具，也可以用专用的 Verilog HDL 编程环境进行设计开发，对所需求的模块进行设计建模。

3.4 仿真验证

对建模文件进行编译，对 RTL 电路进行功能上的仿真验证（前仿真），排除设计的一般错误并进行修正。前仿真验证并没有考虑到具体信号的延迟信息，时序上的问题不能仿真出来，仅验证逻辑上功能的正确性。

3.5 逻辑综合

综合（synthesize），就是在标准单元库和特定的设计约束的基础上，将设计的高层次描述（Verilog 建模）转换为门级网表的过程。逻辑综合的目的是产生物理电路门级结构，并在逻辑、时序上进行一定程度的优化，寻求逻辑、面积、功耗的平衡，增强电路的可测试性。

3.6 布局布线

由综合出的网表与相应的约束文件，利用 FPGA 厂家提供的各种基本标准单元库，对门级电路进行布局布线。到了这一步就将 Verilog 设计的数字电路转换成由 FPGA 厂家标准单元库组成的数字电路。

3.7 时序仿真

布局布线后，电路模型中就有了时延信息。结合在布局布线中获得的精确延迟参数，用专业仿真软件验证电路的时序功能。元器件的不同、布局布线方案不同都会给电路的时序造成不同的影响，严重时会出现时序错误。出错后可能就需要重新修改 RTL，然后重复后面的步骤。这样的过程可能反复多次，直至所有错误完全排除。

3.8 位流生成和调试

完成上面所有步骤后，就可以通过 FPGA 厂家开发工具将工程生成位流文件，然后下载到 FPGA/CPLD 芯片中，最后在电路板上进行调试、验证。

第四课 Verilog 语法_基础语法

软件版本：无

操作系统：WIN10 64bit

硬件平台：适用所有系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑！

1 概述

本节主要讲解基础语法，包含语法格式、注释、标识符、关键字、数值种类与表示和字符串。

2 语法简介

2.1 格式

Verilog 语言是区分字符大小写的，排版格式比较自由，可以在一行内编写，也可跨多行编写。

每个语句必须以分号为结束符。空白符、换行、制表、空格都没有实际的意义，在编译阶段编译器会直接忽略。

例如下面两句编程方式都是等效的。

1、不换行例：

```
always@(*)begin case(s)2'b00:q <= d[0];2'b01:q <= d[1];2'b10:q <= d[2];2'b11:q <= d[3];endcase end
```

2、换行例：

```
always@(*)
begin
    case(s)
        2'b00:q <= d[0];
        2'b01:q <= d[1];
        2'b10:q <= d[2];
        2'b11:q <= d[3];
    endcase
end
```

一般的书写方式推荐使用换行这个形式，比较容易理解和方便阅读。

2.2 注释

注释一般分为两种，一种单行注释，使用“//”；另一种多行注释，使用“/* */”。例：

```
module MUX4_1(
    input [3:0]d,//MUX 输入
    input [1:0]s,//选择信号
    output reg q
);
/*
4 选 1 MUX 设计
```

```
使用的组合逻辑设计
寄存输出
*/
always@(*)
begin
    case(s)
        2'b00:q <= d[0];
        2'b01:q <= d[1];
        2'b10:q <= d[2];
        2'b11:q <= d[3];
    endcase
end
endmodule
```

其中关键符号“//”表示单行注释，“/* */”表示多行注释，使用注释可以使得程序设计可读性较高，方便进行编程理解。

2.3 关键字与标识符

Verilog HDL 中的标识符可以是任意一组字母、数字、\$符号和下划线(下划线)符号的组合，但标识符的第一个字符必须是字母或者下划线开头，不能是\$符号和数字。另外注意，标识符是区分大小写的。

Verilog HDL 定义了一系列保留字，叫做关键字，它仅用于某些上下文中。注意只有小写的关键词才是保留字。下面列出了 Verilog-1364-2005 语言中的所有关键字。

● always ● and ● assign ● automatic ● begin ● buf ● buff0 ● buff1 ● case ● casex ● casez ● cell ● cmos ● config ● deassign ● default ● defparam ● disable ● edge ● else ● end ● endcase ● endconfig ● endfunction ● endgenerate ● endmodule ● endprimitive ● endspecify ● endtable ● endtask ● event	● for ● function ● force ● forever ● fork ● generate ● genvar ● highz0 ● highz1 ● if ● ifnone ● ifcdir ● include ● initial ● inout ● input ● instance ● integer ● join ● large ● liblist ● library ● localparameter ● macromodule ● medium ● module ● nand ● negedge ● nmos ● nor ● not	● notif0 ● notif1 ● noshowcancelled ● notif0 ● notif1 ● pmos ● pulsestyle_oneevent ● pulsestyle_ondetect ● rcmos ● real ● realtime ● reg ● release ● repeat ● rnmos ● rpmos ● rtran ● rtranif0 ● rtranif1 ● scalared ● showcancld ● signed ● small ● specify ● specparam ● strong0 ● strong1 ● supply0 ● supply1 ● table ● task ● time	● tran ● or ● posedge ● output ● parameter ● tranif0 ● tranif1 ● tri ● tri0 ● tri1 ● triand ● trior ● trireg ● unsigned ● use ● uwire ● vertored ● wait ● wand ● weak0 ● weak1 ● while ● wire ● wor ● xnor ● xor ● primitive ● pull0 ● pull1 ● pulldown ● pullup
--	---	---	--

这里仅列出关键字，不进行具体的讲解，等后续语法用到进行详细讲解。亦或感兴趣的自己去查资料了解各关键字含义。

2.4 数值种类及表示方法

2.4.1 数值种类

Verilog 中有四种基本的值来表示数字电路中的电平逻辑:

- 0: 逻辑 0 低电平
- 1: 逻辑 1 高电平
- x 或 X: 未知
- z 或 Z: 高阻态

“x”意味着信号数值的不确定, 信号可能为 1, 也可能为 0。

“z”意味着信号处于高阻态, 常见于信号 (input, reg) 没有驱动时的数字逻辑结果。比如一个 pad 的 input 呈现高阻状态时, 其逻辑值和上下拉的状态有关系。上拉则逻辑值为 1, 下拉则为 0。

2.4.2 数值表示方法

Verilog 中有四种合法的基数格式, 分别是二进制, 八进制, 十进制, 十六进制, 分别使用下列字符标识:

- 二进制 'b / 'B
- 八进制 'o / 'O
- 十进制 'd / 'D
- 十六进制 'h / 'H

数值可以指明位宽, 也可以不指明位宽。一般格式为<位宽><进制><数字>。

比如:

指明位宽

```
assign test=16'h2581;
```

不指明位宽

```
assign test='h2581;
```

2.5 字符串

字符串是由双引号包括起来的字符队列。字符串不能多行书写, 即字符串中不能包含回车符。当字符串存储于寄存器中, 每个字符要占 8 位, 字符以 ASCII 代码形式存储。Verilog HDL 语言的字符串的定义和 C 语言不一样。在 C 语言中需要用而在 Verilog HDL 语言中不需要用 ASCII 代码的 0 字符来表示字符串的结束。例如, 为存储字符串 "www.uisrc.com", 需要 13*8bit 的存储单元。例:

```
reg [0: 13*8-1]      string ;  
assign string = "www. uisrc.com";
```

第五课 Verilog 语法_数据类型及表达式

软件版本：无

操作系统：WIN10 64bit

硬件平台：适用所有系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑！

1 概述

本节主要讲解 Verilog 的数据类型和表达式。

2 数据类型

2.1 wire 线网

wire 线网，仅建立连接，而没有逻辑行为或功能，表示各个单元之间的连线。例：

```
wire [15:0] input_net;
wire reg_flag;
```

wire 是最常用的线网，当然还存在其他的线网。比如：

tri	三态线网	triand	三态线与
trior	三态线或	tri0	三态下拉线网
tri1	三态上拉线网	trireg	三态寄存器
wand	线与逻辑	wor	线或逻辑
supply0	用于模拟 GND 电源负极	supply1	用于模拟 VCC 电源正极

当多个驱动源驱动同一根线的线网，各真值表如下所示：

wire	0	1	x	z
0	0	x	x	0
1	x	1	x	1
x	x	x	x	x
z	0	1	x	z

tri	0	1	x	z
0	0	x	x	0
1	x	1	x	1
x	x	x	x	x
z	0	1	x	z

wor	0	1	x	z
0	0	1	x	0
1	1	1	1	1
x	x	1	x	x
z	0	1	x	z

trio	0	1	x	z
0	0	1	x	0
1	1	1	1	1
x	x	1	x	x
z	0	1	x	z
tri0	0	1	x	z
0	0	x	x	0

wand	0	1	x	z
0	0	0	0	0
1	0	1	x	1
x	0	x	x	x
z	0	1	x	z
tri1	0	1	x	z
0	0	x	x	0

triand	0	1	x	z
0	0	0	0	0
1	0	1	x	1
x	0	x	x	x
z	0	1	x	z

1	x	1	x	1
x	x	x	x	x
z	0	1	x	0

1	x	1	x	1
x	x	x	x	x
z	0	1	x	1

2.2 reg 寄存器

数据类型 `reg` 寄存器是硬件存储单元的一种抽象模型，但它并不直接与物理存储器直接对应。一个 `reg` 变量有一个默认的初始值 `x`。寄存器变量的默认大小是一个比特。例：

```
reg [7:0] reg_ff; // 八比特的寄存器变量
reg flag;         // 单比特寄存器变量
```

2.3 integer 整数

数据类型 `integer` 支持过程赋值语句中的数值计算。整数可用来表示主机的字长。负整数是用补码的形式存储，而且 `integer` 变量默认值为 0。`integer` 类型也是一种寄存器数据类型。

Verilog 运算符是以补码算术形式在整数上进行运算，并且最高位是表示值的符号。例：

```
wire [15:0] data_out;           // 十六比特的寄存器变量
reg [15:0] data_reg;           // 十六比特的寄存器变量
integer i;
always@(*)
begin
    for(i=0; i<15; i=i+1)       // 循环 15 次
    begin
        data_reg[i] = data_out[15 - i]; // 依次赋值
    end
end
```

2.4 real 实数

实数用关键字 `real` 来声明，是用双精度存储，典型的一个 64 位值，也可用十进制或科学计数法来表示。实数默认值为 0。不能把实数类型的变量连接在模块的端口上，如果将一个实数赋值给一个整数，则只有实数的整数部分会赋值给整数。例：

```
real data;
integer data_reg;

initial begin
    data = 30.5;                // 赋值
end

initial begin
    data_reg = data;             // 赋值
end
```

`data_reg` 等于 30。

2.5 time 时间

数据类型 `time` 支持 Verilog 模型程序中关于时间的计算，`time` 变量存储为无符号 64 位的量值。可以使用系统函数 `$time` 获取当前仿真时间。例：

```
time first_time;
initial begin
    #1000;
    first_time = $time;           //获取仿真时间
end
```

2.6 数组

`reg` 类型的一维数组称为存储器，可以理解为数组。这种构造是寄存器变量声明的扩展，用来提供存储器，比如相同字长的多个可寻址的单元。

数组维数没有限制。线网数组也可以用于连接实例模块的端口。数组中的每个元素都可以作为一个标量或者向量，以同样的方式来使用，形如：<数组名>[<下标>]。对于多维数组来讲，用户需要说明其每一维的索引。例：

```
integer    data_1[7:0];
reg  [3:0] data_2[3:0];
wire [7:0] data_3[3:0];
wire      data_4[6:0][5:0];
reg [31:0] data_5[1:0][5:0][8:0][15:0];
```

2.7 字符串

对于字符串，Verilog 没有专门的数据类型对应。一个字符串必须通过一条过程赋值语句存入一个大小合适的寄存器中。一个大小合适的寄存器数组为要保存的字符串的每个字符留有 8 个比特的存储空间。例：

```
reg [8*num_char-1:0] string; //预留 8 比特的数据
```

这个例子表明 `num_char` 个字符中的每一个字符都用 8 个比特来存储编码。如果“hello”被赋给 `string`，那么 `num_char` 至少为 5 才可以保存字符串。如果给一个数组的赋值比将要处理的数组的字符数少，那么从最高位开始使用在未使用的位置上赋值 0。

2.8 常量

Verilog 中的常量可用关键词 `parameter` 来声明，它在声明的同时还可以对其进行常量赋值。一个常量的值在仿真过程中是不会改变的，常量表达式可以用于对一个常量值进行声明。例：

```
parameter data_1 = 31;
parameter data_2 = 131;
parameter data_3 = 231;
parameter data_4 = data_3*3-data_2;
parameter data_5 = 34,data_6 = data_4 - data_1;
```

还有一个常量 `localparam`，它与 `parameter` 的区别是只作用于某一个模块内部，且为常量，而 `parameter` 可以通过调用进行赋值。

3 表达式

Verilog 的表达式是将操作符合操作数结合起来产生一个结果，我们先来介绍操作符和操作数。

3.1 操作数

操作数可以为上一小节讲到的所有数据类型声明的数。比如，常数、整数、线网、寄存器、时间等。

3.2 操作符

3.2.1 算术运算符

算术运算符是双目操作符，对 2 个操作数进行算术运算，包括加 (+)、减 (-)、乘 (*)、除 (/)、求幂 (**)、取模 (%)。这些操作数的数值可以是二进制、八进制、十进制和十六进制的形式。例：

```
reg [3:0] A,B;
wire [4:0] sum,diff1,diff2,neg;
wire [7:0] mult;
assign sum =A+B;
assign diff1=A-B;
assign diff2=B-A;
assign neg=-B;
assign mult = A*B;
```

负数的表示在十进制下直接在前面添加“-”号，二进制中使用补码进行表示负数。

3.2.2 按位运算符

按位运算符有按位取反 (~)、按位与 (&)、按位或 (|)、按位异或 (^) 和按位异或非 (^~或~^)。

按位操作符对 2 个操作数的每 1bit 数据进行按位操作。如果 2 个操作数位宽不相等，则用 0 向左扩展补充较短的操作数。取反操作符只有一个操作数，它对操作数的每 1bit 数据进行取反操作。例：

```
wire [3:0] A=4'b1101,B=4'b1001,C=4'b0001,D=4'b1111;
~A -> 4'b0010
A&B-> 4'b1001
A|D-> 4'b1111
C^D-> 4'b1110
B^~C->4'b1000
```

3.2.3 缩减运算符

缩减运算符包含缩减与 (&)、缩减与非 (~&)、缩减或 (|)、缩减或非 (~|)、缩减异或 (^)、缩减异或非 (^~或~^)。缩减运算符是单目运算符，通过在一个数据上运算可以得到单个位的结果值。例：

```
wire [3:0] A=4'b1101;
&A -> 1'b0
```

```
~&A -> 1'b1
|A -> 1'b1
~|A -> 1'b0
^A -> 1'b1
^~A -> 1'b0
```

3.2.4 逻辑运算符

逻辑运算符有逻辑非 (!)、逻辑与 (&&)、逻辑或 (||)、逻辑相等 (==)、逻辑不等 (!=)、情形相等 (===)、情形不等 (!==)。

Verilog 逻辑运算符作为逻辑连接,对布尔操作数进行运算得到布尔结果。该操作数可以是线网、寄存器或是可以得到布尔结果的表达式。

情形相等 (===) 用来确定两个操作数的对应位上是否完全一致,其中包括了值为 X 和 Z 的比较。

3.2.5 关系运算符

关系运算符包括小于 (<)、大于 (>)、小于等于 (<=)、大于等于 (>=)。它用来比较两个操作数得到一个布尔值。如果操作数是线网或者寄存器类型的,其值可按无符号字对待。如果任何一个位是未知的,或者关系不明确的,则返回的结果是 X。

3.2.6 移位运算符

移位操作符包括左移 (<<),右移 (>>),算术左移 (<<<),算术右移 (>>>)。

移位操作符是双目操作符,两个操作数分别表示要进行移位的向量信号(操作符左侧)与移动的位数(操作符右侧)。算术左移和逻辑左移时,右边低位会补 0。逻辑右移时,左边高位会补 0;而算术右移时,左边高位会补充符号位,以保证数据缩小后值的正确性。例:

```
A = 4'b1100 ;
B = 4'b0010 ;
A = A >> 2 ;      //结果为 4'b0011
A = A << 1;       //结果为 4'b1000
A = A <<< 1 ;     //结果为 4'b1000
C = B + (A>>>2);  //结果为 2 + (-4/4) = 1, 4'b0001
```

3.2.7 条件运算符

Verilog 条件赋值运算符根据一个 condition_expression 的值选择一个表达式用于求值。条件运算符的语法格式如下:

condition_expression ? true_expression : false_expression

如果 condition_expression 为真(逻辑值为 1),则运算结果为 true_expression;如果 condition_expression 为假(逻辑值为 0),则计算结果为 false_expression。例:

```
assign sel    = (data[1:0] == 2'b00) ? sel_1 : sel_2 ;//data 为 00 时,选择 sel_1, 否则为 sel_2
```

条件运算符可以进行嵌套,完成一个多次选择的逻辑。例:

```
assign hsel = (data[1:0] == 2'b00) ? sel_1 : //data 为 00 时, 选择 sel_1
              (data[1:0] == 2'b01) ? sel_2 : //data 为 01 时, 选择 sel_2
              (data[1:0] == 2'b10) ? sel_3 : //data 为 10 时, 选择 sel_3
              (data[1:0] == 2'b11) ? sel_4 ; //data 为 11 时, 选择 sel_4
```

3.2.8 拼接运算符

拼接运算符可以将两个或多个的操作数形成一个单字。这个运算在形成逻辑总线的时候特别有用，拼接的结果仍然按照所给的字的顺序排列。拼接运算可以用到循环和任意程序的嵌套中。例：

```
{4{a}}={a,a,a,a}
{2{a},{2'b01},{3'b110}}}
```

3.2.9 运算符的优先级

Verilog 是从左向右计算表达式的。布尔表达式的计算过程中，如果可以判断最终值为真或假，则马上结束运算。优先级由高到低，同一行优先级相同。

! ~
**
*/ %
+ -
<< >> <<< >>>
< > <= >=
== != === !==
& ~&
^ ^~ ~^
~
&&
?:

第六课 Verilog 语法_过程结构与赋值

软件版本：无

操作系统：WIN10 64bit

硬件平台：适用所有系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑！

1 概述

本节主要讲解过程结构与赋值，需要掌握阻塞和非阻塞赋值的区别。

2 过程结构

过程结构语句有两种，initial 与 always 语句。它们是 RTL 级建模的两种基本语句。一个模块中可以包含多个 initial 和 always 语句。这些语句在模块间并行执行，与其在模块的前后顺序没有关系。但是 initial 语句或 always 语句内部可以理解为是顺序执行的（非阻塞赋值除外）。两种语句在仿真的开始同时立即执行，initial 语句只执行一次，而 always 语句重复执行，直到仿真结束。

2.1 initial 语句

initial 语句的格式如下：

```
initial
begin
语句 1;
语句 2;
.....

语句 n;

end
```

initial 可以对 reg 型变量赋初始值，初始化的过程不需要任何时间。例：

```
reg [7:0] data;
initial
begin
data=8'h32;           //赋值
#10                   //经过 10 个时钟周期
data=8'h12;           //赋值
#10                   //经过 10 个时钟周期
data=8'h02;           //赋值
#10                   //经过 10 个时钟周期
data=8'h22;           //赋值
```

```
end
```

initial 语句可以生成激励波形，用于测试电路的仿真信号。

2.2 always 语句

always 语句是重复执行的语句，不像 initial 语句只执行一次。声明格式如下：

always <时序控制> <语句>例：

```
reg clk;                //reg 变量
initial clk = 0;         //reg 通过 initial 赋值
always #10 clk = ~clk;   //重复翻转
```

使用 always 语句生成一个仿真的时钟信号。

3 过程赋值

过程性赋值是在 initial 或 always 语句块里的赋值，赋值对象是寄存器、整数、实数等类型。这些变量在被赋值后，其值将保持不变，直到重新被赋予新值。

连续性赋值总是处于激活状态，任何操作数的改变都会影响表达式的结果；过程赋值只有在语句执行的时候，才会起作用。这是连续性赋值与过程性赋值的区别。

Verilog 过程赋值包括两种类型：阻塞赋值与非阻塞赋值。

3.1 阻塞赋值

阻塞赋值属于顺序执行，即下一条语句执行前，当前语句一定会执行完毕。阻塞赋值语句使用等号“=”作为赋值符。例：

```
always@(*)
begin
    q1 = d[0];
    q2 = d[1];
    q3 = d[2];
    if(s == 2'b00)
        q = d[0];    //阻塞赋值，同时进行
    else if(s == 2'b01)
        q = d[1];    //阻塞赋值，同时进行
    else if(s == 2'b10)
        q = d[2];    //阻塞赋值，同时进行
    else if(s == 2'b11)
        q = d[3];    //阻塞赋值，同时进行
    else
        ;
end
```

3.2 非阻塞赋值

非阻塞赋值属于并行执行语句，即下一条语句的执行和当前语句的执行是同时进行的，它不会阻塞位于同一个

语句块中后面语句的执行。

非阻塞赋值语句使用 “<=” 作为赋值符。例：

```
always@(posedge clk )
begin
    q1 <= d[0];
    q2 <= d[1];
    q3 <= d[2];
    if(s == 2'b00)
        q <= d[0];    //非阻塞赋值，不会相互影响
    else if(s == 2'b01)
        q <= d[1];    //非阻塞赋值，不会相互影响
    else if(s == 2'b10)
        q <= d[2];    //非阻塞赋值，不会相互影响
    else if(s == 2'b11)
        q <= d[3];    //非阻塞赋值，不会相互影响
    else
        ;
end
```

3.3 阻塞与非阻塞赋值的使用注意

一般在 Verilog 代码设计时，一般不在一个过程结构中混合使用阻塞赋值与非阻塞赋值。两种赋值方式混用时，时序不容易控制，会造成意外的错误。

通常在设计电路时，always 时序逻辑块中多用非阻塞赋值，always 组合逻辑块中多用阻塞赋值。在仿真电路时，initial 块中一般多用阻塞赋值。

为实现在时钟上升沿交换 2 个寄存器值的功能，在 2 个 always 块中使用阻塞赋值。

因为 2 个 always 块中的语句是同时进行的，但是 a=b 与 b=a 是无法判定执行顺序的，这就造成了竞争的局面。

不考虑哪个先执行，不考虑时序问题时，执行顺序总有先后，最后 a 与 b 的值总是相等的。没有达到交换 2 个寄存器值的效果。例：

```
always @(posedge clk) begin
    a = b ;    //阻塞赋值，同时进行
end

always @(posedge clk) begin
    b = a;    //阻塞赋值，同时进行
end
```

但是，如果在 always 块中使用非阻塞赋值，则可以避免上述竞争冒险的情况。

2 个 always 块中语句并行执行，赋值操作右端操作数使用的是上一个时钟周期的值，此时 a<=b 与 b<=a 就可以相互不干扰的执行，达到交换寄存器值的目的。例：

```
always @(posedge clk) begin
    a <= b ;    //非阻塞赋值，对上个周期的值进行操作
end
```

```
always @(posedge clk) begin
    b <= a;    //非阻塞赋值，对上个周期的值进行操作
end
```

总结：为了不出现问题，平时建议规范书写代码，注意几点：

- 1、时序电路用非阻塞
- 2、组合电路用阻塞
- 3、同一个 always 块中阻塞和非阻塞不混用

第七课 Verilog 语法_时序控制与语句块

软件版本：无

操作系统：WIN10 64bit

硬件平台：适用所有系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑！

1 概述

本节讲解时序控制语句相关的语法与几种语句块的使用介绍，需要掌握时序控制的规则和顺序块、并行块、命名块、嵌套块的使用。

2 时序控制简介

Verilog 提供了两类时序控制方法：时延控制和事件控制。时延控制主要用于仿真，事件控制主要用于时序设计，分为边沿触发控制与电平敏感控制。

2.1 时延控制

基于时延的时序控制出现在表达式中，它指定了语句从开始执行到执行完毕之间的时间间隔。时延可以是数字、标识符或者表达式。

通常根据时延在表达式中的位置差异，时延控制又可以分为常规时延与内嵌时延。

2.1.1 常规时延

格式为#<数字> <表达式>例：

```
reg [7:0] data;
initial
begin
#10 data=8'h12; //常规时延
#10 data=8'h02; //常规时延
#10 data=8'h22; //常规时延
end
```

2.1.2 内嵌时延

遇到内嵌延迟语句后，先计算出表达式右端的结果，然后再延迟一定的时间，赋值给目标信号例：

```
reg [7:0] data;
initial
begin
    data=#10 8'h12; //内嵌式时延
    data=#10 8'h02; //内嵌式时延
    data=#10 8'h22; //内嵌式时延
end
```

```
end
```

需要说明的是，这两种时延控制方式的效果是有所不同的。

当延时语句的赋值符号右端是常量时，两种时延控制都能达到相同的延时赋值效果。

当延时语句的赋值符号右端是变量时，两种时延控制可能会产生不同的延时赋值效果。

2.2 事件控制

2.2.1 边沿触发控制

边沿触发在 Verilog 中指寄存器或者线网类型变量发生了值的变化。

关键字 `posedge` 指信号发生边沿正向跳变，`negedge` 指信号发生负向边沿跳变，未指明跳变方向时，则两种情况的边沿变化都会触发相关事件。例：

```
always@(posedge clk )    //关键词 posedge, clk 正向边沿跳变
begin
    q1 <= d[0];
    q2 <= d[1];
    q3 <= d[2];
    if(s == 2'b00)
        q <= d[0];
    else if(s == 2'b01)
        q <= d[1];
    else if(s == 2'b10)
        q <= d[2];
    else if(s == 2'b11)
        q <= d[3];
    else
        ;
end
```

```
always@(negedge clk ) //关键词 negedge, clk 负向边沿跳变
begin
    q1 <= d[0];
    q2 <= d[1];
    q3 <= d[2];
    if(s == 2'b00)
        q <= d[0];
    else if(s == 2'b01)
        q <= d[1];
    else if(s == 2'b10)
        q <= d[2];
    else if(s == 2'b11)
        q <= d[3];
    else
        ;
end
```

```
end
```

```
always@(clk )           //未指明跳变方向, clk 可边沿正向跳变, 可负向边沿跳变
begin
    q1 <= d[0];
    q2 <= d[1];
    q3 <= d[2];
    if(s == 2'b00)
        q <= d[0];
    else if(s == 2'b01)
        q <= d[1];
    else if(s == 2'b10)
        q <= d[2];
    else if(s == 2'b11)
        q <= d[3];
    else
        ;
end
```

说明 always@ (敏感列表), 敏感列表中可以有多个不同的事件, 中间用关键字 “or” 或 “,” 分开。例:

```
always@(negedge clk or negedge rst) //关键词 negedge, clk 负向边沿跳变;关键词 negedge,rst 可负向边沿跳变
begin
    if (!rst_n)
        q <= 0;
    else if(s == 2'b00)
        q <= d[0];
    else if(s == 2'b01)
        q <= d[1];
    else if(s == 2'b10)
        q <= d[2];
    else if(s == 2'b11)
        q <= d[3];
    else
        ;
end
```

```
always@(negedge clk , negedge rst)/关键词 negedge, clk 负向边沿跳变;关键词 negedge,rst 可负向边沿跳变
begin
    if (!rst_n)
        q <= 0;
    else if(s == 2'b00)
        q <= d[0];
    else if(s == 2'b01)
        q <= d[1];
    else if(s == 2'b10)
```

```
    q <= d[2];  
    else if(s == 2'b11)  
        q <= d[3];  
    else  
        ;  
end
```

当组合逻辑输入变量很多时，那么编写敏感列表会很麻烦。此时，更为简洁的写法是“@*”或“@(*)”，表示对语句块中的所有输入变量的变化都是敏感的。例：

```
wire [15:0] data_out;  
reg  [15:0] data_reg;  
integer i;  
always@(*)                //对语句块中的所有输入变量的变化都是敏感的  
begin  
    for(i=0;i<15;i=i+1)  
    begin  
        data_reg[i] = data_out[15 - i];  
    end  
end
```

命名事件控制，用户可以声明 event 类型的变量，并触发该变量来识别该事件是否发生。命名事件用关键字 event 来声明，触发信号用 -> 表示。例如：

```
event start_r ;  
always@( posedge clk)  
begin  
    -> start_r;  
end  
  
always@(start_r)          //事件触发，敏感词  
begin  
    data_buf = {data[0], data[1]};  
end
```

2.2.2 电平敏感控制

Verilog 中支持使用电平作为敏感信号来控制时序，即后面语句的执行需要等待某个条件为真。Verilog 中使用关键字 wait 来表示这种电平敏感情况。例：

```
initial begin  
    wait (start_1) ;    //电平敏感时间，等待 start_1 事件触发  
    forever  
begin  
    @(posedge clk);  
    data_1= {data[0], data[1]};  
end
```

3 语句块介绍

Verilog 中有四种语句块，分别是顺序块，并行块，嵌套块，命名块。

3.1 顺序块

顺序块用关键字 `begin` 和 `end` 来表示。顺序块中的语句是一条条执行的。非阻塞赋值除外。

仿真中，`initial` 块中的阻塞赋值，都是顺序块的实例。例：

```
reg [7:0] data;

initial
begin
    #10 data=8'h12;    //按照时间，顺序执行
    #10 data=8'h02;
    #10 data=8'h22;
end
```

3.2 并行块

并行块有关键字 `fork` 和 `join` 来表示。并行块中的语句是并行执行的，即便是阻塞形式的赋值。并行块中每条语句的时延都是与块语句开始执行的时间相关。例：

```
initial
fork
    #10 data=8'h12;    //与 data 按照时间并行执行，与 data=8'h02 同时进行，一般实际情况我们不这样使用，仅仅是作为示例
    #10 data=8'h02;
join

initial
fork
    #10 data1=8'h32;   //与 data1 按照时间并行执行，与 data1=8'h22 同时进行，一般实际情况我们不这样使用，仅仅是作为示例
    #10 data1=8'h22;
join
```

3.3 嵌套块

顺序块和并行块还可以嵌套使用。例：

```
reg [7:0] data;

initial
begin
    #10 data=8'h12;
    #10 data=8'h02;
    #10 data=8'h22;
    fork
        #10 data=8'h12;
        #10 data=8'h02;
```

```
join  
end
```

3.4 命名块

可以给块语句结构命名。例：

```
reg [7:0] data;  
initial  
begin:start_initial  
#10 data=8'h12;  
#10 data=8'h02;  
#10 data=8'h22;  
fork  
    #10 data=8'h12;  
    #10 data=8'h02;  
join  
end
```

第八课 Verilog 语法_条件与循环语句

软件版本：无

操作系统：WIN10 64bit

硬件平台：适用所有系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑！

1 概述

本节主要讲解 Verilog 语法条件、循环语句，需要掌握 if、case、casex、casez、while、for、repeat、forever 语句的用法。

2 条件语句

2.1 if

if 语句是用来判定所给的条件是否满足，根据判定的结果（布尔值）决定执行给出的两种操作之一，Verilog 语言给出三种形式的 if 语句：

（1）if（表达式）语句；例：

```
always@(negedge clk or negedge rst) //always 触发条件
begin
    if (!rst_n) //rst_n 为 0 时满足条件，进入执行语句
        q <= 0; //q 赋值 0
end
```

（2）if（表达式）

语句 1；

else

语句 2；例：

```
always@(negedge clk or negedge rst) //always 触发条件
begin
    if (!rst_n) //rst_n 为 0 时满足条件，进入执行语句
        q <= 0; //q 赋值 0
    else
        q <= 1; //rst_n 为非 0 的时候进入执行语句，q 赋值 1
end
```

（3）if（表达式）

语句 1；

else if（表达式 1）

语句 2；

else if（表达式 2）

语句.....；

else if (表达式 n-2)

语句 n-2;

else if (表达式 n-1)

语句 n-1;

else

语句 n; 例:

```
always@(negedge clk or negedge rst) //always 触发条件
begin
    if (!rst_n)                //rst_n 为 0 时满足条件, 进入执行语句
        q <= 0;                //q 赋值 0
    else if(s == 2'b00)        //s 为 2'b00 时满足条件, 进入执行语句
        q <= d[0];            //q 赋值 d[0]
    else if(s == 2'b01)        //s 为 2'b01 时满足条件, 进入执行语句
        q <= d[1];            //q 赋值 d[1]
    else if(s == 2'b10)        //s 为 2'b10 时满足条件, 进入执行语句
        q <= d[2];            //q 赋值 d[2]
    else if(s == 2'b11)        //s 为 2'b11 时满足条件, 进入执行语句
        q <= d[3];            //q 赋值 d[3]
    else
        ;
end
```

三种格式的 if 语句在 if 后都有表达式, 该表达式一般为逻辑表达式或者关系表达式。系统对表达式的值进行判断, 若为 0, x, z, 按假处理; 若为 1, 按真处理, 执行指定的语句。

三种形式的 if 语句, 每个 else 前有一个 “;” 整个语句结束的时候有一个 “;”。例:

```
if (!rst_n)                //rst_n 为 0 时满足条件, 进入执行语句
    q <= 0;                //q 赋值 0, 用“;”隔开
else
    q <= 1;                //q 赋值 1, 用“;”隔开
```

因为分号是 Verilog 语法不可缺少的部分, 是 if 内嵌语句所要求的。如果没有分号, 将报语法错误。

在 if 和 else 后可以内嵌操作语句, 比如, begin end, 里面可以同时执行多条语句。例:

```
if(s == 2'b00)            //当 s 等于 2'b00 进入执行语句
begin
    q <= d[0];            //q 赋值 d[0], 用“;”隔开
    q1 <= d[0];           //q1 赋值 d[0], 用“;”隔开
    q2 <= d[1];           //q2 赋值 d[1], 用“;”隔开
    q3 <= d[2];           //q3 赋值 d[2], 用“;”隔开
end
else                        //当 s 不等于 2'b00 进入执行语句
begin
    q <= d[1];            //q 赋值 d[1], 用“;”隔开
    q <= d[0];            //q 赋值 d[0], 用“;”隔开
    q1 <= d[0];           //q1 赋值 d[0], 用“;”隔开
    q2 <= d[1];           //q2 赋值 d[1], 用“;”隔开
    q3 <= d[2];           //q3 赋值 d[2], 用“;”隔开
```

```
end
```

同时允许多个 if 语句进行嵌套使用。

if (表达式)

if (表达式 1)

语句 1;

else

语句 2;

else

if (表达式 2)

语句 3;

else

语句 4;

2.2 case

case 语句是一种多路分支选择语句，if 语句只有两个分支可以选择，而实际应用中常用到多路选择，case 的一般形式如下：

(1) case (表达式) <case 分支>endcase

(2) casex (表达式) <case 分支>endcase

(3) casez (表达式) <case 分支>endcase

case 分支项一般格式如下：

分支表达式: 语句;

默认项 (default 项): 语句; 例:

```
case(s)                //判断参数为 s
    2'b00:q <= d[0];    //s 为 2'b00, q 赋值 d[0]
    2'b01:q <= d[1];    //s 为 2'b01, q 赋值 d[1]
    2'b10:q <= d[2];    //s 为 2'b10, q 赋值 d[2]
    2'b11:q <= d[3];    //s 为 2'b11, q 赋值 d[3]
    default:q <= 0;
endcase
```

case 后的表达式称为控制表达式，case 分支项里面的表达式称为分支表达式。控制表达式通常表示为控制信号的某些位，分支表达式则用这些控制信号的具体状态值来表示，因此分支表达式又可以称为常量表达式。

当控制表达式的值与分支表达式的值相匹配时，就执行分支表达式后面的语句。如果所有的分支表达式的值都没有与控制表达式的值相匹配，就执行 default 后面的语句。

default 项可有可无，一个 case 语句里只准有一个 default 项。

每一个 case 分项的分支表达式的值必须互不相同，否则就会出现矛盾，即对表达式的同一个值，将出现多种执行方案，产生矛盾。

执行完 case 分项后的语句，则跳出该 case 语句结构，终止 case 语句的执行。

在用 case 语句表达式进行比较的过程中，只有当对应位的值能明确进行比较时，比较才能成功。因此，要注意

详细说明 case 分项的分支表达式的值。

case 语句的所有表达式值的位宽必须相等，只有这样，控制表达式和分支表达式才能进行对应位的比较。

下面给出 case, casex, casez 的真值表：

case	0	1	x	z
0	1	0	0	0
1	0	1	0	0
x	0	0	1	0
z	0	0	0	1

casez	0	1	x	z
0	1	0	0	1
1	0	1	0	1
x	0	0	1	1
z	1	1	1	1

casex	0	1	x	z
0	1	0	1	1
1	0	1	1	1
x	1	1	1	1
z	1	1	1	1

通过真值表，可以看到 case, casex, casez 依次增加了等价了的范围，因此，使用时需要分析清楚应用逻辑的表达是否正确。

3 循环语句

Verilog 中的循环语句有 4 个，分别是 forever, repeat, while 和 for 循环。

3.1 forever 语句

forever 语句的格式如下：

forever 语句；或

forever

begin

多条语句；

end

forever 语句常用来产生仿真的周期性波形，它与 always 语句的区别是不能直接写在程序中，必须写在 initial 里。例：

```
reg clk ;    //reg 变量 clk
initial
begin
    clk = 1 ;    //reg 变量 clk 赋初值
    forever
    begin
        clk =!clk ;//reg 变量 clk 翻转
        #10 ;
    end
end
```

此时，forever 产生了一个周期性的交变信号，通常用来生成时钟信号。

forever 语句表示一直循环，可以不需要任何条件表达式，一旦开始执行便无限循环的执行下去，只有系统函数 \$finish 可以退出 forever。

3.2 repeat 语句

repeat 语句的格式如下：

repeat (循环次数表达式) 语句；

或

repeat (循环次数表达式)

begin

多条语句；

end

其表达式通常为常量表达式。如果循环次数是个变量，则循环次数是开始执行 repeat 循环时变量信号的值。即执行期间，变量信号值发生了变化，repeat 执行次数也不会改变。例：

```
reg [3:0] data;
initial
begin
    data = 4'b0000 ;    //data 赋初值
    repeat (10)         //下列操作循环 10 次
begin
    #10 ;
    data = data + 1'b1 ;
end
end
```

此时，repeat (10)执行了 10 次变量 data 的累加操作。

3.3 while 语句

while 语句的格式如下：

while (表达式) 语句；

或

while (表达式)

begin

多条语句；

end

当表达式为真时执行循环里面的语句，当表达式为假时，跳出循环。例：

```
reg [3:0] data;
initial
begin
    data = 4'b0000 ;    //data 赋初值
    while (data <=10)    //while (data <=10)，变量 data 的累加值小于 10 时继续循环，大于 10 后跳出循环
begin
    #10 ;
    data = data + 1'b1 ;
end
end
```

```
end
```

此时，while (data <=10)，变量 data 的累加值小于 10 时继续循环，大于 10 后跳出循环。

3.4 for 语句

for 语句的一般形式如下：

for（表达式 1；表达式 2；表达式 3）语句；

它的执行过程如下：

（1）先求解表达式 1。

（2）求解表达式 2 的值，其值若为真，则执行 for 语句中内嵌的语句，然后执行第（3）步。若为假，则结束循环，执行第（5）步。

（3）求解表达式 3。

（4）执行第（2）步。

（5）结束 for 语句，进行下面的语句执行。

for 语句的应用形式如下：

for（循环变量赋初始值；循环结束条件；循环变量增加）

执行语句；

for 语句的功能可以用 while 语句同样实现，如下：

```
begin
```

```
    循环变量赋初始值；
```

```
    while（循环结束条件）
```

```
    begin
```

```
        执行语句；
```

```
        循环变量增加；
```

```
    end
```

```
end 例：
```

```
integer i;
reg [3:0] data;
initial begin
    data = 4'b0000;           //data 赋初值
    for (i=0; i<=10; i=i+1)   //for 循环执行在条件 i<=10 时进行 data 变量的累加
    begin
        #10;
        data = data + 1'b1;
    end
end
```

此时，for 循环执行在条件 i<=10 时进行 data 变量的累加。

第九课 Verilog 语法_模块端口与例化

软件版本: 无

操作系统: WIN10 64bit

硬件平台: 适用所有系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑!

1 概述

本小节讲解 Verilog 语法的模块端口与例化的方法, 需要掌握模块的端口的类型, 以及有几种例化方式。

2 模块端口

模块在介绍设计方法时有提到子模块, 与此概念基本相通, 但是在 Verilog 语法中, 模块有一定的格式要求。

module 名字

#(参数列表)

(端口列表);

语句块;

endmodule 例:

```
module MUX4_1(  
    input [3:0]d,      //MUX 输入  
    input [1:0]s,      //选择信号  
    output reg q  
);  
always@(*)  
begin  
    case(s)  
        2'b00:q <= d[0];  
        2'b01:q <= d[1];  
        2'b10:q <= d[2];  
        2'b11:q <= d[3];  
        default:q <= 0;  
    endcase  
end  
endmodule
```

模块定义必须以关键字 `module` 开始, 以关键字 `endmodule` 结束。模块名、端口信号、端口声明和可选的参数声明等, 出现在设计使用的语句块之前。

每个模块都必须有端口, 端口是模块与外界交互的接口。对于外部模块来说, 模块内部是不可见的, 对模块的使用只能通过端口连接来进行。

端口的方向有三种, 分别是 `input` (输入端口)、`output` (输出端口) 和 `inout` (输入输出双向端口), 注意端口的声明类型, `input`、`inout` 类型不能声明为 `reg` 数据类型, 因为 `reg` 类型是用于存储数值的, 而输入端口只能反映与其相连的外部信号的变化, 不能保存这些信号的值。

output 可以声明为 wire 或 reg 数据类型。

端口的声明一般有两种，如下所示。例：

```
module MUX4_1(  
    input wire [3:0]d,  
    input wire [1:0]s,  
    output reg      q  
);  
always@(*)  
begin  
    case(s)  
        2'b00:q <= d[0];  
        2'b01:q <= d[1];  
        2'b10:q <= d[2];  
        2'b11:q <= d[3];  
        default:q <= 0;  
    endcase  
end  
endmodule
```

```
module MUX4_1(  
    d,  
    s,  
    q  
);  
input wire [3:0]d;  
input wire [1:0]s;  
output reg      q;  
  
always@(*)  
begin  
    case(s)  
        2'b00:q <= d[0];  
        2'b01:q <= d[1];  
        2'b10:q <= d[2];  
        2'b11:q <= d[3];  
        default:q <= 0;  
    endcase  
end  
endmodule
```

Verilog 语法中，端口默认声明为 wire 型变量不用再次声明端口类型为 wire 型。但是，当端口有 reg 属性时，则 reg 声明不可省略。

3 模块例化

在一个模块中引用另一个模块，对其端口进行相关连接，叫做模块例化。模块例化建立了描述的层次。信号端

口可以通过位置或名称关联，端口连接也必须遵循一些语法规则。

模块例化时端口对应连接的方式有两种，第一种，例化的模块端口与外部信号按照其名字进行连接，端口顺序不一定一致，但要保证端口名字与外部信号匹配。第二种是按照模块声明时端口的顺序与外部信号进行匹配连接，位置要严格保持一致。例：

```
module MUX4_1(  
    d,  
    s,  
    q  
);  
input wire [3:0]d;  
input wire [1:0]s;  
output reg    q;  
  
always@(*)  
begin  
    case(s)  
        2'b00:q <= d[0];  
        2'b01:q <= d[1];  
        2'b10:q <= d[2];  
        2'b11:q <= d[3];  
        default:q <= 0;  
    endcase  
end  
endmodule
```

第一种：

```
MUX4_1 u_m4(  
    .s(data_s),  
    .d(data_d),  
    .q(data_q)  
);
```

第二种：

```
MUX4_1 u_m4(data_d,data_s,data_q);
```

端口连接时我们需要注意以下几点：

对于输入端口，可以连接 `wire` 或 `reg` 型变量。

对于输出端口，必须连接 `wire` 型变量。

对于输入输出端口，必须连接 `wire` 型变量。

端口是否可以悬空？`output` 端口可以悬空时，甚至可以在例化时将其删除。`input` 端口悬空时，逻辑功能表现为高阻状态。但是，例化时一般不能将悬空的 `input` 端口删除，否则编译会报错或者告警，一般来说，建议 `input` 端口不要做悬空处理，一般给定初始常量。

关于位宽匹配，当例化端口与连接信号位宽不匹配时，端口会通过无符号数的右对齐或截断方式进行匹配。

当例化多个相同的模块时，可以使用 `generate` 语句进行多个模块的重复例化。例：

```
genvar i;
```

```
generate
  for(i=1; i<=8; i=i+1)
    begin:gen
      MUX4_1 u_m4(
        .s(data_s[i]),
        .d(data_d[i]),
        .q(data_q[i])
      );
    end
  endgenerate
```

第十课 Verilog 语法_竞争与冒险

软件版本：无

操作系统：WIN10 64bit

硬件平台：适用所有系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑！

1 概述

本小节主要讲解 Verilog 语法竞争与冒险，需要掌握竞争与冒险的概念以及消除方法。

2 竞争与冒险

2.1 什么是竞争与冒险

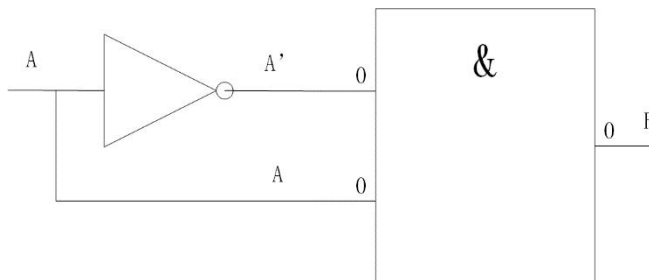
在数字电路设计中，信号传输或者状态变换时在硬件电路上都会有一定的延时。

在组合逻辑电路中，不同路径的输入信号变化传输到同一点门级电路时，在时间上有先有后，这种先后所形成的时间差导致输出产生不应有的尖峰干扰脉冲的现象叫做竞争。

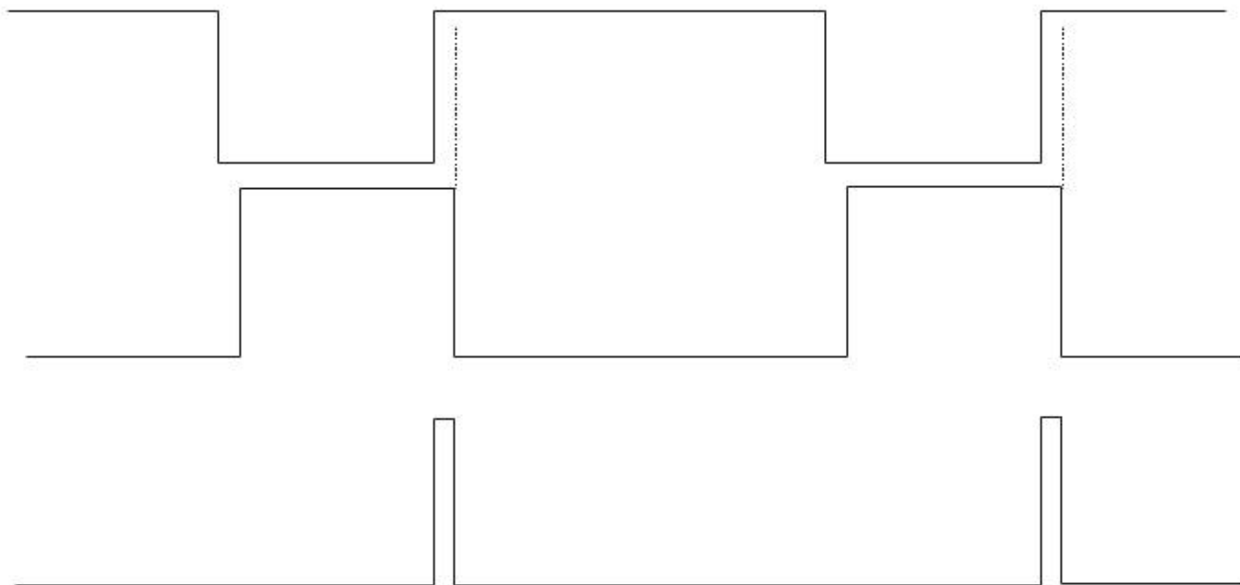
由于竞争的存在，输出信号需要经过一段时间才能达到期望状态，过渡时间内可能产生瞬间的错误输出，这种现象被称为冒险。

竞争不一定有冒险，但冒险一定会有竞争。

例如，对于给定逻辑 $F = A \& A'$ ，电路如图所示。



由于反相器电路的存在，信号 A' 传递到与门输入端的时间相对于信号 A 会滞后，这就可能导致与门最后的输出结果 F 会出现干扰脉冲。如下图所示，产生的干扰脉冲



2.2 如何消除竞争冒险

消除竞争冒险的几个方法：

1. 加封锁脉冲。在输入信号产生竞争冒险的时间内，引入一个脉冲将可能产生尖峰干扰脉冲的门封锁住。封锁脉冲应在输入信号转换前到来，转换结束后消失。
2. 加选通脉冲。对输出可能产生尖峰干扰脉冲的门电路增加一个接选通信号的输入端，只有在输入信号转换完成并稳定后，才引入选通脉冲将它打开，此时才允许有输出。在转换过程中，由于没有加选通脉冲，因此，输出不会出现尖峰干扰脉冲。
3. 接入滤波电容。由于尖峰干扰脉冲的宽度一般都很窄，在可能产生尖峰干扰脉冲的门电路输出端与地之间接入一个容量为几十皮法的电容就可吸收掉尖峰干扰脉冲。
4. 修改逻辑设计，增加冗余项。利用卡诺图，在两个相切的圆之间，增加一个卡诺圈，并加在逻辑表达式之中。
5. 同步电路信号的变化都发生在时钟边沿。对于触发器的 D 输入端，只要毛刺不出现在时钟的上升沿并且不满足数据的建立和保持时间，就不会对系统造成危害，因此可认为 D 触发器的 D 输入端对毛刺不敏感。利用此特性，在时钟边沿驱动下，对一个组合逻辑信号进行延迟打拍，可消除竞争冒险。延迟一拍时钟时，会一定概率的减少竞争冒险的出现。实践表明，最安全的打拍延迟周期是 3 拍，可有效减少竞争冒险的出现。
6. 采用格雷码计数器，计数时相邻的数之间只有一个数据 bit 发生了变化，所以能有效的避免竞争冒险。

第十一课 Verilog 语法_一般设计规范

软件版本: 无

操作系统: WIN10 64bit

硬件平台: 适用所有系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑!

1 概述

本小节讲解 Verilog 语法的一般设计规范, 需要掌握时序或组合电路设计中需要注意的几点, 掌握设计避免出现锁存器。

2 设计规范

上一节课我们讲解了竞争与冒险, 其实我们在编写代码中, 多注意设计规范, 能够减少大多数的竞争与冒险的问题。

在 verilog 编程时需要注意以下几点:

- (1) 时序电路建模时, 用非阻塞赋值 ($\leftarrow$)。
- (2) 锁存器电路建模时, 用非阻塞赋值 ($\leftarrow$)。
- (3) 用 always 和组合逻辑建模时, 用阻塞赋值 ($=$)。
- (4) 在同一个 always 块中建立时序和组合逻辑模型时, 用非阻塞赋值 ($\leftarrow$)。
- (5) 在同一个 always 块中不要既用阻塞赋值又用非阻塞赋值 ($\leftarrow$)。
- (6) 不要在多个 always 块中为同一个变量赋值。

阻塞赋值 “ $=$ ” 对应组合逻辑电路赋值, 并且会阻塞后面的赋值操作, 有先后顺序。非阻塞赋值 “ $\leftarrow$ ” 对应时序逻辑电路赋值, 所有非阻塞赋值操作在同一时刻进行赋值。

2.1 时序逻辑电路中, 对寄存器用非阻塞赋值。

时序电路中使用非阻塞赋值可以有效消除竞争与冒险。

比如下面代码描述, 由于无法确定 a 与 b 阻塞赋值的操作顺序, 也就是实际延迟时间未知, 就有可能带来竞争冒险。

```
always @(posedge clk)
begin
    a = b ;      //阻塞赋值, 关键词“ = ”
    b = a ;      //阻塞赋值, 关键词“ = ”
end
```

如果使用非阻塞赋值, 两个赋值操作是同时进行的, 因此就不会带来竞争冒险。例:

```
always @(posedge clk) begin
    a <= b ;    //非阻塞赋值, 关键词“ <= ”
    b <= a ;    //非阻塞赋值, 关键词“ <= ”
end
```

2.2 组合逻辑电路中，对寄存器用阻塞赋值。

设计实现 $C = A \& B$, $F = C \& D$ 的组合逻辑功能，也就是 $F = A \& B \& D$ ，用非阻塞赋值语句设计如下。

```
always @(*)
begin
    C <= A & B ; //非阻塞赋值, 关键词“ <= ”
    F <= C & D ; //非阻塞赋值, 关键词“ <= ”
end
```

两条赋值语句同时赋值， $F <= C \& D$ 中使用的是信号 C 的旧值，因此导致此时的逻辑是错误的， F 的逻辑值不等于 $A \& B \& D$ 。

而且，此时要求信号 C 具有存储功能，但不是时钟驱动，所以 C 可能会被综合成锁存器 (latch)，导致竞争冒险。

对代码进行如下修改， $F = C \& D$ 的操作一定是在 $C = A \& B$ 之后，此时 F 的逻辑值等于 $A \& B \& D$ ，符合设计。

```
always @(*)
begin
    C = A & B ; //阻塞赋值, 关键词“ = ”
    F = C & D ; //阻塞赋值, 关键词“ = ”
end
```

2.3 同一个 always 块中建立时序和组合逻辑模型时，用非阻塞赋值。

在时序电路中常常可能会涉及组合逻辑，但是如果赋值操作使用非阻塞赋值，会可能导致出现问题。

例：

```
always @(posedge clk or negedge rst_n)
begin
    if (!rst_n) begin
        q <= 1'b0; //非阻塞赋值, 关键词“ <= ”
    end
    else begin
        q <= a || b; //非阻塞赋值, 关键词“ <= ”
    end
end
```

2.4 同一个 always 块中不要既使用阻塞赋值又使用非阻塞赋值。

always 语句涉及的组合逻辑中，既有阻塞赋值又有非阻塞赋值时，可能会导致意外的结果。例：

```
always @(*) begin
    C = A & B ; //阻塞赋值, 关键词“ = ”
    F <= C & D ; //非阻塞赋值, 关键词“ <= ”，不能同时使用
end
```

此时信号 C 阻塞赋值完毕以后，信号 F 才会被非阻塞赋值，结果可能正确。

如果 F 信号有其他的负载， F 的最新值并不能马上传递出去，数据有效时间还是在下一个触发时刻。此时要

求 F 具有存储功能，可能会被综合成 latch，导致竞争与冒险。例：

```
always @(*)
begin
    C <= A & B ;           //非阻塞赋值，关键词“ <= ”
    F = C & D ;           //阻塞赋值，关键词“ = ”
end
```

信号 C 被非阻塞赋值，下一个触发时刻才会有效。而 $F = C \& D$ 虽然是阻塞赋值，但是信号 C 不是阻塞赋值，所以 F 逻辑中使用的还是 C 的旧值。

在时序电路里既有阻塞赋值，又有非阻塞赋值会怎样呢？例：

```
always @(posedge clk or negedge rst_n)
begin
    if (!rst_n)
    begin
        q <= 1'b0;         //非阻塞赋值，关键词“ <= ”
    end
    else
    begin
        q = a & b;         //阻塞赋值，关键词“ = ”
    end
end
```

假如复位信号与时钟同步，那么由于复位导致的信号 q 为 0，是在下一个时钟周期才有效。如果是信号 a 或 b 导致的 q 为 0，则在当期时钟周期内有效。假如 q 还有其他负载，就会导致 q 的时序特别混乱，显然不符合设计需求。

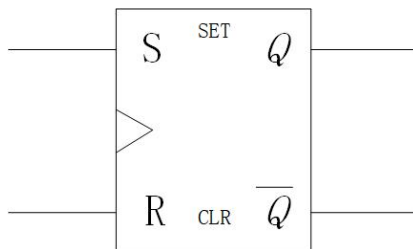
2.5 不允许在多个 always 块中为同一个变量赋值

Verilog 语法中不允许在多个 always 块中为同一个变量赋值。如果这样做了，该信号拥有多驱动源，这种行为是禁止的。当然，也不允许 assign 语句为同一个变量进行多次连线赋值。从信号角度来讲，多驱动时，同一个信号变量在很短的时间内进行多次不同的赋值结果，就有可能产生竞争冒险。从语法来讲，很多编译器检测到多驱动时，也会报错的。

2.6 设计避免产生 latch

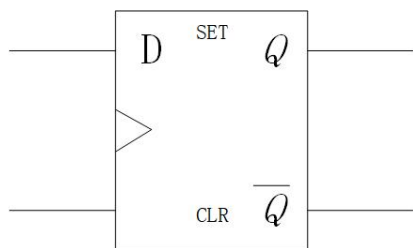
Latch 是锁存器，是一种对脉冲电平敏感的存储单元电路，它们可以在特定输入脉冲电平作用下改变状态。锁存，就是把信号暂存以维持某种电平状态。锁存器的最主要作用是缓存，其次完成高速的控制器与慢速的外设的不同步问题，再其次是解决驱动的问题，最后是解决一个 I/O 口既能输出也能输入的问题。锁存器是利用电平控制数据的输入，它包括不带使能控制的锁存器和带使能控制的锁存器。

如图所示：



触发器 (flip-flop)，是边沿敏感的存储单元，数据存储的动作 (状态转换) 由某一信号的上升沿或者下降沿进行同步的。

如图所示：



寄存器 (register)，在 FPGA 中用来暂时存放中间运算的数据和结果的变量。一个变量声明为寄存器时，它既可以被综合成触发器，也可能被综合成 Latch。但是大多数情况下我们希望它被综合成触发器，但是有时候由于代码书写问题，它会被综合成不期望的 Latch。

锁存器与触发器最大区别在于，锁存器是电平触发，而触发器是边沿触发。锁存器在不锁存数据时，输出随输入变化，但一旦数据锁存时，输入对输出不产生任何影响。

Latch 的主要危害有：

- (1) 输入状态可能多次变化，容易产生毛刺，增加了下一级电路的不确定性。
- (2) 在大部分 FPGA 的资源中，可能需要比触发器更多的资源去实现 Latch 结构。
- (3) 锁存器的出现使得静态时序分析变得更加复杂。

Latch 多用于门控时钟 (clock gating) 的控制，一般设计时，应当避免 Latch 的产生。

Latch 的产生主要有下面几种情况：

1、在组合逻辑中，不完整的 if-else 结构，会产生 latch。

避免此类 latch 的方法主要有两种，一种是补全 if-else 结构，或者对信号赋初值。

但是在时序逻辑中，不完整的 if-else 结构，不会产生 latch，这是因为，寄存器具有存储功能，且其值在时钟的边沿下才会改变。

2、在组合逻辑中，当 case 选项列表不全且没有加 default 关键字，或有多个赋值语句不完整时，也会产生 Latch。

消除此种 latch 的方法也是 2 种，将 case 选项列表补充完整，或对信号赋初值。

补充完整 case 选项列表时，可以罗列所有的选项结果，也可以用 default 关键字来代替其他选项结果。

总之，为避免 latch 的产生，在组合逻辑中，需要注意以下几点：

- 1) if-else 或 case 语句，结构一定要完整；
- 2) 不要将赋值信号放在条件判断中；
- 3) 敏感信号列表建议多用 always@(*)。

第十二课 Verilog 语法_函数与任务

软件版本：无

操作系统：WIN10 64bit

硬件平台：适用所有系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑！

1 概述

本小节讲解 Verilog 语法的函数与任务，需要掌握具体的 task 和 function 语句的使用方法。

2 函数与任务

task 和 function 说明语句分别用来定义任务和函数，利用任务和函数可以把一个很大的程序模块分解成许多较小的任务和函数便于理解和调试。

2.1 task 语句

如果传给任务的变量值和任务完成后接收结果的变量已定义，就可以用一条语句启动任务，任务完成后控制就传回启动过程。如任务内部有定时控制，则启动的时间可以与控制返回的时间不同。任务可以启动其他任务，其他任务又可以启动别的任务，可以启动的任务数量不受限制。不管有多少任务启动，只有当所有的启动任务完成以后，控制才能返回。

任务的定义如下：

```
task <任务名>;  
    <端口与数据类型声明语句>  
    <语句 1>  
    <语句 2>  
    <语句 3>  
    .....  
    <语句 n>  
endtask
```

任务的调用如下：

<任务名> (端口 1, 端口 2,, 端口 n) ;

例：

```
module xor  
    #(parameter      N = 4)      //参数 N 赋值，方便调整位宽  
    (  
        input        clk ,        //输入时钟  
        input        rstn ,        //输入复位  
        input [N-1:0] a ,
```

```
input [N-1:0]    b ,
output [N-1:0]   co
);

reg [N-1:0]      co_t ;
always @(*) begin
    xor_oper(a, b, co_t);
end

reg [N-1:0]      co_r ;
always @(posedge clk or negedge rstn) begin
    if (!rstn) begin
        co_r  <= 'b0 ;
    end
    else begin
        co_r  <= co_t ;
    end
end
assign          co = co_r ;

task xor_oper;                //关键词 task, 执行 task 下列任务, 返回参数
    input [N-1:0]  a;
    input [N-1:0]  b;
    output [N-1:0] co ;
    #30 co         = a ^ b ;
endtask

endmodule
```

2.2 function 语句

函数的目的是返回表达式的值。

定义函数的语法结构如下：

function<返回值的类型或范围>(函数名);

<端口说明语句>

<变量类型说明语句>

begin

<语句>

.....

end

endfunction

例：

```
function [7:0]out_adder;        //关键词 function, 执行 function 下列任务, 返回参数
input [7:0]in_data;
```

```
begin
    out_adder = in_data + 10 - 1;    //in_data+10-1 后结果返回给 out_adder
End+
endfunction
```

函数名是从函数返回的值。函数的定义把函数返回值所赋值寄存器的名称初始化为与函数同名的内部变量。

函数的调用格式如下：

<函数名> (<表达式>,<...>,<表达式>) 例：

```
function [7:0]out_adder;
input [7:0]in_data;
begin
    out_adder = in_data + 10 - 1;
end
endfunction

wire [7:0]out_data;
assign out_data = out_adder(8'h25);
```

函数的使用规则如下：

- (1) 函数的定义不能包含任何的时间控制语句，即任何用#，@，wait 等来标识的语句。
- (2) 函数不能启动任务。
- (3) 定义的函数至少包含一个输入变量。
- (4) 在函数的定义中必须有一条赋值语句给函数中的一个内部变量赋以函数的结果值，该内部变量具有和函数名相同的名字。

Verilog 中函数是不能进行递归调用的。设计模块中某些函数在不同的两个地方被同时并发调用，由于这两个调用同时对同一块地址空间进行操作，那么计算结果将是不确定的。如果在定义函数时使用了 **automatic**，那么该函数将成为自动的或可递归的，即仿真器为每一次函数调用动态的分配新的地址空间，每一个函数调用对各自的地址空间进行操作。

第十三课 Verilog 语法_仿真文件设计

软件版本: 无

操作系统: WIN10 64bit

硬件平台: 适用所有系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑!

1 概述

本小节讲解 Verilog 语法的仿真文件设计, 需要掌握 testbench 的建立方法。

2 仿真文件设计

当完成 verilog 工程设计后, 首先需要进行就是仿真文件的设计, 仿真文件又称为 testbench, 测试激励。

Testbench 文件内容往往包含以下几种:

- (1) 信号变量声明。
- (2) 时钟与复位信号的生成。
- (3) 被测试模块接口信号的波形激励生成。
- (4) 被测试模块的例化。
- (5) 被测试模块接口数据的校验模块。
- (6) 打印仿真信息。

例:

```
module top(  
    input  wire      i_clk,           //输入时钟  
    input  wire      i_rst_n,        //输入复位  
    output wire      o_gray_hsyn,     //输出 hs 信号  
    output wire      o_gray_vsyn,     //输出 vs 信号  
    output wire [7:0] o_gray_data,    //输出 data 数据信号  
    output wire      o_gray_de       //输出 de 像素有效信号  
);  
wire      i_hsyn;  
wire      i_vsyn;  
wire [7:0] i_r;  
wire [7:0] i_g;  
wire [7:0] i_b;  
wire      i_de;  
  
sim_image_tpg u_tpg(  
    .i_clk      (i_clk      ),  
    .o_hsyn     (i_hsyn     ),  
    .o_vsyn     (i_vsyn     ),  
    .o_en       (i_de       ),  
    .o_r        (i_r        ),
```

```
.o_g      (i_g      ),
.o_b      (i_b      )
);

gray2binary_algorithm u_algorithm
(
    .i_clk      (i_clk      ),
    .i_hsyn     (i_hsyn     ),
    .i_vsyn     (i_vsyn     ),
    .i_r        (i_r        ),
    .i_g        (i_g        ),
    .i_b        (i_b        ),
    .i_de       (i_de       ),
    .o_gray_hsyn (o_gray_hsyn ),
    .o_gray_vsyn (o_gray_vsyn ),
    .o_gray_data (o_gray_data ),
    .o_gray_de   (o_gray_de   )
);
endmodule

module gray2binary_algorithm
(
    input  wire      i_clk,
    input  wire      i_hsyn,
    input  wire      i_vsyn,
    input  wire [7:0] i_r,
    input  wire [7:0] i_g, //don't care
    input  wire [7:0] i_b, //don't care
    input  wire      i_de,
    output wire      o_gray_hsyn,
    output wire      o_gray_vsyn,
    output wire [7:0] o_gray_data,
    output wire      o_gray_de
);

//Binarization
assign o_gray_data = i_r >= 75 ? 255 : 0;

assign o_gray_hsyn = i_hsyn;
assign o_gray_vsyn = i_vsyn;
assign o_gray_de   = i_de;

endmodule

// `define VIDEO_1920_1080
// `define VIDEO_1680_1050
// `define VIDEO_1280_1024
`define VIDEO_1280_720 //选择分辨率
```

```

// `define VIDEO_1024_768
// `define VIDEO_800_600
// `define VIDEO_640_480
module sim_image_tpg(
    input  wire      i_clk,
    output reg       o_hsyn,
    output reg       o_vsyn,
    output reg       o_en,
    output reg [7:0] o_r,
    output reg [7:0] o_g,
    output reg [7:0] o_b
);

//1920x1080 148.5Mhz
`ifdef VIDEO_1920_1080
parameter H_ACTIVE      = 1920; // 行数据有效时间
parameter H_FRONT_PORCH = 88;  // 行消隐前肩时间
parameter H_SYNC_TIME   = 44;  // 行同步信号时间
parameter H_BACK_PORCH  = 148; // 行消隐后肩时间

parameter V_ACTIVE      = 1080; // 列数据有效时间
parameter V_FRONT_PORCH = 4;    // 列消隐前肩时间
parameter V_SYNC_TIME   = 5;    // 列同步信号时间
parameter V_BACK_PORCH  = 36;   // 列消隐后肩时间
`endif

//1680x1050 119Mhz
`ifdef VIDEO_1680_1050
parameter H_ACTIVE      = 1680; // 行数据有效时间
parameter H_FRONT_PORCH = 48;   // 行消隐前肩时间
parameter H_SYNC_TIME   = 32;   // 行同步信号时间
parameter H_BACK_PORCH  = 80;   // 行消隐后肩时间

parameter V_ACTIVE      = 1050; // 列数据有效时间
parameter V_FRONT_PORCH = 3;    // 列消隐前肩时间
parameter V_SYNC_TIME   = 6;    // 列同步信号时间
parameter V_BACK_PORCH  = 21;   // 列消隐后肩时间
`endif

//1280x1024 108Mhz
`ifdef VIDEO_1280_1024
parameter H_ACTIVE      = 1280; // 行数据有效时间
parameter H_FRONT_PORCH = 48;   // 行消隐前肩时间
parameter H_SYNC_TIME   = 112;  // 行同步信号时间
parameter H_BACK_PORCH  = 248;  // 行消隐后肩时间

```

```
parameter V_ACTIVE          = 1024; // 列数据有效时间
parameter V_FRONT_PORCH     = 1;    // 列消隐前肩时间
parameter V_SYNC_TIME       = 3;    // 列同步信号时间
parameter V_BACK_PORCH      = 38;   // 列消隐后肩时间
`endif

//1280X720 74.25MHZ
`ifdef VIDEO_1280_720
parameter H_ACTIVE          = 1280; // 行数据有效时间
parameter H_FRONT_PORCH     = 110;  // 行消隐前肩时间
parameter H_SYNC_TIME       = 40;   // 行同步信号时间
parameter H_BACK_PORCH      = 220;  // 行消隐后肩时间

parameter V_ACTIVE          = 720;  // 列数据有效时间
parameter V_FRONT_PORCH     = 5;    // 列消隐前肩时间
parameter V_SYNC_TIME       = 5;    // 列同步信号时间
parameter V_BACK_PORCH      = 20;   // 列消隐后肩时间
`endif

//1024x768 65Mhz
`ifdef VIDEO_1024_768
parameter H_ACTIVE          = 1024; // 行数据有效时间
parameter H_FRONT_PORCH     = 24;   // 行消隐前肩时间
parameter H_SYNC_TIME       = 136;  // 行同步信号时间
parameter H_BACK_PORCH      = 160;  // 行消隐后肩时间

parameter V_ACTIVE          = 768;  // 列数据有效时间
parameter V_FRONT_PORCH     = 3;    // 列消隐前肩时间
parameter V_SYNC_TIME       = 6;    // 列同步信号时间
parameter V_BACK_PORCH      = 29;   // 列消隐后肩时间

`endif

//800x600 40Mhz
`ifdef VIDEO_800_600
parameter H_ACTIVE          = 800; // 行数据有效时间
parameter H_FRONT_PORCH     = 40;   // 行消隐前肩时间
parameter H_SYNC_TIME       = 128;  // 行同步信号时间
parameter H_BACK_PORCH      = 88;   // 行消隐后肩时间

parameter V_ACTIVE          = 600; // 列数据有效时间
parameter V_FRONT_PORCH     = 1;    // 列消隐前肩时间
parameter V_SYNC_TIME       = 4;    // 列同步信号时间
parameter V_BACK_PORCH      = 23;   // 列消隐后肩时间
```

```
`endif

//640x480 25.175Mhz
`ifndef VIDEO_640_480
parameter H_ACTIVE      = 640; // 行数据有效时间
parameter H_FRONT_PORCH = 16 ; // 行消隐前肩时间
parameter H_SYNC_TIME   = 96 ; // 行同步信号时间
parameter H_BACK_PORCH  = 48 ; // 行消隐后肩时间

parameter V_ACTIVE      = 480; // 列数据有效时间
parameter V_FRONT_PORCH = 10 ; // 列消隐前肩时间
parameter V_SYNC_TIME   = 2  ; // 列同步信号时间
parameter V_BACK_PORCH  = 33 ; // 列消隐后肩时间
`endif

parameter H_TOTAL_TIME = H_ACTIVE + H_FRONT_PORCH + H_SYNC_TIME + H_BACK_PORCH; // 计算参数 H_TOTAL_TIME 的值
parameter V_TOTAL_TIME = V_ACTIVE + V_FRONT_PORCH + V_SYNC_TIME + V_BACK_PORCH; // 计算参数 V_TOTAL_TIME 的值

reg      en;
reg [12:0] h_syn_cnt = 'd0;
reg [12:0] v_syn_cnt = 'd0;
reg [ 7:0] image [0 : H_ACTIVE*V_ACTIVE-1];
reg [31:0] image_cnt = 'd0;

// 行扫描计数器
always@(posedge i_clk)
begin
    if(h_syn_cnt == H_TOTAL_TIME-1)
        h_syn_cnt <= 0;
    else
        h_syn_cnt <= h_syn_cnt + 1;
end

always@(posedge i_clk) // 列扫描计数器
begin
    if(h_syn_cnt == H_TOTAL_TIME-1) //h_syn_cnt 计数达到设定值 H_TOTAL_TIME 进入执行语句
    begin
        if(v_syn_cnt == V_TOTAL_TIME-1) //v_syn_cnt 判断, 目标值 V_TOTAL_TIM
            v_syn_cnt <= 0; //达到目标, 数值清零
        else
            v_syn_cnt <= v_syn_cnt + 1; //否则 v_syn_cnt+1
        end
    end
end
```

```

always@(posedge i_clk)                                // 行同步控制
begin
    if(h_syn_cnt < H_SYNC_TIME)                        // h_syn_cnt 未达到 H_SYNC_TIME 计数值
        o_hsyn <= 0;                                  // o_hsyn 信号控制置 0
    else
        o_hsyn <= 1;                                  // o_hsyn 信号控制置 1
End

always@(posedge i_clk)                                // 场同步控制
begin
    if(v_syn_cnt < V_SYNC_TIME)                        // v_syn_cnt 未达到 V_SYNC_TIME 计数值
        o_vsyn <= 0;                                  // o_vsyn 信号控制置 0
    else
        o_vsyn <= 1;                                  // o_vsyn 信号控制置 1
end

always@(posedge i_clk)                                // 坐标使能.
begin
    if(v_syn_cnt >= V_SYNC_TIME + V_BACK_PORCH && v_syn_cnt < V_SYNC_TIME + V_BACK_PORCH + V_ACTIVE)
    begin
        if(h_syn_cnt >= H_SYNC_TIME + H_BACK_PORCH && h_syn_cnt < H_SYNC_TIME + H_BACK_PORCH + H_ACTIVE)
            en <= 1;
        else
            en <= 0;
    end
    else
        en <= 0;
end

//读取 txt 文件到 image 数组中
initial begin
    $readmemh("../matlab_src/image_720_1280_1.txt", image);
end

always@(posedge i_clk)
begin
    if(en)
    begin
        o_r <= image[image_cnt];                      // image 中存储的图像数据依次写入 o_r
        image_cnt <= image_cnt + 1;                  // 输出一个像素数据 image_cnt+1, 输入下一个
    end
    else if(image_cnt == H_ACTIVE*V_ACTIVE) // image_cnt 的值增加到 H_ACTIVE*V_ACTIVE 时为一帧图像
    begin
        o_r <= 8'h00;                                // o_r 数据清零
        image_cnt <= 'd0;                             // image_cnt 计数清零
    end
end

```

```
    else
    begin
        o_r      <= 8'h00;
        image_cnt <= image_cnt;
    end
end

always@(posedge i_clk)
begin
    o_en <= en;
end
endmodule

module top_tb;

reg      clk;
reg      rst_n;

integer  image_txt;

reg [31:0] pixel_cnt;
wire[7:0]  data;
wire      de;

top u_top
(
    .i_clk      (clk      ),
    .i_rst_n    (rst_n    ),
    .o_gray_data (data     ),
    .o_gray_de  (de       )
);

always #(1) clk = ~clk;

initial
begin
    clk  = 1;
    rst_n = 0;
    #100
    rst_n = 1;
end

initial
```

```
begin
    image_txt = $fopen("../matlab_src/image_720_1280_3_out.txt");    //打开存有图像数据的.txt 文件
end

always@(posedge clk or negedge rst_n)    //敏感列表
begin
    if(!rst_n)
    begin
        pixel_cnt <= 0;    //pixel_cnt 计数清零
    end
    else if(de)    //像素有效信号拉高时, pixel_cnt+1
    begin
        pixel_cnt = pixel_cnt + 1;
        $fwrite(image_txt,"%h\n",data);
    end
end

always@(posedge clk )
begin
    if(pixel_cnt == 720*1280)
    begin
        $display("*****");
        $display("*** Success:image_720_1280_3_out.txt is output complete! %t", $realtime, "ps***");
        $display("*****");
        $fclose(image_txt);
        $stop;
    end
end
endmodule
```

2.1 信号声明

testbench 模块声明时，一般不需要声明端口。因为激励信号一般都在 testbench 模块内部，没有外部信号。

声明的变量应该能全部对应被测试模块的端口。当然，变量不一定要与被测试模块端口名字一样。但是被测试模块输入端对应的变量应该声明为 reg 型，如 时钟，复位等，输出端对应的变量应该声明为 wire 型，如 data，de。

2.2 时钟生成

生成时钟的方式有很多种，例如以下两种生成方式也可以借鉴。

例：

```
always #(1) clk = ~clk;    //时钟翻转
initial
begin
    clk = 1;    //时钟赋初值
```

```
rst_n = 0;           //复位赋初值

#100

rst_n = 1;           //复位赋完成，恢复高电平

end
```

需要注意的是，利用取反方法产生时钟时，一定要给 clk 寄存器赋初值。

2.3 复位生成

复位逻辑比较简单，一般赋初值为 0，再经过一段小延迟后，复位为 1 即可。

例：

```
initial
begin
    clk    = 1;
    rst_n = 0;
    #100
    rst_n = 1;

end
```

2.4 激励部分

激励部分该产生怎样的输入信号，是根据被测模块的需要来设计的。本例中采用读取已有的图像数据，按照时序要求读到程序中，生成激励波形。

```
initial begin
    $readmemh("../matlab_src/image_720_1280_1.txt", image);

end
```

2.5 模块例化

利用 testbench 开始声明的信号变量，对被测试模块进行例化连接。

```
top u_top
(
    .i_clk          (clk          ),
    .i_rst_n        (rst_n        ),
    .o_gray_data    (data          ),
    .o_gray_de      (de            )
);
```

2.6 打印信息

```
always@(posedge clk )
begin
    if(pixel_cnt == 720*1280)
    begin
```

```

$display("*****");
$display("*** Success:image_720_1280_3_out.txt is output complete! %t", $realtime, "ps***");
$display("*****");
    $fclose(image_txt);
$stop;
end
end

```

通过打印关键仿真信息，能让我们自动化操作仿真过程。

第十四课 Verilog 语法_流水线设计

软件版本：无

操作系统：WIN10 64bit

硬件平台：适用所有系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑！

1 概述

本小节讲解 Verilog 语法的流水线设计，需要掌握流水线的设计方法。

2 流水线简介

2.1 什么是流水线

流水线的基本思想是：把一个重复的过程分解为若干个子过程，每个子过程由专门的功能元件来实现。将多个处理过程在时间上错开，依次通过各功能段，这样每个子过程就可以与其他子过程并行进行。

例：小张在快递站工作，日常任务就是打包快递，打包快递分为四个步骤，称重、打包、贴单和入库，每个步骤耗时 5 分钟。临近双十一，快递寄送任务变多，小张一个人忙不过来，临时添加三个人一起打包快递，每人负责一个步骤，原先打包 4 个快递需要 80 分钟，现在需要 35 分钟。如图所示：



2.2 设计流水线

设计一个 32 位的全加器，使用四级流水线。例：

```
module full_add (
    input      rst_n,
    input      clk,
    input  [31:0] add_a,
    input  [31:0] add_b,
    input      c_in,
    input      vld,
    output  [31:0] add_sum,
    output      c_out,
    output      sum_vld
);

reg      c_in_ff;
reg [31:0] add_a_ff;
reg [31:0] add_b_ff;
reg [31:0] add_a_1;
reg [31:0] add_b_1;
reg [31:0] add_a_2;
reg [31:0] add_b_2;
reg [31:0] add_a_3;
reg [31:0] add_b_3;

reg      vld_temp;
reg      vld_1;
reg      vld_2;
reg      vld_3;
reg      sum_vld;
reg      cout1;
reg      cout2;
reg      cout3;
reg      c_out;
reg [7:0] sum1;
reg [15:0] sum2;
reg [23:0] sum3;
reg [31:0] add_sum;

always@(posedge clk or negedge rst_n) //输入数据的四级缓存
begin
    if (!rst_n)
        begin
            c_in_ff  <= 'd0;
            add_a_ff <= 'd0;
            add_a_1  <= 'd0;
            add_a_2  <= 'd0;
```

```

    add_a_3 <= 'd0;
    add_b_ff <= 'd0;
    add_b_1 <= 'd0;
    add_b_2 <= 'd0;
    add_b_3 <= 'd0;
    vld_temp <= 'd0;
    vld_1 <= 'd0;
    vld_2 <= 'd0;
    vld_3 <= 'd0;
    sum_vld <= 'd0;

end

else
begin
    c_in_ff <= c_in;
    add_a_ff <= add_a;
    add_a_1 <= add_a_ff;
    add_a_2 <= add_a_1;
    add_a_3 <= add_a_2;
    add_b_ff <= add_b;
    add_b_1 <= add_b_ff;
    add_b_2 <= add_b_1;
    add_b_3 <= add_b_2;
    vld_temp <= vld;
    vld_1 <= vld_temp;
    vld_2 <= vld_1;
    vld_3 <= vld_2;
    sum_vld <= vld_3;

end

end

always@(posedge clk or negedge rst_n) //四级流水线设计
begin
    if (!rst_n)
    begin
        sum1 <= 8'b0;
        sum2 <= 16'd0;
        sum3 <= 24'd0;
        add_sum <= 32'd0;
        cout1 <= 1'b0;
        cout2 <= 1'b0;
        cout3 <= 1'b0;
        c_out <= 1'b0;

    end

    else
    begin
        {cout1,sum1} <= 9'b0+add_a_ff[7:0] +add_b_ff[7:0] +c_in_ff;
        {cout2,sum2} <= {9'b0+add_a_1[15:8] +add_b_1[15:8] +cout1,sum1};
        {cout3,sum3} <= {9'b0+add_a_2[23:16] +add_b_2[23:16] +cout2,sum2};
    end
end

```

```
        {c_out,add_sum }   <= {9'b0+add_a_3[31:24] +add_b_3[31:24] +cout3,sum3};  
  
    end  
  
end  
  
endmodule
```

流水线设计就是将路径系统的分割成一个个数字处理单元，并在各个处理单元之间插入寄存器来暂存中间阶段的数据。被分割的单元能够按阶段并行的执行，相互间没有影响。所以最后流水线设计能够提高数据的吞吐率，即提高数据的处理速度。

第十五课 Verilog 语法_同步与异步设计

软件版本：无

操作系统：WIN10 64bit

硬件平台：适用所有系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑！

1 概述

本小节主要讲解 Verilog 语法的同步与异步设计，需要掌握同步时钟和异步时钟的设计方法。

2 同步时钟

数字电路设计中，一般认为，频率相同或频率比为整数倍、且相位相同或相位差为固定的两个时钟称为同步时钟。换句话说，时钟同源且频率比为整数倍的两个时钟为同步时钟。

大致有三类同步时钟：

（1）同源同频率同相位

时钟频率和相位均相同，是同步的。时钟间数据传输只要满足正常的建立时间和保持时间即可，不需要特殊的同步设计。

（2）同源同频不同相位

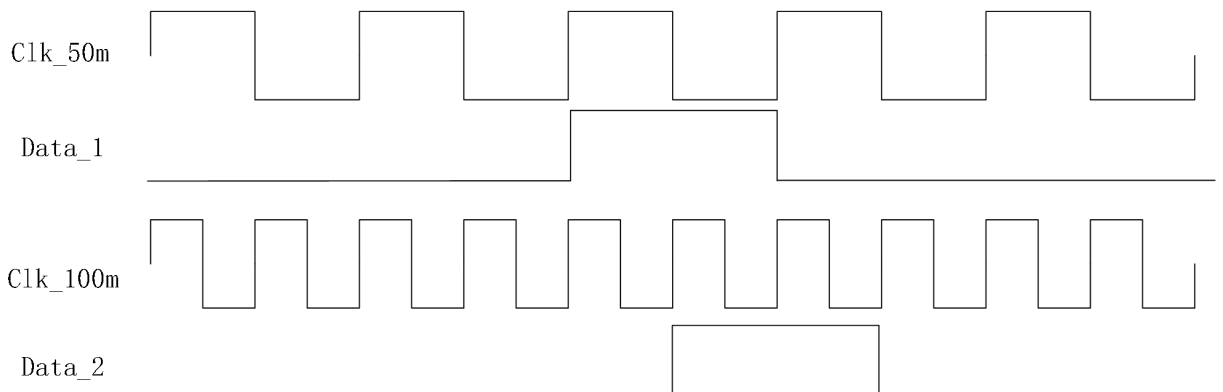
两个时钟同频但不同相位时，只要相位差保持固定，也可以认为是同步的。因为只要控制两个时钟间传输的数据延迟在合理范围内，就不会导致时序问题。固定的相位差可以理解为同源时钟下两个时钟因路径不同而导致的偏斜。

（3）同源不同频但存在整数倍分频比

一个时钟往往是另一个时钟的分频，即便存在相位差也是固定的。

当单 bit 信号从慢时钟域传递到快时钟域时，因为同源，只要满足建立时间和保持时间，快时钟域总会采集到从慢时钟域传递来的信号。

例：



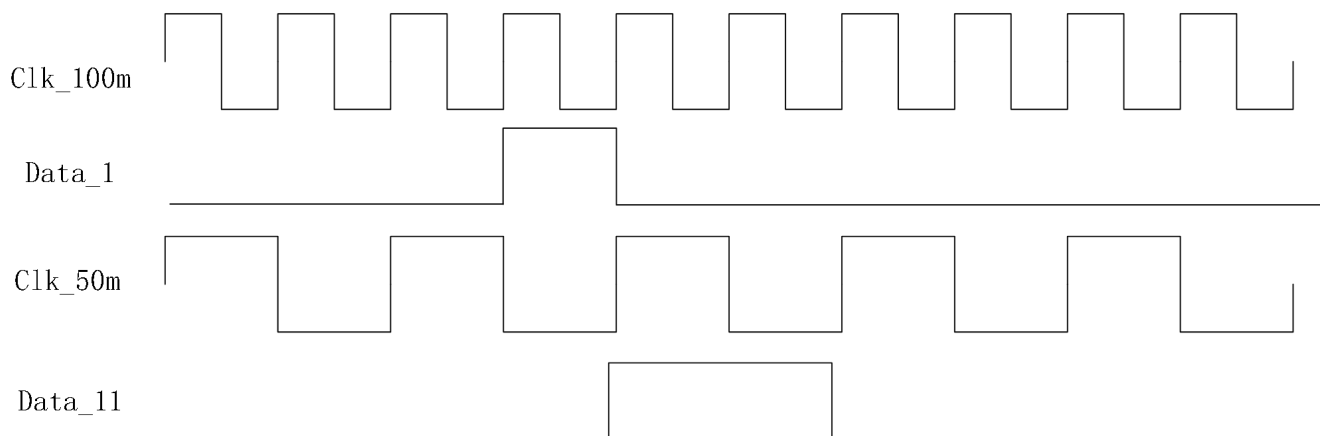
如果要求 clk_100m 域的信号 data_2 只需要持续一个时钟周期，则需要对 data_1 进行上升沿检测。例：

```
reg [1:0] data_ff ;  
always @(posedge clk_100m) //按照 clk_100m 的时钟进行采样
```

```
begin
    data_ff <= {data_ff[0], data_1} ;           //对 data_1 的数据进行寄存
End                                              //data_ff[0]表示最新采样的数据，!data_ff[1]为上一个时钟周期采样的数据
assign data_2 = !data_ff[1] && data_ff[0];      //最新的电平为 1，之前的电平为 0，判断为上升沿检测
```

当单 bit 信号从快时钟域传递到慢时钟域时，只要慢时钟域能安全采集到从快时钟域传递来的信号，就不存在异步问题。

例：



但是如果快时钟域信号过窄，慢时钟域可能会漏掉该信号，如上图 data_1 到 data_11 的传输。此时就需要对快时钟域的窄脉冲信号进行展宽。例：

```
reg [1:0]    data_ff ;
reg          data_1f ;
always @(posedge clk_100m)           //按照 clk_100m 的时钟进行采样
begin
    data_ff <= {data_ff[0], data_1} ; //对 data_1 的数据进行寄存
end
always @(posedge clk_50m)           //按照 clk_50m 的时钟进行采样，属于慢速时钟域采样
begin
    data_1f <= |data_ff ;           //窄脉冲信号进行缩减或操作
end
assign data_11 = data_1f ;          //data_1f 采样信号赋值给 data_11
```

3 异步时钟

异步时钟主要分为三类：

(1) 非同源时钟

由两个不同的时钟源产生的两个时钟是异步的。即便两个时钟频率相同，但是也不能保证每次上电后两者的相位或相位差是相同的，所以信号间的传输与时钟关系是不确定的。

(2) 同源但频率比不是整数倍

此时两个时钟间相位差也可能会有多个，例如同源的 63MHz 时钟和 24MHz 时钟，它们之间也会出现多个相位差，一般情况下进行异步时钟处理。

(3) 同源虽频率比为整数倍但不满足时序要求

上一小节介绍同步时钟时，当信号从快时钟域传递到慢时钟域时，只要慢时钟域能安全采集到从快时钟域传来

的信号，就不存在异步问题。但如果信号在快时钟域翻转速率过快，慢时钟域可能不会安全的采集到从快时钟域传来的信号，这可以认为是异步问题。

第十六课 Verilog 语法_跨时钟域设计

软件版本: 无

操作系统: WIN10 64bit

硬件平台: 适用所有系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑!

1 概述

本小节主要讲解 Verilog 语法的跨时钟域设计, 需要掌握跨时钟域时快慢时钟之间信号是如何同步的。

2 跨时钟域慢速到快速时钟

由慢时钟到快时钟的信号传递, 就传输的信号位宽一般分为两种, 单比特信号和多比特信号。下面我们分开进行讨论。

2.1 单比特信号

上一节课同步异步设计有提到慢时钟到快时钟的设计方法, 一般分析, 快时钟域的信号总能采集到慢时钟域的信号, 但是如果存在异步时钟域可能会导致采样数据出错, 因此需要进行时钟同步处理。同步处理针对单比特信号采取打三拍进行同步。

异步时钟域的信号从一个时钟域进入另一个时钟域之前, 将该信号用三级触发器缓存两次, 可有效降低因为时序而导致的亚稳态问题。例:

```
module async_data(  
    input      clk_1,                //输入时钟 clk_1  
    input      data_1,               //输入数据 data_1  
    input      clk_2,                //输入时钟 clk_2  
    output     data_2                //输出数据 data_2  
);  
  
reg [2:0]    data_ff ;               //寄存器变量 data_ff  
always @(posedge clk_2)              //clk_2 敏感 posedge, 上升沿触发  
begin  
    data_ff <= {data_ff[1:0], data_1}; //data_1 数据采样  
End                                     //data_ff[0]表示最新采样的数据, !data_ff[1]为上一个时钟周期采样的数据  
assign data_2 = !data_ff[2] && data_ff[1]; //最新的电平为 1, 之前的电平为 0, 判断为上升沿检测
```

clk_1 为慢时钟, clk_2 为快时钟, 将 data_1 在快时钟域进行三拍缓存, 检测信号的上升沿, 此时将信号进行了同步化。

2.2 多比特信号

对于多比特信号的传输, 我们要考虑两种情况, 即单时钟周期突发信号和多时钟周期连续信号的传输。

针对单时钟周期突发信号我们可以采用利用打拍法延迟采样。例：

```

module async_data_1(
    input          clk_1,          //输入时钟 clk_1
    input [7:0]     data_in,        //输入数据 data_1
    input          din_en,          //输入突发信号
    input          clk_2,          //输入时钟 clk_2
    output reg[7:0] data_out,        //输出数据 data
    output reg      dout_en         //输出突发数据有效信号
);

reg [2:0]  din_en_ff ;
wire din_en_pos;

always @(posedge clk_2 )
begin
    din_en_ff  <= {din_en_ff[1:0], din_en} ;    //输入突发信号打拍缓存 2 拍
end

assign din_en_pos = !din_en_ff[2] && din_en_ff[1] ; //din_en_ff[1]为新缓存的信号，为 1 时有效，din_en_ff[2]为上一拍缓存的信号
                                                    //为 0 时有效，所以 din_en_pos 为捕捉 din_en 信号上升沿

always @(posedge clk_2)
begin
    if (din_en_pos)
        data_out  <= din ;                //din_en_pos 信号拉高，说明数据有效，将输入数据输出
    else
        ;
end

always @(posedge clk_2 )
begin
    dout_en <= din_en_pos ;                //输出突发数据有效信号为 din_en_pos 信号
end

endmodule

```

针对多时钟周期连续信号我们可以采用检测慢时钟的边沿进行采样。如果两个时钟的频率相差较小，可能还需要对数据进行延迟缓存，以保证采集到的是当拍时钟的数据；如果两个时钟的频率相差较大，数据采样时刻可以通过计数的方法获得，而不用对数据进行缓存。例：

```

module async_data_2(
    input          clk_1,
    input [7:0]     data_in,
    input          din_en,
    input          clk_2,
    output reg[7:0] data_out,
    output reg      dout_en
);

```

```
reg [3:0]    din_en_ff ;
wire din_edge_pos;
reg [3:0] cnt ;

always @(posedge clk_2 )
begin
    din_en_ff  <= {din_en_ff[2:0], clk_1} ;
end

assign din_edge_pos = !din_en_ff[2] && din_en_ff[1] ;

always @(posedge clk_2)                                //clk_2 为驱动时钟，进行计数
    if (din_edge_pos & din_en)                          //din_edge_pos 信号为捕捉到上升沿，且 din_en 信号持续拉高
        cnt <= 4'h0 ;
    else if (cnt != 4'hf)                                //计数值为 4'hf
        cnt <= cnt + 1'b1 ;
end

always @(posedge clk_2)                                //clk_2 为驱动时钟
begin
    if (din_en && cnt == 7)                              //din_en 信号持续拉高并且完成 7 次计数时，确认数据有效
        data_out  <= din ;                              //数据完成赋值
    else
        ;
end

always @(posedge clk_2 )
begin
    if(din_en && cnt == 8)                                //din_en 信号持续拉高并且完成 8 次计数时，确认数据有效
        dout_en <= 1 ;                                   //输出突发数据有效信号进行拉高
    else
        dout_en <= 0;
end

endmodule
```

3 跨时钟域快速到慢速时钟

由快时钟到慢时钟的信号传递，就传输的信号位宽一般分为两种，单比特信号和多比特信号。下面我们分开进行讨论。

3.1 单比特信号

由快时钟到慢时钟的单比特信号传递，我们需要考虑信号本身是单时钟周期的脉冲信号还是连续多个周期的电平信号。针对单时钟周期的脉冲信号，最好的传送方法是握手法，假设单时钟周期脉冲信号的高电平为有效信号，其基本原理如下。

(1) 快时钟域对脉冲信号进行检测，检测有高电平时输出一个高电平信号。快时钟域输出高电平信号时，保持输出信号为高电平状态。

(2) 慢时钟域对快时钟域的检测信号进行延迟打拍采样。因为此时的脉冲信号被快时钟域保持拉高状态，延迟打拍会采集到该信号。

(3) 慢时钟域确认采样得到高电平检测信号后，再反馈信号给快时钟域。

(4) 快时钟域对反馈信号进行延迟打拍采样。如果检测到反馈信号为高电平，证明慢时钟域已经接收到有效的高电平信号。如果此时快时钟域自身逻辑不再要求脉冲信号为高电平状态，拉低快时钟域的脉冲信号即可。

此方法是通过相互握手的方式对窄脉冲信号进行脉宽扩展。例：

```
module pulse_fast_2_slow(
input          rst_n,                //复位信号
input          clk_fast,             //快时钟
input          pulse_fast,           //快时钟握手信号
input          clk_slow,             //慢时钟
output         pulse_slow            //慢时钟握手信号
);

wire           clear_up ;
reg            pulse_fast_detect ;
reg [1:0]      pulse_fast_detect_ff ;
reg [1:0]      pulse_slow_2_fast ;

always@(posedge clk_fast or negedge rst_n)
begin
    if (!rst_n)
        pulse_fast_detect <= 1'b0 ;           //pulse_fast_detect 拉低复位
    else if (clear_up)
        pulse_fast_detect <= 1'b0 ;           //pulse_fast_detect 拉低复位
    else if (pulse_fast)                    //在快时钟域中对快时钟握手信号进行抓取
        pulse_fast_detect <= 1'b1 ;           //信号抓取后，将 pulse_fast_detect 持续拉高，可能为多个时钟周期，方便慢时钟域捕捉
end

always@(posedge clk_slow or negedge rst_n)
begin
    if (!rst_n)
        pulse_fast_detect_ff <= 3'b0 ;        //缓存三拍清零
    else
        pulse_fast_detect_ff <= {pulse_fast_detect_ff[0], pulse_fast_detect} ; //在慢时钟域中对 pulse_fast_detect 信号进行抓取
End                                           //不要一翻转立刻捕捉到，只要求延时捕捉

assign pulse_slow = pulse_fast_detect_ff[1] ; //取中间一拍进行输出给快时钟域

always@(posedge clk_fast or negedge rst_n)
begin
```

```
if (!rst_n)
    pulse_slow_2_fast <= 1'b0 ;
else
    pulse_slow_2_fast <= {pulse_slow_2_fast[0], pulse_slow};           //快时钟域对慢时钟握手信号进行捕捉
end

assign clear_up = !pulse_fast && pulse_slow_2_fast[1];                //捕捉到回应, 将 pulse_fast_detect 拉低
endmodule
```

3.2 多比特信号

当多位宽数据进行同步时, 如果数据变化速率过快, 就不能再使用延迟打拍采样的方法。因为此时数据各 bit 位变化的时间参差不齐, 用异步时钟进行打拍采样, 可能会采集到因路径延迟不同而导致的错误数据。解决此类异步问题的常用方法是采用异步 FIFO (First In First Out)进行数据的交换。异步 FIFO 在后面的一节课专门介绍。

第十七课 Verilog 语法_复位设计

软件版本：无

操作系统：WIN10 64bit

硬件平台：适用所有系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑！

1 概述

本小节讲解 Verilog 语法的复位设计，需要掌握复位电路的同步复位、异步复位、异步复位同步化和异步复位同步释放。

2 复位电路简介

为确保系统上电后有一个明确、稳定的初始状态，或系统运行状态紊乱时可以恢复到正常的初始状态，数字系统设计中一定要有复位电路模块。复位电路异常可能会导致整个系统的功能异常，所以在一定程度上，复位电路的重要性也不亚于时钟电路。

复位电路可分类为同步复位、异步复位、异步复位同步化和异步复位同步释放。

2.1 同步复位

同步复位是指复位信号在时钟有效边沿到来时有效。如果没有时钟，无论复位信号怎样变化，电路也不执行复位操作。

同步复位的代码描述如下：

```
module sync_reset(  
    input      rst_n,  
    input      clk,  
    input      din,  
    output reg dout  
);  
  
always @(posedge clk)      //敏感列表只有时钟，没有时钟，复位也不会被执行  
begin  
    if(!rst_n)              //该复位为 clk 时钟的同步时钟复位信号  
        dout <= 1'b0 ;  
    else  
        dout <= din ;  
end  
  
endmodule
```

同步复位的优点：信号间是同步的，能滤除复位信号中的毛刺，有利于时序分析。

同步复位的缺点：复位信号的宽度必须大于一个时钟周期，否则可能会漏掉复位信号。

2.2 异步复位

异步复位是指无论时钟到来与否，只要复位信号有效，电路就会执行复位操作。

异步复位的代码描述如下：

```
module async_reset(  
    input      rst_n,  
    input      clk,  
    input      din,  
    output reg dout  
);  
  
always @(posedge clk or negedge rst_n) //敏感列表有时钟和复位，两者都能触发下列执行语句  
begin  
    if(!rstn) //该复位可以为异步复位，同样符合敏感列表 negedge rst_n 要求，进行复位  
        dout <= 1'b0 ;  
    else  
        dout <= din ;  
    end  
  
endmodule
```

异步复位的优点：异步复位信号不经过处理直接引用，设计相对简单，信号识别快速方便。

异步复位的缺点：复位信号与时钟信号无确定的时序关系，异步复位很容易引起时序上 removal 和 recovery 的不满足。且异步复位容易受到毛刺的干扰，产生意外的复位操作。

2.3 异步复位同步化

异步复位同步化是指先将复位信号进行同步化操作，同步后进行异步复位操作。例：

```
module async_reset_sync(  
    input      rst_n,  
    input      clk,  
    input      din,  
    output reg dout  
);  
  
reg rst_n_f1, rst_n_f2;  
always@(posedge clk )  
begin  
    if (!rst_n)  
    begin  
        rst_n_f1 <= 1'b0;  
        rst_n_f2 <= 1'b0;  
    end  
    else  
    begin  
        rst_n_f1 <= rst_n; //使用同步时钟对异步复位进行采集  
    end  
end
```

```
rst_n_f2 <= rst_n_f1;           //此时的 rst_n_f2 为 rst_n 的通过同步时钟采集的信号，既满足复位的要求，又满足同步时钟

end

end

always@(posedge clk or negedge rst_n_f2)    //此处虽然依然为异步复位写法，但是 rst_n_f2 是同步时钟采集到的复位，已同步化
begin
    if (!rst_n_f2)
        dout <= 1'b0;
    else
        dout <= din;
end

endmodule
```

2.4 异步复位同步释放

大多数数字系统设计时都会使用异步复位电路。为消除异步复位的缺点，复位电路往往会采用异步复位同步释放的设计方法。即复位信号到来时不受时钟信号的同步，复位信号释放时需要进行时钟信号的同步。

异步复位同步释放的代码描述如下：

```
module async_reset_sync_release(
input      rst_n,
input      clk,
input      din,
output reg dout
);

reg  rst_n_f1, rst_n_f2;
always@(posedge clk or negedge rst_n)
begin
    if (!rst_n)
    begin
        rst_n_f1 <= 1'b0;    //异步复位到来时，也是同样采用低电平
        rst_n_f2 <= 1'b0;    //异步复位到来时，也是同样采用低电平
    end
    else
    begin
        rst_n_f1 <= 1'b1;    //异步复位结束时，由同步时钟进行一定的延时，产生两拍的延时信号
        rst_n_f2 <= rst_n_f1; //既满足异步时钟的产生要求，又可以延长异步时钟，让他满足一定的时钟周期再结束
    end
end

always@(posedge clk or negedge rst_n_f2)
begin
    if (!rst_n_f2)
        dout <= 1'b0;
    else

```

```
dout <= din;  
  
end  
  
endmodule
```

第十八课 Verilog 语法_时钟分频设计

软件版本：无

操作系统：WIN10 64bit

硬件平台：适用所有系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑！

1 概述

本小节讲解 Verilog 语法的时钟分频设计，需要掌握时钟的特性，以及如何时钟分频设计。

2 时钟分频

在 FPGA 的硬件电路设计中，PCB 板上需要有时钟产生源，常见的外部时钟源有 RC/LC 振荡电路，无源/有源晶体振荡器，利用石英晶体的压电效应产生谐振信号。此类时钟源频率精度高，稳定性好，噪声低，温漂小。有源晶振中，往往还加入了压控或温度补偿，时钟的相位和频率都有较好的特性。

有时我们在时序电路设计中，有些模块工作频率会低于外部时钟频率，此时就需要对时钟进行一定的分频得到频率较低的时钟。在 FPGA 内部常常存在一定的 PLL（锁相环，Phase Locked Loop）资源，它可以对时钟进行分频或者倍频的操作。

本小节主要介绍如何使用寄存器等资源实现时钟偶数分频、奇数分频和小数分频的。

2.1 偶数分频

偶数分频直接使用寄存器进行取反操作，可以得到二分频、四分频、六分频、八分频等，且占空比是 50%。例：

```
module even_div
(
    input    rst_n,           //输入复位
    input    clk,             //输入时钟
    output   clk_div          //分频时钟
);
parameter DIV_NUM = 6;      //分频参数

reg [15:0] clk_cnt;
reg clk_out;

always @(posedge clk or negedge rst_n)
begin
    if (!rst_n)
    begin
        clk_cnt    <= 'b0 ;
    end
    else if(clk_cnt == ((DIV_NUM/2) -1))
```

```
begin
    clk_cnt    <= 'b0;
end
else
begin
    clk_cnt    <= clk_cnt + 1'b1;           //分频计数器
end
end

always @(posedge clk or negedge rst_n)
begin
    if (!rst_n)
    begin
        clk_out    <= 'b0 ;
    end
    else if(clk_cnt == ((DIV_NUM/2) -1))    //计数到分频参数的一半
    begin
        clk_out    <= ~clk_out;           //产生翻转
    end
    else
    begin
        clk_out    <= clk_out;           //否则保持不变
    end
end

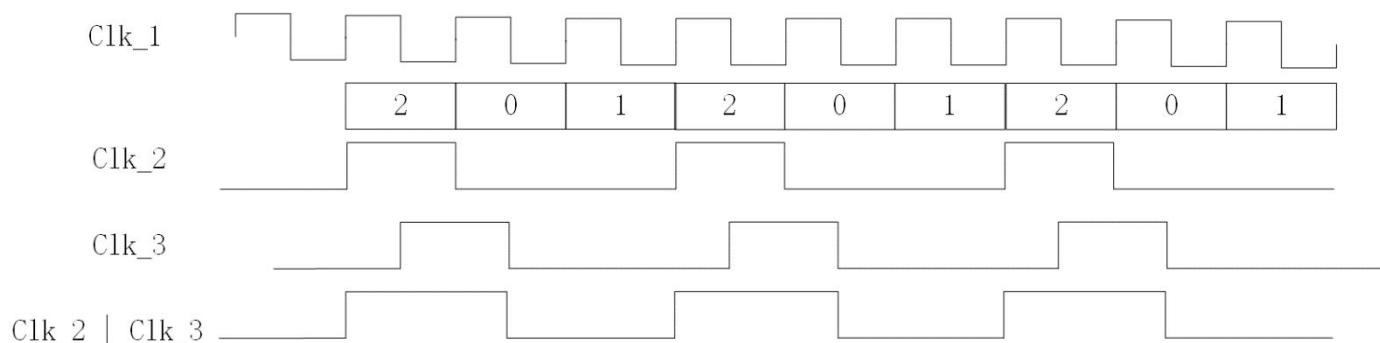
assign clk_div = clk_out;

endmodule
```

2.2 奇数分频

如果奇数分频不要求占空比是 50%，可以按照类似的偶数分频进行处理。如果奇数分频要求占空比是 50%，我们以三分频为例进行讲解如何进行奇数分频。

时序如图所示：



例：

```
module odd_div(
```

```
input          rst_n,
input          clk,
output         clk_div
);

parameter DIV_NUM = 3;

reg [15:0] clk_cnt;
reg clk_out;
reg clk_div_1;
reg clk_div_2;

always @(posedge clk or negedge rst_n)
begin
    if (!rst_n)
    begin
        clk_cnt    <= 'b0;
    end
    else if (clk_cnt == (DIV_NUM - 1))           //产生计数
    begin
        clk_cnt    <= 'b0;                       //clk_cnt 清零复位
    end
    else
    begin
        clk_cnt    <= clk_cnt + 1'b1;           //clk_cnt 为 0、1、2
    end
end

always @(posedge clk or negedge rst_n)         //clk 上升沿触发
begin
    if (!rst_n)
    begin
        clk_div_1 <= 1'b0;
    end
    else if (clk_cnt == (DIV_NUM >> 1)-1 )       //clk_cnt 为 0 时, clk_div_1 为 0
    begin
        clk_div_1 <= 1'b0;
    end
    else if (clk_cnt == DIV_NUM-1)               //clk_cnt 为 2 时, clk_div_1 为 1
    begin
        clk_div_1 <= 1'b1;
    end
    else
    begin
        clk_div_1 <= clk_div_1;                 //clk_cnt 为 2 时, clk_div_1 为 0
    end
end
```

```
end

always @(negedge clk or negedge rst_n)           //clk 下降沿触发
begin
    if (!rst_n)
    begin
        clk_div_2 <= 1'b0;
    end
    else if (clk_cnt == (DIV_NUM >> 1)-1 )        //clk_cnt 为 0 时, clk_div_1 为 0
    begin
        clk_div_2 <= 1'b0;
    end
    else if (clk_cnt == DIV_NUM-1)                //clk_cnt 为 2 时, clk_div_1 为 1
    begin
        clk_div_2 <= 1'b1;
    end
    else
    begin
        clk_div_2 <= clk_div_2;
    end
end

assign clk_div = clk_div_1 | clk_div_2 ;          //clk_div_1 与 clk_div_2 产生三分频时钟

endmodule
```

2.3 小数分频

使用 Verilog 不能进行精确的小数分频，但是可以在长时间内达到的平均频率接近小数分频的频率。

举例说明使用平均频率求解小数分频的方法，例如进行 6.7 倍分频，需要保证源时钟 67 个周期的时间等于分频时钟 10 个周期的时间即可。此时需要在 67 个源时钟周期内进行 3 次 6 分频，7 次 7 分频。

具体的实现过程如下：

当进行分频时考虑 67 个源时钟周期需要进行 10 次分频，即需要 3 次每 6 个周期进行分频和 7 次每 7 个周期进行分频，即完了 6.7 倍分频。我们进行分频时需要将两个分频进行穿插交替，不能一直按照一种倍数频率进行分频，否则会造成相位抖动过大。

如何决定分频的前后顺序呢？我们根据差值进行比较来选择相应的分频倍数，具体过程如下：

- (1) 第一次进行分频， $67 - 10 \times 6 = 7 < 10$ ，第一次进行 6 分频。
- (2) 第二次进行分频，加上上次的 $7 + 7 = 14 \geq 10$ ，第二次进行 7 分频。
- (3) 第三次进行分频，加上上次的 $7 + 4 = 11 \geq 10$ ，第三次进行 7 分频。
- (4) 第四次进行分频，加上上次的 $7 + 1 = 8 < 10$ ，第四次进行 6 分频。
- (5) 第五次进行分频，加上上次的 $7 + 8 = 15 \geq 10$ ，第五次进行 7 分频。
- (6) 第六次进行分频，加上上次的 $7 + 5 = 12 \geq 10$ ，第六次进行 7 分频。

- (7) 第七次进行分频, 加上上次的 $7+2=9 < 10$, 第七次进行 6 分频。
- (8) 第八次进行分频, 加上上次的 $7+9=16 \geq 10$, 第八次进行 7 分频。
- (9) 第九次进行分频, 加上上次的 $7+6=13 \geq 10$, 第九次进行 7 分频。
- (10) 第十次进行分频, 加上上次的 $7+3=10 \geq 10$, 第十次进行 7 分频。

例:

```

module frac_div(
input          rst_n ,
input          clk,
output reg     clk_div
);
parameter     SOURCE_NUM = 67;
parameter     DEST_NUM   = 10;
parameter     SOURCE_DIV = SOURCE_NUM/DEST_NUM;
parameter     DEST_DIV   = SOURCE_DIV + 1;
parameter     DIFF_ADD   = SOURCE_NUM - SOURCE_DIV*DEST_NUM;

reg [7:0]      cnt_end_num;
reg [7:0]      div_cnt;
wire[7:0]      diff_cnt;
reg [7:0]      diff_cnt_ff;
wire           diff_cnt_en;

always @(posedge clk or negedge rst_n)
begin
    if (!rst_n)
    begin
        div_cnt  <= 'b0 ;
        clk_div  <= 1'b0 ;
    end
    else if (div_cnt == cnt_end_num)
    begin
        div_cnt  <= 'b0 ;
        clk_div  <= 1'b1 ;
    end
    else
    begin
        div_cnt  <= div_cnt + 1'b1 ;
        clk_div  <= 1'b0 ;
    end
end

assign diff_cnt_en = div_cnt == cnt_end_num ;
assign diff_cnt    = diff_cnt_ff >= DEST_NUM ?
                    diff_cnt_ff -10 + DIFF_ADD :
                    diff_cnt_ff    + DIFF_ADD ;

```

```
always @(posedge clk or negedge rst_n)
begin
    if (!rst_n)
    begin
        diff_cnt_ff <= 0 ;
    end
    else if (diff_cnt_en)
    begin
        diff_cnt_ff <= diff_cnt ;
    end
end

always @(posedge clk or negedge rst_n)
begin
    if (!rst_n)
        cnt_end_num <= SOURCE_DIV-1 ;
    else if (diff_cnt >= 10)
        cnt_end_num <= DEST_DIV-1 ;
    else
        cnt_end_num <= SOURCE_DIV-1 ;
end

endmodule
```

如果对时钟精度等要求比较高建议使用 PLL 进行分频。

第十九课 Verilog 语法_FIFO 设计

软件版本：无

操作系统：WIN10 64bit

硬件平台：适用所有系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑！

1 概述

本小节主要讲解 Verilog 语法的 FIFO 设计，需要掌握 FIFO 的基本原理，掌握同步 FIFO 和异步 FIFO 的结构。

2 同步 FIFO

FIFO 表示先入先出，它是一种存储器结构。同步 FIFO 是使用单一时钟同时进行读取和写入操作的，数据流和相关的控制逻辑在同一个时钟域内处理。

同步 FIFO 的接口设计如下：

fifo_clk	fifo 的时钟信号
fifo_rst_n	fifo 的复位信号
fifo_wren	fifo 的写有效信号
fifo_wrdata	fifo 的写数据
fifo_rden	fifo 的读有效信号
fifo_rddata	fifo 的读数据
fifo_full	fifo 的状态满信号
fifo_empty	fifo 的状态空信号
fifo_afull	fifo 的状态将满信号
fifo_aempty	fifo 的状态将空信号
fifo_room_avail	fifo 的状态可写入数信号
fifo_data_avail	fifo 的状态可读出数信号

同步 FIFO 具体的设计如下：

```

module synch_fifo#(
    parameter FIFO_AFULL_SIZE   = 1,
    parameter FIFO_AEMPTY_SIZE  = 1,
    parameter FIFO_ADDR_WIDTH   = 4,
    parameter FIFO_WIDTH        = 8,
    parameter FIFO_DEPTH        = 2**FIFO_ADDR_WIDTH
) (
    fifo_clk      ,
    fifo_rst_n    ,
    fifo_wren     ,
    fifo_wrdata   ,
    fifo_rden     ,

```

```

    fifo_rddata    ,
    fifo_full      ,
    fifo_empty     ,
    fifo_afull     ,
    fifo_aempty    ,
    fifo_room_avail,
    fifo_data_avail
);

input  wire          fifo_clk      ;
input  wire          fifo_rst_n    ;
input  wire          fifo_wren     ;
input  wire [FIFO_WIDTH-1:0]      fifo_wrdata  ;
input  wire          fifo_rden     ;
output wire [FIFO_WIDTH-1:0]      fifo_rddata  ;
output reg          fifo_full      ;
output reg          fifo_empty     ;
output wire          fifo_afull     ;
output wire          fifo_aempty    ;
output reg [FIFO_ADDR_WIDTH:0]     fifo_room_avail;
output wire [FIFO_ADDR_WIDTH:0]     fifo_data_avail;

localparam FIFO_DEPTH_MINUS1 = FIFO_DEPTH - 1;

//*****REG*****
reg [FIFO_ADDR_WIDTH-1:0] wr_ptr,wr_ptr_nxt;
reg [FIFO_ADDR_WIDTH-1:0] rd_ptr,rd_ptr_nxt;
reg [FIFO_ADDR_WIDTH:0]   num_entries,num_entries_nxt;

wire          fifo_full_nxt;
wire          fifo_empty_nxt;
wire [FIFO_ADDR_WIDTH:0] fifo_room_avail_nxt;

//write pointer control logic
//*****
always@(*)
begin
    wr_ptr_nxt = wr_ptr;
    if(fifo_wren)
    begin
        if(wr_ptr == FIFO_DEPTH_MINUS1)
            wr_ptr_nxt = 'd0;
        else
            wr_ptr_nxt = wr_ptr + 1'b1;
    end
end
end

```

```
//read pointer control logic
//*****

always@(*)
begin
    rd_ptr_nxt = rd_ptr;
    if(fifo_rden)
    begin
        if(rd_ptr == FIFO_DEPTH_MINUS1)
            rd_ptr_nxt = 'd0;
        else
            rd_ptr_nxt = rd_ptr + 1'b1;
        end
    end
end

//calculate number of occupied entries in the fifo
//*****

always@(*)
begin
    num_entries_nxt = num_entries;
    if (fifo_wren && fifo_rden)
    begin
        num_entries_nxt = num_entries;
    end
    else if (fifo_wren)
    begin
        num_entries_nxt = num_entries + 1'b1;
    end
    else if (fifo_rden)
    begin
        num_entries_nxt = num_entries - 1'b1;
    end
end

assign fifo_full_nxt      = (num_entries_nxt == FIFO_DEPTH);
assign fifo_empty_nxt     = (num_entries_nxt == 'd0);
assign fifo_data_avail    = num_entries;
assign fifo_room_avail_nxt = FIFO_DEPTH - num_entries_nxt;
assign fifo_afull         = (fifo_room_avail_nxt <= FIFO_AFULL_SIZE)? 1:0;
assign fifo_aempty        = (fifo_data_avail <= FIFO_AEMPTY_SIZE ) ? 1:0;

//the fifo output
//*****

always@(posedge fifo_clk or negedge fifo_rst_n)
begin
```

```
if (!fifo_rst_n)
begin

    wr_ptr          <= 'd0;
    rd_ptr          <= 'd0;
    num_entries     <= 'd0;
    fifo_full       <= 1'b0;
    fifo_empty      <= 1'b1;
    fifo_room_avail <= FIFO_DEPTH;

end

else
begin

    wr_ptr          <= wr_ptr_nxt;
    rd_ptr          <= rd_ptr_nxt;
    num_entries     <= num_entries_nxt;
    fifo_full       <= fifo_full_nxt;
    fifo_empty      <= fifo_empty_nxt;
    fifo_room_avail <= fifo_room_avail_nxt;

end

end

//the ram instantiation sdp
//*****

sdp_ram_sync #(
    .DATA_W          (FIFO_WIDTH      ),
    .ADDR_WIDTH      (FIFO_ADDR_WIDTH ),
    .REG_OUT          (0              )
)u_ram              (
    .data_in          (fifo_wrdata    ),
    .wraddress        (wr_ptr         ),
    .wren             (fifo_wren      ),
    .clk              (fifo_clk       ),
    .data_out         (fifo_rddata    ),
    .rdaddress        (rd_ptr         ),
    .rden             (fifo_rden      ),
    .rst_n            (fifo_rst_n     )
);

endmodule

module sdp_ram_sync#(
    parameter DATA_W    = 1,
    parameter ADDR_WIDTH= 9,
    parameter REG_OUT    = 1,
```

```
parameter U_DLY      = 1
)(
    input      [1      -1:0]  clk      ,
    input      [1      -1:0]  rst_n    ,
    input      [1      -1:0]  wren     ,
    input      [ADDR_WIDTH -1:0] wraddress ,
    input      [DATA_W    -1:0] data_in ,
    input      [1      -1:0]  rden     ,
    input      [ADDR_WIDTH -1:0] rdaddress ,
    output     [DATA_W    -1:0] data_out

);

localparam ADDR_NUM = 2**ADDR_WIDTH;

reg [DATA_W    -1:0] mem [ADDR_NUM -1:0];
reg [DATA_W    -1:0] q_tmp      ;
reg [DATA_W    -1:0] q_tmp_1d   ;

always@(posedge clk or negedge rst_n)
begin
    if(!rst_n)
        mem[wraddress] <= #U_DLY 'd0;
    else if(wren==1'b1)
    begin
        mem[wraddress] <= #U_DLY data_in;
    end
end

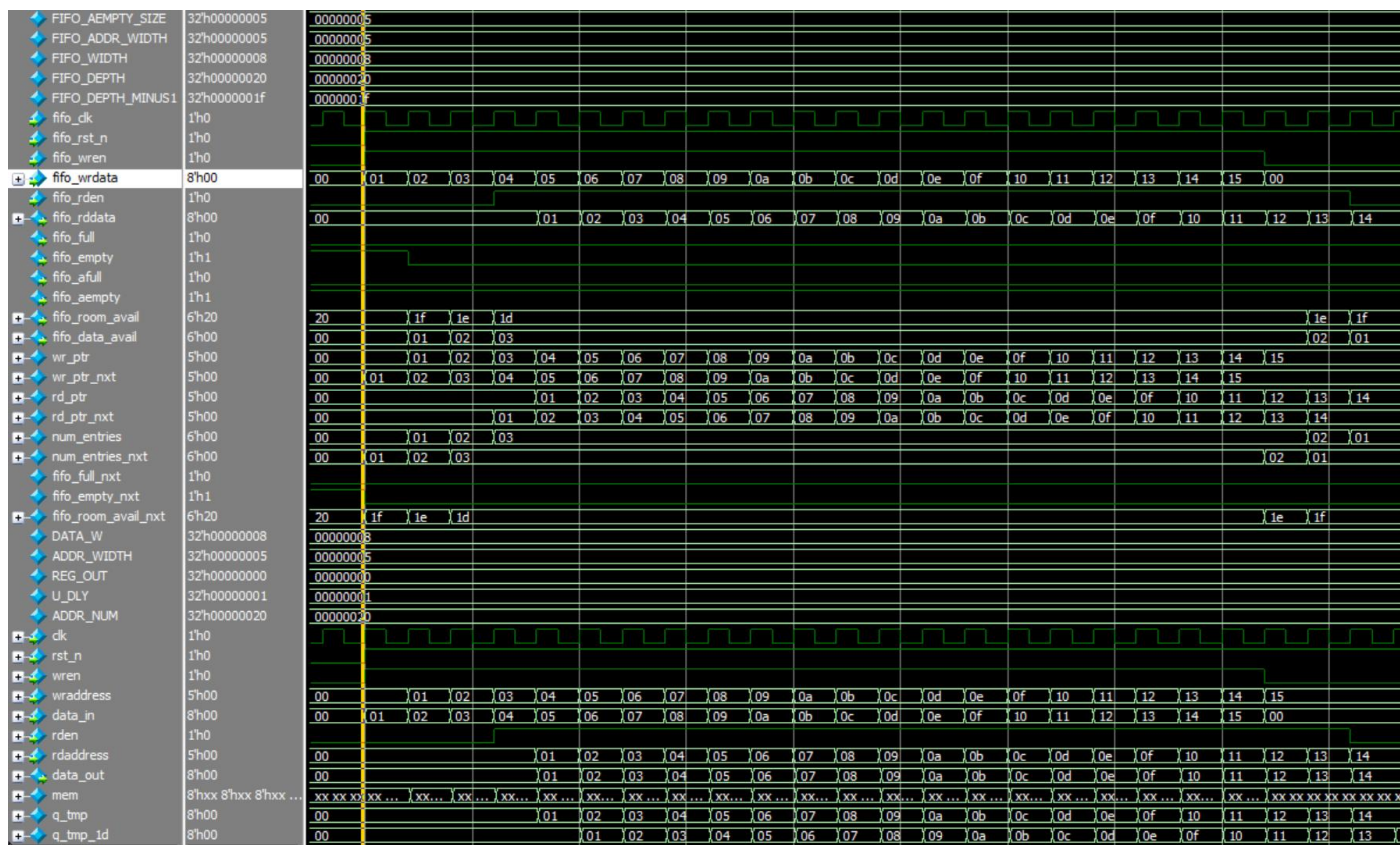
always@(posedge clk or negedge rst_n)
begin
    if(!rst_n)
        q_tmp <= #U_DLY 'd0;
    else if(rden==1'b1)
    begin
        q_tmp <= #U_DLY mem[rdaddress];
    end
end

always@(posedge clk )
begin
    q_tmp_1d <= #U_DLY q_tmp;
end

assign data_out =(REG_OUT == 1) ? q_tmp_1d : q_tmp ;
```

endmodule

仿真结果如图所示:

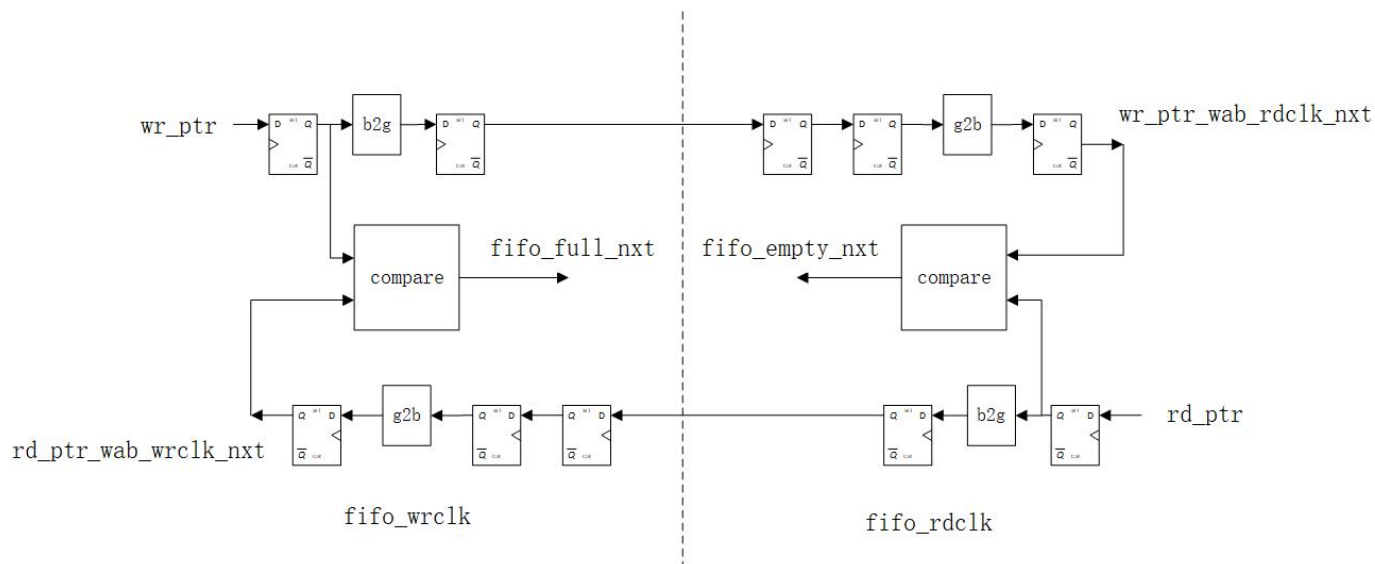


3 异步 FIFO

在实际的应用中常常需要多个时钟域的数据传递，需要使用异步 FIFO 将数据从一个时钟域传递到另一个时钟域。

从原理上来讲，同步 FIFO 和异步 FIFO 类似，但是由于异步 FIFO 与两个时钟相关，电路复杂度变高。对于异步 FIFO 的写入操作和读出操作的方式与同步 FIFO 类似，写入和读出操作都有自己的信号集，其复杂度在于产生 FIFO 的状态信号。

通过比较写入指针和读取时钟的值，可以产生空，满等状态信号，指针的跨时钟传递经过了格雷码和二进制的转换以及打两拍进行跨时钟的处理。如下图所示



写入指针首先被转换为格雷码，并被寄存到写入时钟域的触发器中。然后，再经过读取时钟域同步，打两拍，后面进行格雷码到二进制的转换电路，进行寄存。此时写入指针已被传送到读取时钟域，用于和读取时钟域的读指针进行比较，求得空状态信号。

当读取指针被传送到写入时钟域时，相对于读取时钟域中的读取指针将会有 3-4 个的时钟周期延迟。这意味着可用于写入数据的位置要比显示的可能要多三到四个。这是异步 FIFO 操作保守的一面，如此才不会有数据上溢出。当 FIFO 读取数据时，读指针数据在三到四个周期内将会跟上实际的读指针值。另外 FIFO 具有一定的深度，其中有足够多的数据被读取，因此不会影响数据传送的性能。例：

```
module asynch_fifo#(
    parameter FIFO_AFULL_SIZE  = 1,
    parameter FIFO_AEMPTY_SIZE = 1,
    parameter FIFO_PTR         = 4,
    parameter FIFO_WIDTH       = 8
    // parameter FIFO_DEPTH = 16
)(
    fifo_wrclk      ,
    fifo_wr_rst_n   ,
    fifo_wren       ,
    fifo_wrdata     ,
    reset_wrptr     ,
    fifo_rdclk      ,
    fifo_rd_rst_n   ,
    fifo_rden       ,
    fifo_rddata     ,
    reset_rdptr     ,
    fifo_full       ,
    fifo_empty      ,
    fifo_afull      ,
    fifo_aempty     ,
    fifo_room_avail,
    fifo_data_avail
);
input wire          fifo_wrclk      ;
input wire          fifo_wr_rst_n   ;
input wire          fifo_wren       ;
input wire [FIFO_WIDTH-1:0] fifo_wrdata ;
input wire          reset_wrptr     ;
input wire          fifo_rdclk      ;
input wire          fifo_rd_rst_n   ;
input wire          fifo_rden       ;
output wire [FIFO_WIDTH-1:0] fifo_rddata ;
input wire          reset_rdptr     ;
output reg          fifo_full       ;
output reg          fifo_empty      ;
output wire          fifo_afull      ;
output wire          fifo_aempty     ;
```

```

output reg [FIFO_PTR:0]      fifo_room_avail;
output reg [FIFO_PTR:0]      fifo_data_avail;

localparam FIFO_DEPTH        = (1<<FIFO_PTR);
localparam FIFO_TWICEDEPTH_MINUS1 = 2*FIFO_DEPTH - 1;

//*****REG*****

reg [FIFO_PTR:0] wr_ptr_wab,wr_ptr_wab_nxt;//extra wraparound bit
wire [FIFO_PTR:0] fifo_room_avail_nxt ;
wire fifo_full_nxt ;
wire [FIFO_PTR:0] wr_ptr ;//write ptr without wraparound bit
reg [FIFO_PTR:0] rd_ptr_wab,rd_ptr_wab_nxt;//extra wraparound bit
wire [FIFO_PTR:0] fifo_data_avail_nxt ;
wire fifo_empty_nxt ;
wire [FIFO_PTR:0] rd_ptr ;//read ptr without wraparound bit

reg [FIFO_PTR:0] wr_ptr_wab_gray ;
wire [FIFO_PTR:0] wr_ptr_wab_gray_nxt ;
reg [FIFO_PTR:0] wr_ptr_wab_gray_sync1 ;
reg [FIFO_PTR:0] wr_ptr_wab_gray_sync2 ;
reg [FIFO_PTR:0] wr_ptr_wab_rdcclk ;
wire [FIFO_PTR:0] wr_ptr_wab_rdcclk_nxt ;

reg [FIFO_PTR:0] rd_ptr_wab_gray ;
wire [FIFO_PTR:0] rd_ptr_wab_gray_nxt ;
reg [FIFO_PTR:0] rd_ptr_wab_gray_sync1 ;
reg [FIFO_PTR:0] rd_ptr_wab_gray_sync2 ;
reg [FIFO_PTR:0] rd_ptr_wab_wrclk ;
wire [FIFO_PTR:0] rd_ptr_wab_wrclk_nxt ;

//write pointer control logic
//*****

always@(*)
begin
    wr_ptr_wab_nxt = wr_ptr_wab;
    if(reset_wrptr)
    begin
        wr_ptr_wab_nxt = 'd0;
    end
    else if(fifo_wren&&(wr_ptr_wab == FIFO_TWICEDEPTH_MINUS1))
    begin
        wr_ptr_wab_nxt = 'd0;
    end
    else if(fifo_wren)
    begin
        wr_ptr_wab_nxt = wr_ptr_wab + 1'b1;
    end
end

```

```
end

always@(posedge fifo_wrclk or negedge fifo_wr_rst_n)
begin
    if(!fifo_wr_rst_n)
    begin
        wr_ptr_wab          <= 'd0;
    end
    else
    begin
        wr_ptr_wab          <= wr_ptr_wab_nxt;
    end
end

//convert the binary wr_ptr to gray,flop it,and then pass it to read domain
//*****
binary_to_gray#(.PTR(FIFO_PTR)) u_binary_to_gray_wr
(
    .binary_value    (wr_ptr_wab_nxt          ),
    .gray_value      (wr_ptr_wab_gray_nxt      )
);

always@(posedge fifo_wrclk or negedge fifo_wr_rst_n)
begin
    if(!fifo_wr_rst_n)
    begin
        wr_ptr_wab_gray     <= 'd0;
    end
    else
    begin
        wr_ptr_wab_gray     <= wr_ptr_wab_gray_nxt;
    end
end

//synchronize wr_ptr_wab_gray into read clock domain
//*****
always@(posedge fifo_rdclk or negedge fifo_rd_rst_n)
begin
    if(!fifo_rd_rst_n)
    begin
        wr_ptr_wab_gray_sync1    <= 'd0;
        wr_ptr_wab_gray_sync2    <= 'd0;
    end
    else
    begin
        wr_ptr_wab_gray_sync1    <= wr_ptr_wab_gray;
    end
end
```

```
        wr_ptr_wab_gray_sync2    <= wr_ptr_wab_gray_sync1;

    end

end

//convert wr_ptr_wab_gray_sync2 back to binary form
//*****
gray_to_binary#(.PTR(FIFO_PTR)) u_gray_to_binary_wr
(
    .gray_value    (wr_ptr_wab_gray_sync2 ),
    .binary_value  (wr_ptr_wab_rdclk_nxt  )
);

always@(posedge fifo_rdclk or negedge fifo_rd_rst_n)
begin
    if(!fifo_rd_rst_n)
    begin
        wr_ptr_wab_rdclk    <= 'd0;
    end
    else
    begin
        wr_ptr_wab_rdclk    <= wr_ptr_wab_rdclk_nxt;
    end
end

//read pointer control logic
//*****
always@(*)
begin
    rd_ptr_wab_nxt = rd_ptr_wab;
    if(reset_rdptra)
    begin
        rd_ptr_wab_nxt = 'd0;
    end
    else if(fifo_rden && (rd_ptr_wab == FIFO_TWICEDEPTH_MINUS1))
    begin
        rd_ptr_wab_nxt = 'd0;
    end
    else if(fifo_rden)
    begin
        rd_ptr_wab_nxt = rd_ptr_wab + 1'b1;
    end
end

always@(posedge fifo_rdclk or negedge fifo_rd_rst_n)
begin
```

```
    if(!fifo_rd_rst_n)
    begin
        rd_ptr_wab          <= 'd0;
    end
    else
    begin
        rd_ptr_wab          <= rd_ptr_wab_nxt;
    end
end

//convert the binary rd_ptr to gray and then pass it to write clock domain
//*****
binary_to_gray#(.PTR(FIFO_PTR)) u_binary_to_gray_rd
(
    .binary_value  (rd_ptr_wab_nxt          ),
    .gray_value    (rd_ptr_wab_gray_nxt     )
);

always@(posedge fifo_rdclk or negedge fifo_rd_rst_n)
begin
    if(!fifo_rd_rst_n)
    begin
        rd_ptr_wab_gray     <= 'd0;
    end
    else
    begin
        rd_ptr_wab_gray     <= rd_ptr_wab_gray_nxt;
    end
end

//synchronize rd_ptr_wab_gray into write clock domain
//*****
always@(posedge fifo_wrclk or negedge fifo_wr_rst_n)
begin
    if(!fifo_wr_rst_n)
    begin
        rd_ptr_wab_gray_sync1    <= 'd0;
        rd_ptr_wab_gray_sync2    <= 'd0;
    end
    else
    begin
        rd_ptr_wab_gray_sync1    <= rd_ptr_wab_gray;
        rd_ptr_wab_gray_sync2    <= rd_ptr_wab_gray_sync1;
    end
end
end
```

```
//convert rd_ptr_wab_gray_sync2 back to binary form
//*****
gray_to_binary#(.PTR(FIFO_PTR)) u_gray_to_binary_rd
(
    .gray_value      (rd_ptr_wab_gray_sync2 ),
    .binary_value    (rd_ptr_wab_wrclk_nxt  )
);

always@(posedge fifo_rdclk or negedge fifo_rd_rst_n)
begin
    if(!fifo_rd_rst_n)
    begin
        rd_ptr_wab_wrclk      <= 'd0;
    end
    else
    begin
        rd_ptr_wab_wrclk      <= rd_ptr_wab_wrclk_nxt;
    end
end

assign wr_ptr = wr_ptr_wab[FIFO_PTR-1:0];
assign rd_ptr = rd_ptr_wab[FIFO_PTR-1:0];

//the ram instantiation sdp
//*****

sdp_ram #(
    .DATA_W      (FIFO_WIDTH  ),
    .ADDR_WIDTH  (FIFO_PTR    ),
    .REG_OUT     (0           )
)u_ram(
    .data_in      (fifo_wrdata ),
    .wraddress     (wr_ptr      ),
    .wren          (fifo_wren   ),
    .clk_w         (fifo_wrclk  ),
    .data_out      (fifo_rddata ),
    .rdaddress     (rd_ptr      ),
    .rden          (fifo_rden   ),
    .clk_r         (fifo_rdclk  ),
    .rst_n_w       (fifo_wr_rst_n ),
    .rst_n_r       (fifo_rd_rst_n )
);
```

```

//generate fifo_full:pointers equal,but the warp around bits are different
//*****

assign fifo_full_nxt      =  (wr_ptr_wab_nxt[FIFO_PTR]      != rd_ptr_wab_wrclk_nxt[FIFO_PTR]) &&
                             (wr_ptr_wab_nxt[FIFO_PTR-1:0] == rd_ptr_wab_wrclk_nxt[FIFO_PTR-1:0]);

assign fifo_room_avail_nxt =  (wr_ptr_wab[FIFO_PTR] == rd_ptr_wab_wrclk[FIFO_PTR])?
                             (FIFO_DEPTH - (wr_ptr_wab[FIFO_PTR-1:0] - rd_ptr_wab_wrclk[FIFO_PTR-1:0])):
                             (rd_ptr_wab_wrclk[FIFO_PTR-1:0] - wr_ptr_wab[FIFO_PTR-1:0]);

//generate fifo_empty:pointers are equal including the warp around bits
//*****

assign fifo_empty_nxt      =  (rd_ptr_wab_nxt[FIFO_PTR:0] == wr_ptr_wab_rdclk_nxt[FIFO_PTR:0]);

assign fifo_data_avail_nxt =  (rd_ptr_wab[FIFO_PTR] == wr_ptr_wab_rdclk[FIFO_PTR])?
                             (wr_ptr_wab_rdclk[FIFO_PTR-1:0] - rd_ptr_wab[FIFO_PTR-1:0]):
                             (FIFO_DEPTH - (rd_ptr_wab[FIFO_PTR-1:0] - wr_ptr_wab_rdclk[FIFO_PTR-1:0]));

assign fifo_afull          = (fifo_room_avail_nxt <= FIFO_AFULL_SIZE)? 1:0;
assign fifo_aempty         = (fifo_data_avail_nxt <= FIFO_AEMPTY_SIZE ) ? 1:0;

always@(posedge fifo_wrclk or negedge fifo_wr_rst_n)
begin
    if(!fifo_wr_rst_n)
    begin
        fifo_full          <= 1'b0;
        fifo_room_avail    <= 'd0;
    end
    else
    begin
        fifo_full          <= fifo_full_nxt;
        fifo_room_avail    <= fifo_room_avail_nxt;
    end
end

always@(posedge fifo_rdclk or negedge fifo_rd_rst_n)
begin
    if(!fifo_rd_rst_n)
    begin
        fifo_empty         <= 1'b1;
        fifo_data_avail    <= 'd0;
    end
    else
    begin
        fifo_empty         <= fifo_empty_nxt;
        fifo_data_avail    <= fifo_data_avail_nxt;
    end
end

```

```
end

end

endmodule

module binary_to_gray#(
    parameter PTR    = 4    //"1","0"
)(
    binary_value    ,
    gray_value
);

input  wire [PTR:0]  binary_value ;
output wire [PTR:0]  gray_value   ;

generate
    genvar i;
    for( i = 0 ; i < PTR ; i = i + 1 )
    begin
        assign gray_value[i] = binary_value[i] ^ binary_value[ i + 1 ];
    end
endgenerate

assign gray_value[PTR] = binary_value[PTR];

endmodule

module gray_to_binary#(
    parameter PTR    = 4    //"1","0"
)(
    binary_value    ,
    gray_value
);

input  wire [PTR:0]  gray_value   ;
output wire [PTR:0]  binary_value ;

generate
    genvar i;
    for( i = 0 ; i < PTR ; i = i + 1 )
    begin
        assign binary_value[i] = binary_value[i + 1] ^ gray_value[i];
    end
endgenerate
```

```
end

endgenerate

assign binary_value[PTR] = gray_value[PTR];

endmodule

module sdp_ram#(
    parameter DATA_W    = 1,
    parameter ADDR_WIDTH= 9,
    parameter REG_OUT    = 1,
    parameter U_DLY      = 1
)(
    input    [1          -1:0]  clk_w      ,
    input    [1          -1:0]  rst_n_w   ,
    input    [1          -1:0]  wren      ,
    input    [ADDR_WIDTH -1:0]  wraddress ,
    input    [DATA_W     -1:0]  data_in   ,
    input    [1          -1:0]  clk_r      ,
    input    [1          -1:0]  rst_n_r   ,
    input    [1          -1:0]  rden      ,
    input    [ADDR_WIDTH -1:0]  rdaddress ,
    output   [DATA_W     -1:0]  data_out

);

localparam  ADDR_NUM  = 2**ADDR_WIDTH;

reg [DATA_W -1:0] mem [ADDR_NUM -1:0];
reg [DATA_W -1:0] q_tmp    ;
reg [DATA_W -1:0] q_tmp_1d ;

always@(posedge clk_w or negedge rst_n_w)
begin
    if(!rst_n_w)
        mem[wraddress] <= #U_DLY 'd0;
    else if(wren==1'b1)
    begin
        mem[wraddress] <= #U_DLY data_in;
    end
end

always@(posedge clk_r or negedge rst_n_r)
begin
    if(!rst_n_r)
```

```

        q_tmp <= #U_DLY 'd0;

    else if(rden==1'b1)

        begin

            q_tmp <= #U_DLY mem[raddress];

        end

    end

end

always@(posedge clk_r )

begin

    q_tmp_1d <= #U_DLY q_tmp;

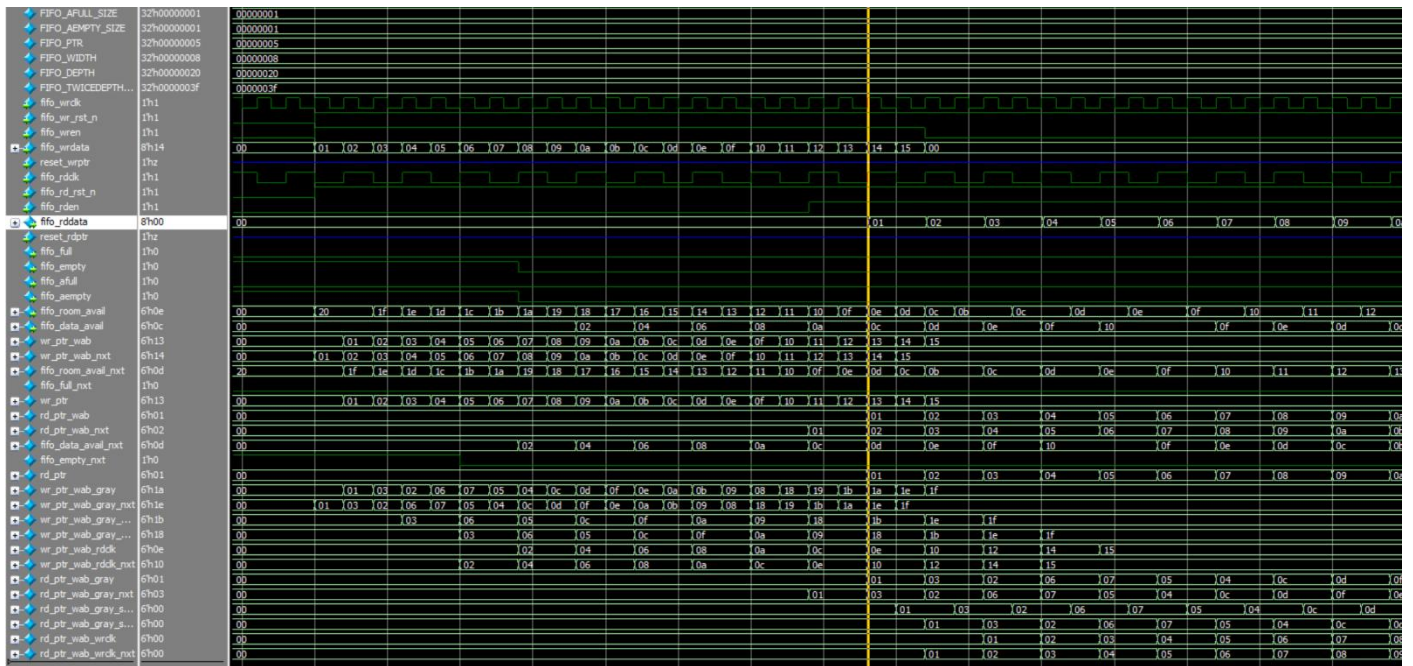
end

assign data_out =(REG_OUT == 1) ? q_tmp_1d : q_tmp ;

endmodule

```

仿真结果如图所示:



第二十课 Verilog 语法_低功耗设计

软件版本：无

操作系统：WIN10 64bit

硬件平台：适用所有系列 FPGA

登录“米联客”FPGA 社区-www.uisrc.com 视频课程、答疑解惑！

1 概述

本小节讲解 Verilog 语法的低功耗设计，需要掌握几种低功耗设计的方法。

2 低功耗简介

低功耗技术在当今得到越来越广泛的发展，在你的身边存在着许多的消费电子产品，无一不含有低功耗技术的成果。比如现在的智能手机，体积、重量都比之前有了很大提升，低功耗技术使得手机电池的续航能力进一步增强；无线蓝牙耳机，做到非常小的体积内，容纳小容量电池，也是得益于低功耗技术的发展。

目前的低功耗设计主要从芯片设计和系统设计两个方面考虑。随着半导体工艺的飞速发展和芯片工作频率的提高，芯片的功耗迅速增加，而功耗增加又将导致芯片发热量的增大和可靠性的下降。因此，功耗已经成为深亚微米集成电路设计中的一个重要考虑因素。为了使产品更具竞争力，工业界对芯片设计的要求已从单纯追求高性能、小面积转为对性能、面积、功耗的综合要求。而微处理器作为数字系统的核心部件，其低功耗设计对降低整个系统的功耗具有重要的意义。

3 常用的低功耗设计技术

（1）动态电压调节

动态功耗与工作电压的平方成正比，功耗将随着工作电压的降低以二次方的速度降低，因此降低工作电压是降低功耗的有力措施。但是，仅仅降低工作电压会导致传播延迟加大，执行时间变长。然而，系统负载是随时间变化的，因此并不需要微处理器所有时刻都保持高性能。动态电压调节技术降低功耗的主要思路是根据芯片工作状态改变功耗管理模式，从而在保证性能的基础上降低功耗。在不同模式下，工作电压可以进行调整。为了精确地控制电压，需要采用电压调度模块来实时改变工作电压，电压调度模块通过分析当前和过去状态下系统工作情况的不同来预测电路的工作负荷。

（2）门控时钟

在微处理器中，很大一部分功耗来自时钟。时钟是惟一在所有时间都翻转的信号，而且很多情况下引起不必要的门的翻转，因此降低时钟的开关活动性将对降低整个系统的功耗产生很大的影响。门控时钟包括门控逻辑模块时钟和门控寄存器时钟。门控逻辑模块时钟对时钟网络进行划分，如果在当前的时钟周期内，系统没有用到某些逻辑模块，则暂时切断这些模块的时钟信号，从而明显地降低开关功耗。采用“与”门实现的时钟控制电路。门控寄存器时钟的原理是当寄存器保持数据时，关闭寄存器时钟，以降低功耗。然而，门控时钟易引起毛刺，必须对信号的时序加以严格限制，并对其进行仔细的时序验证。

（3）可变频率时钟

根据系统性能要求,配置适当的时钟频率以避免不必要的功耗。门控时钟实际上是可变频率时钟的一种极限情况(即只有零和最高频率两种值),因此,可变频率时钟比门控时钟技术更加有效,但需要系统内嵌时钟产生模块 PLL,增加了设计复杂度。

(4) 并行结构

并行结构的原理是通过牺牲面积来降低功耗。将一个功能模块复制为 $n(n \geq 2)$ 个相同的模块,这些模块并行计算后通过数据选择器选择输出,采用二分频的并行结构。

(5) 流水线技术

流水线技术本质上也是一种并行。把某一功能模块分成 n 个阶段进行流水作业,每个阶段由一个子模块来完成,在子模块之间插入寄存器。若工作频率不变,对某个模块的速度要求仅为原来的 $1/n$,则工作电压可以降低为原来的 $1/n$,电容的变化不大,功耗可降低为原来的 $1/n^2$,面积基本不变,但增加了控制的复杂度。

(6) 低功耗单元

设计低功耗单元是降低功耗的一个重要方法,包括调整单元尺寸、改进电路结构和版图设计。用户可以根据负载电容和电路延时的需要选择不同尺寸的电路来实现,这样会导致不同的功耗,因此可以根据需要设计不同尺寸的单元。同时,为常用的单元选择低功耗的实现结构,如触发器、锁存器和数据选择器等。

(7) 低功耗状态机编码

状态机编码对信号的活动性具有重要影响,通过合理选择状态机状态的编码方法,减少状态切换时电路的翻转,可以降低状态机的功耗。其原则是:对于频繁切换的相邻状态,尽量采用相邻编码。例如:Gray 码在任何两个连续的编码之间只有一位数值不同,在设计计数器时,使用 Gray 码取代二进制码,则计数器的改变次数几乎减少一半,显著降低了功耗;在访问相邻的地址空间时,其跳变次数显著减少,有效地降低了总线功耗。

(8) Cache 的低功耗设计

作为现代微处理器中的重要部件,Cache 的功耗约占整个芯片功耗的 30%~60%,因此设计高性能、低功耗的 Cache 结构,对降低微处理器的功耗有明显作用。Cache 低功耗设计的关键在于降低失效率,减少不必要的操作。通常用来降低 Cache 功耗的方法有以下两种:一种是从存储器的结构出发,设计低功耗的存储器,例如采用基于 CAM 的 Cache 结构;另一种是通过减少对 Cache 的访问次数来降低功耗。

4 Verilog 中使用的低功耗设计方法

4.1 并行设计

在 Verilog 中对同一功能模块进行并行设计,可以一定程度上降低功耗,资源变多了,速度变快了。

并行处理,可以同时处理多条执行语句,使执行效率变高。在满足工作要求的情况下,采用并行处理,可降低系统工作频率,减少功耗。

比如设计乘法器时,在 200MHz 的工作频率下,需要一个 DSP 模块进行计算可以满足设计要求,如果使用 100MHz 的工作频率,那么需要 2 个 DSP 模块进行计算可以满足设计的要求,同时功耗降低了。

4.2 流水线设计

在 Verilog 中对同一功能模块进行流水线设计,可以一定程度上降低功耗,资源变多了,速度变快了。类似于并行设计,多级流水线技术可以将效率提升,同时需要更多地资源消耗。

流水线设计可以将一个较长的路径分成 N 级流水线。路径长度缩短为原来路径长度的 $1/N$ 。此时如果时钟频率不变, 则在一个周期内, 只需要对电容 C/N 进行充放电, 而不是对原来的电容 C 进行充放电。因此在相同的频率要求下, 可以采用较低的电源电压来驱动系统, 使功耗降低。

4.3 资源共用

当设计中存在一些功能相同的运算模块, 可以通过资源共享的方法避免同一个模块多次出现, 减少资源的消耗。

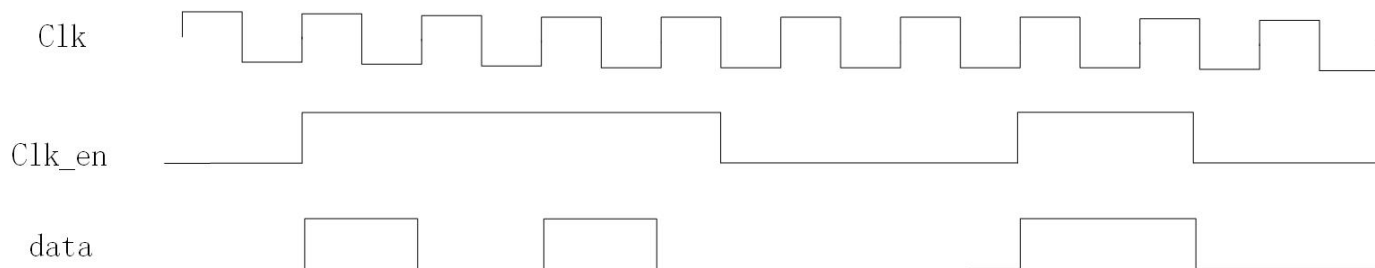
4.4 状态编码

对于一些变化频繁的信号, 翻转率较高, 功耗相对较大。可以利用状态编码的方式来降低器件的开关次数, 减少整体功耗。

例如进行状态机设计时, 使用格雷码代替二进制编码, 状态机切换前后的状态编码如果只有 1bit 的差异, 就会减少翻转率, 降低功耗。

4.5 门控时钟

时钟信号是变化频繁的信号, 翻转率较高, 功耗相对较大。如果在功能允许的情况下, 适时关闭时钟, 使得时钟可以控制, 用时开启, 不用时关闭, 可以降低整体的功耗。



直接将时钟使能控制信号与时钟做"与"逻辑, 就可以实现门控时钟。该方法缺点很明显。由于时序或抖动的原因, 时钟使能信号与时钟进行"与"逻辑后, 容易产生毛刺, 会对数字电路产生严重影响。解决这个问题的方法就是使用锁存器, 在时钟下降沿对时钟使能信号进行锁存, 并保持一个时钟周期内不变。锁存后的信号再与时钟进行"与"逻辑操作, 可将门控时钟中的毛刺消除掉。